Live Typing UX Research — Batch 1 (14/04/26)

Batch 1 — all 17 paired topics captured 14/04/26

Generated 2026-04-14 · 17 topic“s”

Contents

Batch 1 — Summary . . . 8
Topic: `batch-vs-chunked-inference-accuracy` . . . 10
Prompt . . . 10
Does end-of-utterance batch inference give better accuracy than chunked streaming, or is it just different frontend engineering? . . . 10
The question . . . 10
Related . . . 10
Output . . . 10
End-of-utterance batch inference is genuinely more accurate than chunked streaming — it’s not just frontend engineering . . . 10
TL;DR . . . 11
Background . . . 11
Why batch is more accurate: the attention-mechanism argument . . . 11
Why chunked streaming is less accurate beyond the attention argument . . . 13
How big is the gap in practice? . . . 13
What frontend engineering can and cannot fix . . . 14
What the chunked-offline middle ground actually buys you . . . 15
Specific to your Parakeet + Handy setup . . . 15
Recommendation for the workspace’s “ideal UI” spec . . . 16
Caveats . . . 16
References . . . 16
Topic: `cursor-dictation-vs-clipboard-stt-adoption` . . . 18
Prompt . . . 18
Cursor-level dictation vs record-then-paste STT — who loves which, and where . . . 18
The question . . . 18
Output . . . 18
Cursor-level dictation vs record-then-paste STT — adoption patterns across industries, users, and operating systems . . . 18
TL;DR . . . 18
Background — the two modes . . . 19
The best-loved tools, by mode . . . 19
Which industries and users favour which mode . . . 21
Popularity across operating systems . . . 23
Why the split exists — design pressures . . . 23
Where the two modes land — verdict . . . 24
References . . . 24
Topic: `custom-vocabulary-in-stt` . . . 26
Prompt . . . 26
Custom vocabulary in transcription tools — how is it actually implemented? . . . 26
The question . . . 26
Output . . . 26
Custom vocabulary in STT — what’s actually happening under the hood, and how big the list can get . . . 26
TL;DR . . . 26
Background — why this feature exists at all . . . 27
The five mechanisms, in depth . . . 28

How a typical “custom words” UI maps to these mechanisms . . . 31
Is it a frontend feature? . . . 32
Is it system-prompted? . . . 33
When degradation actually sets in . . . 33
Recommendations for this workspace’s use case (live desktop dictation) . . . 34
References . . . 34
Topic: `dictation-formatting-built-in-vs-supplementary` . . . 35
Prompt . . . 35
Dictation formatting features — built-in model support vs supplementary post-processing . . . 35
The question . . . 35
Follow-up — 2026-04-14 . . . 35
Output . . . 35
Dictation formatting features — built-in vs supplementary, across local and cloud models . . . 36
TL;DR . . . 36
The four formatting requirements, restated precisely . . . 36
Cloud models — what’s built in . . . 37
Local models — what’s built in . . . 39
Supplementary models — when you need a post-processor . . . 41
Putting it together — what a “full coverage” pipeline looks like . . . 42
Bottom line . . . 43
Follow-up — 2026-04-14 . . . 43
Topic: `focus-loss-during-dictation` . . . 47
Prompt . . . 47
Focus loss during dictation . . . 47
The question . . . 47
Output . . . 47
Handling focus loss between dictation start and transcript arrival . . . 47
TL;DR . . . 47
Background . . . 48
The patterns . . . 48
What’s actually implementable on Wayland . . . 49
A recommended layered default . . . 50
Verification . . . 50
Caveats / things that can go wrong . . . 51
References . . . 51
Topic: `handy-inference-vs-typing-delay` . . . 52
Prompt . . . 52
When Handy prints the transcript over 10–20 seconds, is that real-time inference or an artificial frontend delay? . . . 52
The question . . . 52
Related . . . 52
Follow-up questions . . . 52
Output . . . 52
Handy’s 10–20 second print delay is almost entirely keystroke-injection pacing, not inference . . . 53
TL;DR . . . 53
Background . . . 53

Stage 1: the inference itself . . . 53
Stage 2: the injection — this is almost certainly the real culprit . . . 54
So is it “artificial”? . . . 56
What you can do about it . . . 56
Verification — quick test you can run . . . 57
References . . . 57
Caveats . . . 58
Follow-up — 2026-04-14: one-shot automatic injection on Wayland . . . 58
Follow-up — 2026-04-14: Handy’s “Paste Method” setting, decoded . . . 60
Topic: `inference-cadence-and-sentence-entry` . . . 63
Prompt . . . 63
Inference cadence and sentence entry for pause-for-thought dictators . . . 63
The question . . . 63
Output . . . 63
Inference cadence and sentence entry — finding the UX sweet spot for pause-for-thought dictators . . . 63
TL;DR . . . 63
Background . . . 64
The three intervals, with numbers . . . 64
Why Deepgram feels right . . . 65
Sentence and paragraph boundary detection . . . 66
The UI friction points, explicitly named . . . 66
Recommendation . . . 67
References . . . 67
Caveats . . . 67
Topic: `live-typing-models-saas-and-local` . . . 68
Prompt . . . 68
Leading STT models for live typing — SaaS/API and locally runnable . . . 68
The question . . . 68
Output . . . 68
Leading STT models for live typing — SaaS and local . . . 68
TL;DR . . . 68
Background . . . 69
SaaS / API — the streaming leaders . . . 69
Locally runnable — what Handy ships . . . 70
Architecture commonalities and differences . . . 71
Recommendation (for Daniel’s prototype) . . . 72
References . . . 73
Topic: `local-stt-builtin-vad` . . . 74
Prompt . . . 74
Local STT engines with built-in VAD (or equivalent silence-handling architecture) . . . 74
The question . . . 74
Output . . . 74
Local STT engines and silence handling — which ones have built-in VAD, which need it bolted on, and which sidestep the problem architecturally . . . 74
TL;DR . . . 74
Background — what “silence hallucination” is and why it matters for dictation . . . 75
Engines that handle silence *architecturally* . . . 76
Engines that ship with VAD bundled but *optional* . . . 77

Engines that do NOT handle silence — will hallucinate without external VAD 78
The alternative architecture — push-to-talk makes VAD almost unnecessary 79
Quick reference table 79
Recommendation for this workspace 80
References 80
Topic: `local-stt-inference-engines-gpu` 82
Prompt 82
Local STT inference engines and GPU acceleration (NVIDIA vs AMD) 82
The question 82
Output 82
Local STT inference engines — GPU acceleration on NVIDIA vs AMD 82
TL;DR 82
Background 83
The engines — what they are and what they accelerate on 83
The arch split — engine × vendor × backend 85
Commonalities and points of difference 86
ROCm reality check on consumer AMD 86
Recommendation 86
References 87
Topic: `partial-transcript-rewriting` 88
Prompt 88
What is the dynamic-rewriting display in tools like Deepgram called? 88
The question 88
Related 88
Output 88
The dynamic-rewriting display: interim results, stabilization, and where the work happens 88
TL;DR 89
Background 89
What the methodology is called 90
Frontend or backend? 91
How a single utterance looks on the wire 92
Why it’s done this way 93
Implications for an “ideal” desktop live-typing UI 93
Caveats 93
References 94
Topic: `pause-tolerant-dictation-profile-and-stack` 95
Prompt 95
Pause-tolerant dictation — codify this user profile and recommend an STT stack 95
User profile 95
What the answer should contain 95
Output 95
The pause-tolerant prompt-dictator — a codified user profile and the STT stack that matches it 96
TL;DR 96
The user profile, codified 96
Derived specification 97
Recommended cloud stack 99
Recommended local stack 101

Recommended frontends that match this UX . . . 102
Recommended architecture — the spec . . . 103
Anti-patterns for this user . . . 104
References . . . 105
Caveats . . . 106
Topic: `streaming-vs-batch-injection` . . . 107
Prompt . . . 107
Streaming injection vs batch transcription on stop . . . 107
The question . . . 107
Output . . . 107
Streaming injection vs utterance-final injection in live dictation . . . 107
TL;DR . . . 107
Background . . . 108
The terminology . . . 108
What you're actually using vs what you're describing . . . 109
Tradeoffs between the two . . . 109
A note on Parakeet specifically . . . 110
Recommended framing for the workspace's "ideal UI" spec . . . 111
Caveats / things that can go wrong . . . 111
References . . . 111
Topic: `stt-enthusiastic-industry-adoption` . . . 112
Prompt . . . 112
Industries that adopted speech-to-text enthusiastically through recognition of its effectiveness . . . 112
The question . . . 112
Output . . . 112
Industries that adopted STT enthusiastically through sheer recognition of effectiveness . . . 112
TL;DR . . . 112
Background . . . 112
Strongest cases . . . 113
Borderline and mandated-origin cases . . . 114
The clearest answer . . . 115
References . . . 115
Topic: `vad-for-live-typing` . . . 116
Prompt . . . 116
VAD (voice activity detection) for live typing . . . 116
The question . . . 116
Output . . . 116
VAD for live typing: what's native, what's bolted on, and how to wire them together . 116
TL;DR . . . 116
Background — why VAD matters specifically for live typing . . . 117
What "native VAD" actually means in different ASR systems . . . 117
Pairing a separate VAD with an ASR — the standard pipelines . . . 118
Tradeoffs and tuning . . . 120
Recommendation for this workspace . . . 121
References . . . 121
Topic: `voice-dictation-hotkey-count-tradeoffs` . . . 122
Prompt . . . 122

Hotkey count tradeoffs for voice dictation control (single key, macro pads) 122
The question 122
Output 122
Hotkey count tradeoffs for voice dictation: from single toggle to a four-key macro pad 122
TL;DR 123
Background 123
The progression by key count 123
Tradeoffs at a glance 125
Recommendation 125
Caveats 125
References 126
Topic: `whisper-vs-streaming-asr-for-dictation` 127
Prompt 127
Why Whisper isn’t ideal for live dictation, and how live STT models rewrite on the fly 127
The question 127
Output 127
Why Whisper isn’t built for live dictation, and how streaming STT models rewrite on the fly 127
TL;DR 127
Background 128
Why Whisper is structurally offline 128
What live-suited models do differently 129
How filler-word removal works without a multimodal LLM 131
Where this leaves Whisper 134
Caveats 134
References 135

Batch 1 — Summary

Slug	Prompt
`batch-vs-chunked-inference-accuracy`	Does end-of-utterance batch inference give better accuracy than chunked streaming, or is it just different frontend engineering?
`cursor-dictation-vs-clipboard-stt-adoption`	Cursor-level dictation vs record-then-paste STT — who loves which, and where
`custom-vocabulary-in-stt`	Custom vocabulary in transcription tools — how is it actually implemented?
`dictation-formatting-built-in-vs-supplementary`	Dictation formatting features — built-in model support vs supplementary post-processing
`focus-loss-during-dictation`	Focus loss during dictation
`handy-inference-vs-typing-delay`	When Handy prints the transcript over 10–20 seconds, is that real-time inference or an artificial frontend delay?
`inference-cadence-and-sentence-entry`	Inference cadence and sentence entry for pause-for-thought dictators
`live-typing-models-saas-and-local`	Leading STT models for live typing — SaaS/API and locally runnable
`local-stt-builtin-vad`	Local STT engines with built-in VAD (or equivalent silence-handling architecture)
`local-stt-inference-engines-gpu`	Local STT inference engines and GPU acceleration (NVIDIA vs AMD)
`partial-transcript-rewriting`	What is the dynamic-rewriting display in tools like Deepgram called?
`pause-tolerant-dictation-profile-and-stack`	Pause-tolerant dictation — codify this user profile and recommend an STT stack
`streaming-vs-batch-injection`	Streaming injection vs batch transcription on stop
`stt-enthusiastic-industry-adoption`	Industries that adopted speech-to-text enthusiastically through recognition of its effectiveness

Slug	Prompt
`vad-for-live-typing`	VAD (voice activity detection) for live typing
`voice-dictation-hotkey-count-tradeoffs`	Hotkey count tradeoffs for voice dictation control (single key, macro pads)
`whisper-vs-streaming-asr-for-dictation`	Why Whisper isn’t ideal for live dictation, and how live STT models rewrite on the fly

Topic: `batch-vs-chunked-inference-accuracy`

Prompt

Does end-of-utterance batch inference give better accuracy than chunked streaming, or is it just different frontend engineering?

Asked: 14/04/26 **Slug:** `batch-vs-chunked-inference-accuracy` **Response:** `ideas/batch-vs-chunked-inference-accuracy.md`

The question

Is there any meaningful difference in accuracy when you have a model that does the inference at the end — in the pattern I've described for my local Parakeet with Handy, whereby you wait until you're finished dictating and then it does it all in one go?

In terms of the accuracy and the attention mechanism, is it going to struggle more than if you use that chunked approach, or is it just different frontend engineering?

Related

- `streaming-vs-batch-injection` — separates the recognition-mode axis from the injection-mode axis.
- `whisper-vs-streaming-asr-for-dictation` — why offline-trained models like Whisper resist being made streaming.

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

End-of-utterance batch inference is genuinely more accurate than chunked streaming — it's not just frontend engineering

Question: `../questions/batch-vs-chunked-inference-accuracy.md` **Written:** 14/04/26 **Stack:** Desktop live voice typing. Local NVIDIA Parakeet (offline TDT-0.6B-v2 checkpoint) driven by Handy in a "press hotkey, speak, release, get text" pattern (offline ASR

+ utterance-final injection — see `streaming-vs-batch-injection`). Compared against the same model family's streaming-configured variants and the broader streaming RNN-T / Conformer-Transducer family.

TL;DR

The accuracy difference is real, measurable, and traces directly to the attention mechanism — it is not just frontend engineering. End-of-utterance batch inference lets the model attend bidirectionally over the entire encoded clip; chunked streaming inference forces a causal or short-lookahead encoder that has not seen the future of the utterance when it commits each token. On clean speech the gap is typically **1–4 absolute WER points** in favour of batch; on hard material (technical jargon, long sentences, names, code-switching, low SNR) it can widen to 5–10 points or more.

There are *also* training-objective and post-processing differences that compound the architectural one. A streaming model is trained to emit good *partials* (low emission latency, stability), which trades against final accuracy. A batch model is trained purely for final WER. So the streaming model is doubly disadvantaged: a weaker context window *and* a divided objective.

What "frontend engineering" *can* buy you is cosmetic — perceived latency, the look of words appearing as you speak, the option to stop early. It cannot recover the right-context information the encoder never saw.

Background

The question reduces to: when a transducer emits the token for word *N*, how much of the surrounding audio has the encoder been allowed to attend to?

- **Batch (offline) inference:** the encoder sees frames `[1 … T]` (the whole utterance) before the decoder emits any token. Every token decision is made with full bidirectional context.
- **Chunked streaming inference:** the encoder is invoked on chunk *k* with access to past chunks `[k-c … k]` and possibly a small lookahead `[k+1 … k+l]` (commonly 80–480 ms, sometimes up to ~1 s). Token emission happens within or shortly after each chunk. The encoder *cannot* see frames beyond `k+l` when committing the chunk's tokens.

That's the whole story, mechanically. Everything else is a consequence.

Why batch is more accurate: the attention-mechanism argument

1. Bidirectional self-attention disambiguates phonetically ambiguous spans

English (and most languages) are riddled with phonetic ambiguity that is only resolvable from later context:

- "I read the book" — past or present tense? Disambiguated by tense markers downstream.
- "recognise speech" vs "wreck a nice beach" — disambiguated by topical coherence over seconds.

- Garden-path sentences ("the horse raced past the barn fell"), proper noun boundaries ("New York Times" vs "new york times"), prosodic boundaries that fall after a content word — all benefit from right context.

In a bidirectional encoder, frame *t* can attend to frames `t+1, t+2, …, T`. The representation for the audio around "read" carries information from the inflection on the verb that comes 800 ms later. In a causal encoder, that information is structurally unavailable when the token is emitted.

This isn't a minor refinement — it's exactly the kind of disambiguation transformers were invented to do. Stripping it out is the cost of streaming.

2. Cross-attention over a long encoder output beats monotonic alignment

Whisper-class offline architectures use a transformer decoder with **cross-attention over the entire encoder output**. The decoder, when emitting token *N*, can attend to *any* frame in the encoded clip — beginning, middle, or end — and weight those frames by relevance.

Streaming ASR replaces this with a **monotonic alignment** mechanism (RNN-T, CTC, or TDT). The decoder is structurally constrained to emit text in lockstep with audio time. Monotonic alignment is what makes streaming *possible* — but it forfeits the ability to "look back" with content-conditioned attention.

For offline Parakeet-TDT-0.6B specifically, the architecture is still RNN-T-flavoured (transducer with limited audio context), not full cross-attention — so the gap to a hypothetical offline cross-attention model is smaller than the Whisper-vs-streaming gap. But the streaming-Parakeet vs offline-Parakeet gap still exists and follows the same logic at smaller magnitude: more context per emission decision = better decision.

3. Beam search at the end of an utterance prunes more confidently

Beam-search decoding with a language model rescore is much more effective when the whole utterance is available:

- Final beam selection sees the full sentence-level probability of each candidate. A high-likelihood mid-sentence path that becomes ungrammatical by the end of the sentence can be pruned in favour of an alternative that pays a small acoustic cost mid-sentence to gain a much larger language-model win.
- LM rescoring with a sentence-level model (or external transformer LM) is straightforward in batch and awkward in streaming (you don't have the future sentence yet).

Streaming systems use online beam search with shallow LM fusion at each emission step, which makes locally optimal but not globally optimal choices.

4. Punctuation, casing, and ITN are bidirectional problems

The downstream cleanup chain (punctuation/casing restoration, inverse text normalisation) inherently needs both left and right context to do its job well:

- Comma vs period vs question mark depends on what follows.
- "twenty twenty six" → "2026" requires seeing all four words before deciding it's a year.

- Capitalisation of “Smith” depends on whether “Dr.” or “Mr.” appeared earlier *and* on whether a name-like context follows.

In batch mode, the cleanup model sees the entire utterance and produces a coherent, well-punctuated result. In streaming mode, the cleanup model has to make local decisions and may revise punctuation as more text arrives — visible as the “comma turns into a period” flicker in live-caption UIs. The final text after enough revisions can match batch quality, but the intermediate states are noisier and the final state can still be slightly worse if the model committed too early.

Why chunked streaming is less accurate beyond the attention argument

Training objective is divided

A streaming model is trained against a multi-objective loss:

- WER on the final transcript (the same as batch).
- Emission latency (penalises tokens emitted “late” relative to the audio they correspond to — see FastEmit, alignment-restricted RNN-T, minimum-latency training).
- Partial stability (sometimes — penalises partials that get heavily revised).

These auxiliary losses *trade against* final WER. A model optimised purely for final WER (the batch case) can spend all its capacity there. A streaming model has to compromise.

Chunk-boundary artefacts

When utterances span chunk boundaries, the model loses some context across them depending on the streaming strategy:

- **Cache-aware streaming Conformer** (NeMo’s streaming config) maintains an attention cache across chunks, mitigating but not eliminating the loss.
- **Chunked-causal attention** with small lookahead loses information at every boundary.
- **VAD-segmented chunked-offline** (“streaming Whisper” approaches) loses *all* context across boundaries because each chunk is re-encoded from scratch.

Long sentences that cross boundaries see worse punctuation, worse name handling, and occasionally word-level errors at the seams.

Per-utterance overhead is amortised differently

This is a subtle point but worth flagging: streaming models are typically trained and tuned for sustained speech. Their behaviour on very short utterances (1–2 words) can be slightly worse than offline models, which are trained on a broader distribution including short isolated utterances. (Offline Whisper has the *opposite* problem with very short clips because of the 30 s padding issue, but offline Parakeet does not — it’s well-behaved across utterance lengths.)

How big is the gap in practice?

Numbers from the public benchmarks, as of 2025:

- **Offline Parakeet-TDT-0.6B-v2** sits at the top of the Open ASR Leaderboard with WER around 6–7% averaged across LibriSpeech / TEDLIUM / GigaSpeech / SPGISpeech / VoxPopuli / Earnings22 / AMI etc.
- **Streaming-configured Parakeet** (RNNT or Conformer with chunked attention) typically sits 1–3 absolute WER points worse on the same benchmarks, depending on chunk size and lookahead.
- **Whisper-large-v3** (offline) is in a similar range to offline Parakeet on most of those benchmarks.
- **Commercial streaming ASR** (Deepgram Nova-2/3, AssemblyAI Universal-Streaming, Soniox) sits 2–4 points worse than offline state-of-the-art on the same data, though they make up much of that ground with aggressive post-processing and domain biasing.

Translated into user-visible terms: on a 50-word dictation snippet, a 2-point WER gap is one extra word wrong on average. On a 200-word legal/medical/technical paragraph with names and acronyms, it can be 5–10 extra word errors.

The gap is **not constant across material**. It is small or invisible on:

- Short, common-vocabulary utterances ("send a message to John").
- Clean audio in well-represented accents.
- Material similar to the model's training distribution.

The gap is large on:

- Technical jargon, code-switching, proper nouns.
- Long sentences with subordinate clauses.
- Anything where the utterance-final words disambiguate earlier ones.
- Low SNR or far-field audio.
- Domain-specific vocabulary the streaming model wasn't biased for.

What frontend engineering can and cannot fix

Frontend engineering — the chunking strategy, VAD endpointing, partial/final discipline, injection policy — controls the *user experience* of streaming, not the *accuracy ceiling* of the encoder.

Things frontend can buy you:

- Perceived latency (words appearing while you speak).
- Self-correction loop (stop early if misheard).
- Hands-free / VAD-gated operation.
- Stable injection (commit only finals, never partials).

Things frontend cannot buy you:

- The right-context information the causal encoder didn't see.
- The bidirectional attention disambiguation.
- The benefit of sentence-level LM rescoring on the full utterance.
- The cleaner punctuation/casing/ITN that comes from seeing the whole sentence.

If accuracy is the priority, batch wins. If perceived latency is the priority, streaming wins. The two are genuinely in tension at the model level — not just at the UI level.

What the chunked-offline middle ground actually buys you

A common frontend pattern is **chunked-offline**: run an offline model on VAD-segmented chunks of audio, injecting each chunk's output as it completes. This is what most "streaming Whisper" tools do (`whisper-streaming`, `WhisperLive`, faster-whisper with VAD).

It is worth being precise about what this gives up vs full-utterance batch:

- Each chunk is still processed with full bidirectional attention *within the chunk*. So you keep most of the within-chunk accuracy benefit.
- You lose context *across* chunk boundaries — the model has no memory between chunks. Sentences that span boundaries suffer.
- Long-sentence LM rescoring is local to each chunk.
- Punctuation/casing is per-chunk and may be inconsistent across boundaries.

In practice, a sensibly tuned chunked-offline pipeline (with 5–15 s VAD chunks and overlap) gets within 1 WER point of full-utterance batch on most material, while *appearing* live. This is a more accurate way to "feel streaming" than running a true streaming model — at the cost of a chunk-sized latency floor (you don't see anything until the first VAD pause).

So the practical accuracy ranking, best to worst, is roughly:

1. **Offline batch on full utterance** (your current Parakeet + Handy setup).
2. **Chunked-offline with long chunks (10–30 s) and VAD**.
3. **Chunked-offline with short chunks (1–3 s) and VAD**.
4. **True streaming with large lookahead (~480 ms–1 s)**.
5. **True streaming with small lookahead (~80–160 ms)**.
6. **Strictly causal streaming (no lookahead)**.

Each step down trades accuracy for either latency, perceived liveness, or hands-free operation.

Specific to your Parakeet + Handy setup

You're at the top of that ranking — **offline batch on full utterance**. You are paying the perceived-latency cost (no words appear until you toggle off) and getting the maximum accuracy the Parakeet family offers. The TDT-0.6B-v2 checkpoint is, as of writing, near the top of every offline benchmark.

If you ever switch to a streaming-configured Parakeet (NeMo supports it), expect:

- 1–3 absolute WER points regression on clean speech.
- A larger regression on technical / long-sentence material.
- A meaningful engineering jump in the injection layer (partial/final discipline, cursor management, undo behaviour).
- A different post-processing burden — the offline pipeline can run punctuation/ITN on the complete sentence; the streaming pipeline has to do it incrementally.

That regression is the *real* cost of moving to live-typing UX. It's not just engineering; the model is structurally seeing less.

Recommendation for the workspace's "ideal UI" spec

Treat the recognition-mode choice as an **accuracy-vs-perceived-latency dial**, not an architectural detail. Document the WER cost of each step explicitly so future-you can make an informed tradeoff. Concretely:

- For dictation that targets **publish-quality prose, code, technical text, or anything reviewed carefully**, prefer offline batch. Accept the perceived latency.
- For dictation that targets **chat, ephemeral messages, search queries**, prefer streaming or chunked-offline with short chunks. Accuracy is secondary to feeling responsive.
- For **hands-free operation**, you have no choice — streaming with VAD endpointing is the only option. Accept the WER hit.
- Consider a **hybrid mode**: stream partials into an overlay (visible feedback) while running offline batch in the background on the same audio, then commit the offline result on toggle-off. This is the most accurate "feels live" pattern, at the cost of running both pipelines.

Caveats

- **The numbers cited are benchmark averages.** Your domain (technical communications, AI/automation jargon, mixed English with occasional Hebrew) may sit far from the benchmark distributions. Run a small WER comparison on your own audio if accuracy matters at the margin.
- **Streaming models are improving fast.** The 2025 generation of streaming models (Universal-Streaming, Nova-3, streaming Parakeet variants) has narrowed the gap meaningfully vs the 2022–2023 cohort. Don't extrapolate from old comparisons.
- **The cleanup chain matters as much as the encoder.** Two systems with the same raw WER can have very different *user-perceived* accuracy after punctuation, casing, ITN, and disfluency removal. Streaming systems often invest more in this layer to compensate.
- **"Streaming" is overloaded.** Some APIs marketed as streaming are chunked-offline with VAD under the hood (this is fine, but it's not the same architectural class). Check whether the model emits true partials that get revised, or just one final per chunk.
- **LM fusion changes the picture.** A streaming model with a strong external LM and lookahead can sometimes match or beat an offline model without LM fusion. Vendor benchmarks may or may not include external LM rescoring; read the methodology.
- **End-to-end latency is not just model latency.** A "fast" streaming model with 600 ms of audio buffering and 200 ms of injection round-trip is not faster end-to-end than a "slow" offline model with no buffering.

References

- Open ASR Leaderboard (where offline vs streaming models can be compared on identical data): https://huggingface.co/spaces/hf-audio/open_asr_leaderboard
- NVIDIA Parakeet-TDT-0.6B-v2 model card (offline checkpoint): https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2
- NVIDIA NeMo streaming ASR documentation (streaming Conformer / Parakeet configurations): https://docs.nvidia.com/deeplearning/nemo/user-guide/docs/en/main/asr/intro.html
- Graves, *Sequence Transduction with Recurrent Neural Networks* (RNN-T): https://arxiv.org/abs/1211.3711

- Yu et al., *FastEmit: Low-latency Streaming ASR with Sequence-level Emission Regularization* (a representative streaming auxiliary loss): https://arxiv.org/abs/2010.11148
- Mahadeokar et al., *Alignment Restricted Streaming Recurrent Neural Network Transducer*: https://arxiv.org/abs/2011.03072
- Gulati et al., *Conformer: Convolution-augmented Transformer for Speech Recognition*: https://arxiv.org/abs/2005.08100
- Handy (the desktop dictation client used in this setup): https://github.com/cjpais/Handy
- Related in this workspace:
  - `streaming-vs-batch-injection` — separates recognition mode from injection mode.
  - `whisper-vs-streaming-asr-for-dictation` — why offline-trained models like Whisper resist streaming.

Topic: `cursor-dictation-vs-clipboard-stt-adoption`

Prompt

Cursor-level dictation vs record-then-paste STT — who loves which, and where

Asked: 14/04/26 **Slug:** `cursor-dictation-vs-clipboard-stt-adoption` **Response:** `ideas/cursor-dictation-vs-clipboard-stt-adoption.md`

The question

Research the best-loved tools for dictation at the cursor level and identify which industries and users find this approach helpful, as opposed to the alternative speech-to-text capture method: a recording tool that transcribes in one go, puts the result on the clipboard, and leaves the user to paste it into the text editor themselves.

Compare popularity across operating systems, and identify the use cases that work better or worse for each of the two modes.

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Cursor-level dictation vs record-then-paste STT — adoption patterns across industries, users, and operating systems

Question: `questions/cursor-dictation-vs-clipboard-stt-adoption.md` **Written:** 14/04/26 **Stack:** Desktop-class STT (macOS / Windows / Linux). Excludes mobile dictation and meeting-transcription tools.

Related reading in this repo:

- `ideas/streaming-vs-batch-injection.md` — the injection-time UX tradeoff underneath this question.
- `ideas/live-typing-models-saas-and-local.md` — which models are capable of each mode.
- `ideas/whisper-vs-streaming-asr-for-dictation.md` — why the model choice locks you into one of the two modes.

TL;DR

Cursor-level dictation is the *dominant* voice-input modality in healthcare (Dragon Medical One into Epic/Cerner) and in accessibility power-user circles (Dragon Professional, Talon,

Voice Control, Voice Access). Record-then-paste clipboard tools (Superwhisper, MacWhisper, AudioPen, VoiceNotes, Aqua Voice in its batch mode) have exploded since Whisper's 2022 release and now dominate the **knowledge-worker / long-form / thinking-aloud** niche — especially on macOS, and especially for users who want an LLM to clean up the transcript before it's used. Wispr Flow (2024–25) is the most visible attempt to pull general knowledge workers back to cursor-level dictation with LLM polishing applied in-flight.

The split isn't about which is better; it's about **whether the user is entering text they've already composed mentally (short, interactive, cursor-level wins)** or **composing *while* speaking (long-form, pause-heavy, clipboard + cleanup wins)**.

Background — the two modes

Cursor-level dictation ("live typing"): the tool synthesises keystrokes or synthetic input events into whatever window has focus. Text appears in-place, word-by-word or utterance-by-utterance. Examples: macOS built-in Dictation, Windows Voice Access, Dragon NaturallySpeaking / Dragon Medical One, Talon Voice, Wispr Flow, nerd-dictation, Aqua Voice (live mode).

Record-then-paste ("clipboard STT" / "batch dictation"): the user records an utterance (hold-to-record, toggle, or file drop), the tool transcribes (usually Whisper or a hosted equivalent) *after* recording ends, and the result lands on the clipboard or in a notes buffer. Examples: Superwhisper (primary mode), MacWhisper, AudioPen, VoiceNotes.com, Whisper Memos, Whispering, BetterDictation, Aqua Voice (batch mode), many `whisper.cpp`-wrapping scripts.

The two overlap: several tools offer both modes (Aqua, Superwhisper, VoiceInk), and "paste" can be automated via synthetic paste so the line between them blurs on the output side. The user-experience difference is whether **feedback arrives during speech** (cursor-level) or **after it** (clipboard).

The best-loved tools, by mode

Cursor-level dictation — flagship tools

Tool	OS	Audience	Why it's loved
Dragon Medical One (Nuance / Microsoft)	Windows	US physicians, radiologists	Real-time dictation into Epic / Cerner / Meditech templates; domain-trained for medical vocabulary; structured macros ("normal chest" → full paragraph). The *de facto* EMR input method for a huge slice of US medicine.
Dragon Professional / Legal	Windows	Attorneys, transcriptionists, power dictators	Decades of refinement; custom vocabulary; full voice-

Tool	OS	Audience	Why it's loved
			control of the OS; enterprise deployment. Still the benchmark despite Nuance's declining focus.
Talon Voice	macOS, Windows, Linux	Accessibility (RSI, motor-impairment), voice coders	Voice-driven computing including coding-specific grammars; paired with Cursorless for structural code editing. A cult favourite.
Apple Dictation / Voice Control	macOS, iPadOS	General Mac users, accessibility	Built-in, improved markedly with on-device models in Sequoia/Tahoe; Voice Control is a full hands-free OS driver.
Windows Voice Access	Windows 11	Accessibility, hands-free users	Replaced Windows Speech Recognition; on-device, continuous, reasonable accuracy.
Wispr Flow	macOS, Windows (2025)	Knowledge workers, engineers, PMs	Streaming dictation + inline LLM polish (disfluency removal, capitalisation, light rewriting) — the first cursor-level tool to feel "AI-native" rather than "speech-recognition-native".
Aqua Voice (live mode)	macOS, Windows	Knowledge workers, voice-first writers	Cursor injection plus command grammar ("new paragraph", "delete that").
nerd-dictation + VoiceInk / Whispering	Linux, macOS	Hobbyists, privacy-conscious users	Scripts around Vosk / `whisper.cpp` that inject via `xdotool` / `ydotool` / macOS accessibility APIs.

Record-then-paste — flagship tools

Tool	OS	Audience	Why it's loved
Superwhisper	macOS (Windows in beta)	Mac power users, writers, engineers	Fast Whisper inference, customisable LLM cleanup step, global hotkey, paste-to-focused-app. The reference product for this category.
MacWhisper	macOS	Writers, researchers, podcasters	File-based and live-hotkey modes; well-polished UI; strong Whisper model management.
AudioPen	Web (cross-OS)	Content creators, knowledge workers, journaling	Raw transcript + "polished" LLM rewrite — the "think out loud, get coherent prose" workflow. Very popular with founders / writers on social media.
VoiceNotes.com	Web	Similar to AudioPen + heavier note-management	Tags, todos, structured outputs.
Whisper Memos	iOS + Mac sync	Mobile-first thinkers	Long-form voice capture → transcript + email.
Whispering / BetterDictation / VoiceInk	macOS	Indie / OSS users	Open-source Whisper wrappers with hotkey capture + clipboard output.
Aqua Voice (batch mode)	macOS, Windows	Same user as live mode, switched for long inputs	—
Willow / Talk-To-Me / ad-hoc `whisper.cpp` scripts	Linux, cross-OS	Linux users, tinkerers	Usually DIY — no dominant polished product on Linux.

Which industries and users favour which mode

Cursor-level wins

Healthcare (US, UK, DE, AU). The single largest real-world deployment of cursor-level dictation. Physicians dictate directly into the EMR during or immediately after patient

encounters. The interactive loop — say it, see it, correct it, move on — is essential because the doctor is *simultaneously* navigating the patient record. Paste-from-clipboard would break the clinical workflow. Dragon Medical One is the incumbent; scribing tools like Abridge, Suki, DeepScribe, Ambience are shifting some of this workload to ambient scribing, but dictation (cursor-level) remains the baseline modality.

Radiology. A separate sub-genre: radiologists dictate reports into structured templates while reading images. PowerScribe (Nuance) owns this; the whole workflow assumes cursor-level injection into template fields.

Legal. Historically digital handheld dictation → transcription pool (a *human*-in-the-loop batch workflow). Modern shift: Dragon Legal for cursor-level into Word / iManage. Still mixed — older attorneys often prefer record-and-send, younger ones adopt live dictation.

Accessibility. Cursor-level is the only viable option for users who cannot use a keyboard at all. Talon, Dragon, Voice Control, Voice Access dominate. A record-then-paste tool cannot drive an OS — you can't navigate menus, click buttons, or move windows from a clipboard.

Voice-coding. Talon + Cursorless, Serenade (now defunct-ish), Cursor / VS Code voice extensions. Command grammars, not free-form prose. Strictly cursor-level (and typically structural rather than character-level).

Short-form chat / messaging. Slack, Teams, iMessage, WhatsApp Web. The response is already composed in the user's head; they just want it out faster than typing. Cursor-level with a tight latency feel beats clipboard for anything under ~2 sentences.

Record-then-paste wins

Long-form writing with thinking pauses. Blog posts, essays, journal entries, email drafts where the user is *composing while speaking*. Cursor-level dictation punishes this: stray "uhm", abandoned sentences, and false-start retries all get committed live. Clipboard + LLM cleanup absorbs all of it — AudioPen's "polished" mode is the entire value proposition.

Podcaster / creator workflows. Record a riff, get it cleaned up into a draft script or LinkedIn post. This is a pure batch job — there is no "cursor" that needs live text.

Idea capture / voice journaling. Whisper Memos, AudioPen, VoiceNotes. The unit of work is an *utterance*, not a typing session.

Field / mobile → desktop handoff. Inspectors, insurance adjusters, real-estate agents dictating notes on a phone that sync to a desktop EMR / CRM. Fundamentally asynchronous.

Noisy / non-ideal environments. Cursor-level dictation is brittle when the user has to re-record — the broken attempt is already in the document. Batch mode lets the user discard and restart.

Privacy-sensitive drafting. Reviewing the transcript before it enters the target application is sometimes a hard requirement (legal, HR, medical notes outside the EMR).

Pause-for-thought composers. Users whose speech cadence is "say a sentence, think for 15s, say the next sentence". Live-typing tools with pause-based endpointing either commit too eagerly or drop audio; record-then-paste sidesteps the whole endpointing problem. (See `ideas/inference-cadence-and-sentence-entry.md` for why.)

Mixed / depends on user

Knowledge workers (PMs, engineers, designers, consultants). The contested territory. Historically a clipboard-STT crowd on macOS (Superwhisper). Wispr Flow is the main tool trying to flip them to cursor-level. Outcome currently unclear; users with long-form + thinking pauses stay on clipboard, users with short interactive bursts migrate to cursor-level.

Academics and researchers. Lean batch — transcripts of their own thinking that then get structured into notes / papers. Obsidian + Whisper workflows are common.

Popularity across operating systems

macOS. The richest ecosystem for *both* modes. Built-in dictation has genuinely improved since Apple moved to on-device models. The indie-tool boom (Superwhisper, MacWhisper, Aqua, Wispr Flow, VoiceInk, Whispering) happened predominantly on macOS because accessibility APIs for simulated keystrokes and paste actions are well-documented and uniformly supported. Clipboard STT is dominant among Mac knowledge workers; cursor-level is growing via Wispr.

Windows. Historically the strongest home of cursor-level dictation because of Dragon. Enterprise healthcare / legal workflows run on Windows. Dragon Medical One is installed at nearly every US hospital of meaningful scale. Windows Voice Access has narrowed the gap for general users but remains a second choice to Dragon. Indie clipboard STT tools lag macOS by 12–24 months.

Linux. A long way behind both. There is no polished commercial cursor-level dictation on Linux. Real users are split between:

- Talon Voice (the only serious cross-OS accessibility solution that works on Linux).
- `nerd-dictation` + Vosk / `whisper.cpp` (hobbyist).
- DIY scripts with `ydotool` (Wayland) / `xdotool` (X11) feeding Whisper batch output.

Linux clipboard STT is essentially “roll your own” — and the injection side (global hotkey, focus-aware paste) is hardest on Wayland because of sandboxing. This is why Linux users disproportionately settle for record-then-paste even when they’d prefer cursor-level.

iOS / Android (for desktop comparison). Excluded by the question, but worth a note: mobile OSes have had excellent built-in cursor-level dictation for years, which is why desktop cursor-level dictation expectations keep rising.

Why the split exists — design pressures

1. **Cursor-level dictation has a hard tradeoff between latency and accuracy.** Commit too early and you strand partials; commit too late and the user can’t course-correct. See `ideas/inference-cadence-and-sentence-entry.md`.
2. **Whisper (the dominant OSS model) is batch-native.** Its attention over the full utterance is why it’s so accurate and why it cannot stream natively. Teams who want accuracy over liveness build batch/paste workflows; teams who want liveness use streaming models (Soniox, Deepgram Nova-3, AssemblyAI Universal-Streaming) and accept lower accuracy per partial. See `ideas/whisper-vs-streaming-asr-for-dictation.md` and `ideas/batch-vs-chunked-inference-accuracy.md`.

3. **LLM post-processing favours batch.** Running a cleanup / punctuation / disfluency-removal pass is much simpler on a complete utterance than on a streaming partial — so AI-native clipboard tools got to market faster than AI-native cursor-level tools.
4. **Cursor-level requires OS-level accessibility integration.** macOS and Windows expose APIs; Linux/Wayland makes it genuinely hard. This shapes what's even buildable.

Where the two modes land — verdict

Workflow	Best mode	Why
Short reply in a chat / form field	Cursor-level	Composed in head; cursor feedback beats round-trip to clipboard.
Clinical note in EMR	Cursor-level (Dragon Medical)	Structured templates + in-context correction + navigation demand it.
Legal brief dictation	Cursor-level or human-in-loop batch	Both survive; firm-specific.
Voice-driven OS control	Cursor-level	Clipboard can't click buttons.
Blog post / long-form writing	Record-then-paste + LLM polish	Cleanup of disfluencies is essential.
Journal / idea capture	Record-then-paste	Utterance is the unit, not a cursor.
Field notes → desktop CRM	Record-then-paste	Asynchronous by nature.
Coding (structural edits)	Cursor-level with command grammar	Talon / Cursorless — syntax sensitivity kills free-form paste.
Email drafts	Either — depends on user cadence	Short: cursor-level. Long/ruminative: batch.
Pause-for-thought composition	Record-then-paste	Endpointing fails on live composition.

References

Tools

- Dragon Medical One — https://www.nuance.com/healthcare/provider-solutions/speech-recognition/dragon-medical-one.html
- Dragon Professional — https://www.nuance.com/dragon.html
- Talon Voice — https://talonvoice.com/
- Wispr Flow — https://wisprflow.ai/
- Superwhisper — https://superwhisper.com/
- MacWhisper — https://goodsnooze.gumroad.com/l/macwhisper
- AudioPen — https://audiopen.ai/
- VoiceNotes — https://voicenotes.com/
- Aqua Voice — https://withaqua.com/

- Whispering — https://github.com/braden-w/whispering
- VoiceInk — https://github.com/Beingpax/VoiceInk
- nerd-dictation — https://github.com/ideasman42/nerd-dictation
- Windows Voice Access — https://support.microsoft.com/en-us/topic/use-voice-access-to-control-your-pc-author-text-with-your-voice-4d21cb7c-0b5a-4f7c-8c7c-0c49e61e6bae

Industry context

- Abridge / Suki / DeepScribe / Ambience (ambient clinical scribing — adjacent, not identical to dictation).
- Cursorless (voice-coding structural editing) — https://www.cursorless.org/
- PowerScribe (radiology dictation) — https://www.nuance.com/healthcare/diagnostics-solutions/radiology-reporting/powerscribe-one-radiology-reporting.html

Caveats

- The “most popular tool” by install count is very different from the “most loved tool” by active use. Apple Dictation has vastly more installs than anything here, but it’s often the *starting point* users abandon for Wispr/Superwhisper/Dragon once they commit to voice as a primary input.
- Wispr Flow is new enough (GA 2024, Windows 2025) that its retention data isn’t yet settled. Treat “winning” claims with scepticism.
- Dragon’s long-term trajectory under Microsoft is uncertain; Nuance has quietly deprecated consumer Dragon on Mac, and enterprise Dragon’s roadmap is less public than it used to be.

Topic: `custom-vocabulary-in-stt`

Prompt

Custom vocabulary in transcription tools — how is it actually implemented?

Asked: 14/04/26 **Slug:** `custom-vocabulary-in-stt` **Response:** `ideas/custom-vocabulary-in-stt.md`

The question

Many transcription tools contain a custom word feature because they often get specific or niche words wrong. How is this feature actually implemented at the technical level? Is it a front-end feature, system-prompted, or something else? How large can this custom dictionary grow without degrading inference notably?

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Custom vocabulary in STT — what's actually happening under the hood, and how big the list can get

Question: `../questions/custom-vocabulary-in-stt.md` **Written:** 14/04/26 **Stack:** Desktop live voice typing. Covers cloud streaming ASR (Google, Deepgram, AssemblyAI, Azure, AWS Transcribe, Speechmatics, Soniox), local engines (Whisper / faster-whisper, NVIDIA Parakeet / Riva, Vosk / Kaldi, wav2vec2), and post-processing tools (Handy, Wispr Flow, Superwhisper, MacWhisper, Aqua, plus LLM-based rewrite passes).

TL;DR

"Custom word" features are almost never a single thing — they're a **family of five mechanisms** that sit at very different points in the pipeline, and most serious STT products expose two or three of them under one UI label. In rough order from deepest to shallowest:

1. **Pronunciation lexicon injection** — teach the acoustic/lexicon layer how a word is *spoken* (G2P entries).
2. **Decoder biasing / shallow fusion** — at beam-search time, add a log-prob boost to hypotheses that match the user's phrase list.
3. **Contextual attention biasing (CLAS-style)** — embed the phrase list and let the decoder attend to it as a soft "remember these" signal.

4. **Prompt/context conditioning** — Whisper's `initial_prompt`, or an LLM post-pass prompted with your glossary.
5. **Pure post-processing find-and-replace** — regex or fuzzy-match rewrites on the finalised text. No acoustic awareness at all.

The UX-level "custom dictionary" is essentially a frontend form that routes into one or more of these. It is **not** generally a system prompt in the LLM sense, except in the specific case of Whisper's prompt field and LLM-rewrite tools like Wispr Flow or a Handy post-pass.

Practical size limits before inference degrades:

Mechanism	Typical ceiling	Where degradation shows up
Whisper `initial_prompt`	~224 tokens (≈150–180 words)	Hard cap — tokens past the limit are silently dropped; too much content biases the model to hallucinate the glossary.
Cloud decoder boost (Google, Deepgram, AssemblyAI, Azure, AWS)	500–5,000 phrases per request, 10k–100k in a server-side "custom class"	False positives (the model hears the boosted word when you didn't say it); decoding latency grows roughly linearly past a few thousand entries.
Contextual attention (Google CLAS, NVIDIA Riva `word_boost`)	~1,000–3,000 bias phrases	Diminishing returns past a few hundred; very long lists dilute the attention signal.
WFST / lexicon injection (Kaldi, Riva, Vosk)	10k–100k+ words	Mostly graph-size / RAM limited rather than accuracy-limited.
LLM rewrite pass (Wispr Flow, custom Handy pipeline)	Whatever fits the LLM context window (thousands of terms)	Latency and cost scale linearly with prompt size; the LLM may over-correct and swap in a glossary term that wasn't actually said.
Post-hoc find-and-replace	Effectively unlimited	Won't fix anything the ASR didn't produce in a recognisable shape in the first place.

The honest headline: if the ASR has never **heard** the word — no acoustic pattern, no pronunciation — no amount of frontend biasing will conjure it from nothing. Custom vocabulary is a **tilt**, not a teach. Teaching (new pronunciations, new tokens) requires either a G2P entry or actual fine-tuning.

Background — why this feature exists at all

ASR models are statistical. They emit the most probable token sequence given the audio. "Probable" is determined by:

1. **Acoustic likelihood** — how well the waveform matches the phoneme sequence for a candidate word.
2. **Language-model likelihood** — how plausible that word sequence is in normal text.

Niche words lose on both fronts. A company called *Rosehill* has:

- An acoustic profile ambiguous with *rose hill*, *roast hill*, *Rose Hill* (two words), *rosetta*…
- A vanishingly small language-model prior — in training data, “rose” is followed by “garden”, “petals”, “wine”, not “hill” as a surname.

So the decoder quite reasonably picks *rose hill*. Custom vocabulary features exist to **re-weight the probabilities in the user’s favour**, without retraining. The art is that you want to nudge the balance just enough to win on the cases where the user actually said the niche word, *without* hallucinating it everywhere else.

That tradeoff — **recall gain vs. false-positive cost** — is the single most important thing to keep in mind when evaluating any tool’s custom-word implementation.

The five mechanisms, in depth

1. Pronunciation lexicon injection (G2P entries)

Where it lives: inside the ASR’s lexicon / grapheme-to-phoneme (G2P) stage, before decoding.

What it does: tells the system “the written form *Rosehill* is pronounced /ˈroʊz.hɪl/”. From then on, the decoder knows that audio matching those phonemes can be emitted as *Rosehill* as a single unit.

Where you see it:

- **Kaldi / Vosk** — you edit `lexicon.txt` and rebuild the decoding graph (HCLG).
- **NVIDIA Riva** — custom pronunciations via the `inverse_text_normalization` and lexicon override configs.
- **Google Speech-to-Text v2** — you can attach phoneme hints in a `CustomClass` for very rare words, but most users stop at the plain-text phrase.
- **Azure Custom Speech** — dedicated pronunciation upload (.txt with SAPI or IPA).
- **AWS Transcribe** — vocabulary entries accept `IPA`, `SoundsLike`, and `DisplayAs` fields.

Strengths: the *deepest* and most accurate form of customisation — it actually teaches the engine a new word, not just re-ranks existing hypotheses. Once a pronunciation is registered, the word competes fairly with the rest of the vocabulary.

Weaknesses: requires phonetic knowledge (IPA or an ASCII phoneme set the engine understands), not exposed by most consumer tools, and not available at all in end-to-end models that have no explicit lexicon (Whisper, Parakeet, wav2vec2, Conformer-CTC trained with BPE tokens — all of them produce subword tokens directly from audio, with no pronunciation table in the middle).

Size ceiling: tens of thousands of entries is routine in production Kaldi / Riva deployments. The cost is decoding-graph compile time, not runtime inference.

2. Decoder biasing / shallow fusion

Where it lives: during beam search at inference time.

What it does: for each hypothesis in the beam, if it contains one of the user's boost phrases (or a prefix of it), add a **log-probability bonus** — typically +2 to +15 in log space. That pushes the hypothesis up the ranking so it survives pruning and can win the final argmax.

Where you see it:

- **Deepgram** — `keywords` (legacy) and `keyterm` (Nova-3) parameters; you can pass the term plus an intensifier (e.g. `Rosehill:5`).
- **AssemblyAI** — `word_boost` with a `boost_param` of `low` / `default` / `high`.
- **AWS Transcribe** — custom vocabulary list with implicit boost.
- **Google Speech-to-Text** — `SpeechAdaptation` with `PhraseSet` and a per-phrase `boost` float.
- **Local Kaldi / Vosk** — word-level biasing by editing the LM or injecting a class-based LM.
- **Whisper + ctranslate2 (faster-whisper)** — partial support via logit processors, but not exposed in the typical wrapper UIs.

Strengths: cheap, fast, request-scoped (no retraining, no server-side state). The list can be swapped per utterance, which is why "context-aware" tools can load a different vocabulary depending on the focused application.

Weaknesses: it's a **tilt**, not a teach. If the acoustic model never emitted *Rosehill* as a candidate in the beam — e.g. because the subword tokens *rose* + *hill* dominated — no amount of boost can promote a hypothesis that doesn't exist. This is why users observe that custom words sometimes "just don't work": the bias fires only when the correct tokens were already in contention.

The symmetric failure is overshooting: boost too high and the model starts emitting *Rosehill* whenever you say anything resembling "rose hill", "roast hill", or even unrelated audio.

Size ceiling: empirically, the top ~100–500 boosted phrases do most of the work. Past a few thousand, you start seeing:

- **Beam saturation** — too many low-bias phrases compete with each other, diluting the signal.
- **Latency growth** — string-matching against the bias list during beam search is O(beam × phrases × max-phrase-length). Providers mitigate with tries / Aho-Corasick, but you still see ms-scale growth past ~10k.
- **False positive drift** — the broader the list, the higher the chance a common utterance happens to trigger a bias.

Typical documented caps: Deepgram 100 keyterms/request (Nova-3), AssemblyAI 1,000 word-boost entries, Google 5,000 phrases per SpeechAdaptation (with a separate CustomClass ceiling of 10k–100k items server-side), Azure up to ~2,048 phrases per request, AWS Transcribe 50k entries in a vocabulary.

3. Contextual attention biasing (CLAS and friends)

Where it lives: inside the decoder network itself, as a learned attention mechanism.

What it does: at inference, the phrase list is embedded and passed as an additional attention target alongside the encoder output. The decoder learns to "look at" the bias list when the current audio is ambiguous, and softly copy from it.

Where you see it: Google's **CLAS** (Contextual Listen, Attend, and Spell) paper, Meta's **Neural Associative Memory**, NVIDIA's **context-biasing adapters** for Conformer/Parakeet. Usually not directly exposed to end users — it powers the server-side implementation of Google's phrase hints and similar cloud features.

Strengths: unlike shallow fusion, the network has been **trained** to use the bias list — so it generalises better to morphological variants and doesn't require the token sequence to already be in the beam. It's the closest thing to "teach at inference time" without retraining.

Weaknesses: requires a model trained with the biasing mechanism baked in; retrofitting to a vanilla Whisper/Parakeet is a research project, not a config flag. Also, the attention has a finite "slot budget" — past a few hundred phrases, the quality of the biasing signal degrades because the attention weights spread too thin.

Size ceiling: 500–3,000 phrases in published benchmarks; some providers page through larger lists by selecting a subset per request based on the application context.

4. Prompt / context conditioning

This is the one that most resembles what the user asked about with "is it system-prompted?" — and yes, for a small but growing slice of tools, it literally is.

Where it lives: either in the ASR's own prompt field (Whisper) or in a downstream LLM rewrite pass.

Two sub-flavours:

4a. Whisper-style `initial_prompt`

Whisper accepts an optional text prompt that is injected into the decoder's context as if it were prior transcript. The model then continues decoding conditioned on that prior. In practice this:

- Biases toward the vocabulary, casing, punctuation style, and language of the prompt.
- Has a **hard 224-token cap** (half of Whisper's 448-token decoder context window). Tokens past that are silently truncated from the *front*.
- Does not "teach" pronunciation — if the audio doesn't match the word, the prompt can't save you. But it does sharply improve recall on words the audio *could* produce, by raising their language-model prior.

This is what tools like **MacWhisper**, **Superwhisper**, **whisperX**, **faster-whisper**, **Aqua**, and many Whisper-wrapper apps use when they expose a "custom vocabulary" field. It's genuinely a prompt — but at the ASR level, not the LLM level.

224 tokens is the *only* hard ceiling in this whole document. Past it, content is dropped. Keep custom vocabulary under ~150 words and you'll never hit it.

4b. LLM rewrite pass

The ASR produces a raw transcript. A second stage (a small LLM, a GPT-4o-mini call, or a local Llama) rewrites it, with a system prompt like:

> Correct the following transcript for spelling and proper nouns. Known terms: Rosehill, DSR Holdings, Speechmatics, Parakeet, …

This is exactly how **Wispr Flow**, some **Handy** users' post-pipelines, and LLM-enhanced Whisper wrappers implement "custom dictionary". It is, literally, system-prompted.

Strengths: extremely flexible. Can handle style, formatting, abbreviation expansion, tone adjustment, not just vocabulary. Can fix words the acoustic model mangled badly, because it has full sentence context.

Weaknesses:

- **Latency**: even a fast local model adds 200–1000 ms to finalisation — a killer for streaming.
- **Cost**: per-transcript API calls add up.
- **Overcorrection**: the LLM will sometimes insert a glossary term that the user didn't say, simply because it's in the system prompt and the surrounding text is semantically adjacent.
- **Hallucination risk**: the LLM can rewrite beyond the narrow scope of vocabulary correction and invent content. Mitigated with tight prompts and temperature=0, but never fully eliminated.

Size ceiling: whatever fits the LLM context. A 128k-context model can hold tens of thousands of glossary entries, but you'll see latency and cost creep well before you run out of tokens. Practical sweet spot: a few hundred terms; past ~2,000 you're paying real cost per utterance.

5. Post-processing find-and-replace

Where it lives: a regex / fuzzy-match step on the final transcript, after the ASR is done.

What it does: `rose hill → Rosehill`, `deep gram → Deepgram`, `g p t 4 → GPT-4`.

Where you see it: almost every wrapper tool has this as a fallback, even if they also use one of the deeper mechanisms. It's the "replacements" or "text substitutions" tab.

Strengths: trivial to implement, zero latency impact, deterministic, can do things like casing/hyphenation that acoustic biasing can't.

Weaknesses: can only fix what the ASR already emitted *in a shape you can match*. If the ASR heard *Rosehill* as *roast still*, no regex will catch it unless you write a fuzzy-match rule — and fuzzy rules are where spurious replacements happen ("roast beef" → "Rosehill beef").

Size ceiling: thousands of rules is fine; the ordering matters more than the count (first-match-wins versus longest-match-wins).

How a typical "custom words" UI maps to these mechanisms

Tool	Custom word feature calls into	Scope	Hard limit
Deepgram Nova-3	Decoder biasing (`keyterm`) + contextual biasing (server-side)	Per-request	~100 keyterms; higher via custom models
AssemblyAI	Decoder biasing (`word_boost`)	Per-request	1,000 words

Tool	Custom word feature calls into	Scope	Hard limit
Google Speech-to-Text	Decoder biasing (`PhraseSet`) + contextual (CLAS) + optional phoneme hints	Per-request or server-side class	5k phrases/request, 100k in server class
Azure Speech	Lexicon injection (pronunciation file) + phrase list + Custom Speech fine-tune	Mix of per-request and model-level	2,048 phrases, larger via Custom Speech
AWS Transcribe	Lexicon (Sounds-Like/IPA) + custom vocabulary filter	Model-level	50k entries
Speechmatics	Decoder biasing (`custom_dictionary`) + optional Sounds-Like	Per-request	~1k entries documented
Whisper wrappers (MacWhisper, Superwhisper, faster-whisper, Aqua)	`initial_prompt`	Per-utterance	224 tokens
Wispr Flow	LLM rewrite pass with glossary in system prompt	Per-utterance	LLM context
Handy (default)	Mostly post-processing find-and-replace; some users add LLM rewrite	Per-utterance	Unlimited rules; LLM-limited if rewrite added
Vosk / Kaldi	Lexicon injection + optional LM class	Model-level (requires graph rebuild)	Tens of thousands
NVIDIA Parakeet / Riva	Lexicon + word boost + context biasing adapters	Mix	Thousands

Most consumer-facing apps **don't disclose which mechanism they use**. Treat any tool that claims "custom vocabulary just works for millions of words" with skepticism — the physics of beam search says otherwise for the shallow-fusion path, and the 224-token cap is a hard physical limit for Whisper-prompt path.

Is it a frontend feature?

Partially yes, partially no, depending on the tool:

- **Frontend storage and UI** — always. The list of custom words lives in the app's config file, database, or cloud settings, and is shown to the user as a form.
- **Frontend enforcement** — only for post-processing find-and-replace (mechanism 5) and for the LLM-rewrite case (mechanism 4b).
- **Backend enforcement** — for everything else. The word list is serialised into the ASR request (cloud) or the decoder config (local), and the biasing happens inside the model.

In cloud STT products, "custom vocabulary" is typically a server-side object that the client references by ID (e.g. Google's `PhraseSet` resource, AWS Transcribe's `Vocabulary`). The frontend stores the *list*, but the *biasing* happens during decoding on the server.

Is it system-prompted?

Only in two specific cases:

1. **Whisper `initial_prompt`** — yes, literally a prompt. Limited to 224 tokens.
2. **LLM-rewrite tools (Wispr Flow, post-ASR GPT passes)** — yes, a system prompt to a second-stage LLM, not to the ASR itself.

For everything else — Deepgram, AssemblyAI, Google, Azure, AWS, Speechmatics, Kaldi, Vosk, Parakeet, Riva — it is **not** a prompt. It's a structured parameter (list of strings, optionally with boost weights or pronunciations) that modifies beam search or lexicon lookup. Calling it a "system prompt" would misrepresent the mechanism.

When degradation actually sets in

Users asking "how big can this dictionary be" are usually worried about one of three things. Each has a different answer:

Accuracy degradation

- **Pronunciation lexicon / WFST**: essentially no accuracy ceiling up to 100k+ entries, as long as the new words don't *collide* phonetically with high-frequency existing words.
- **Decoder biasing**: degrades above a few hundred phrases if boosts are high, because false positives start firing on unrelated audio. With low boosts and a few thousand entries, accuracy is usually fine — but the gain per entry is diminishing.
- **Whisper prompt**: degrades fast past ~100 entries — the model starts hallucinating glossary terms into unrelated utterances. The cap is 224 tokens anyway.
- **LLM rewrite**: degrades with list size and with generic terms; rare proper nouns stay fine, but common English words in the glossary cause the model to over-apply them.

Latency degradation

- **Pronunciation lexicon**: zero runtime cost — it's all compile-time.
- **Decoder biasing**: low ms-scale growth; usually imperceptible below 10k phrases.
- **Whisper prompt**: the prompt tokens go through the decoder, so longer prompt = more autoregressive steps = worse first-token latency. Small effect (~10–50 ms) at the 224-token cap.
- **LLM rewrite**: dominant latency cost. Scales roughly linearly with prompt size for the prefill phase. A 2,000-word glossary at ~1.3 tokens/word = 2,600 tokens of prompt, which is 40–200 ms of prefill on a fast model and much more on a slow one. Added to the streaming output phase.

Memory / deployment overhead

- **WFST/lexicon**: the decoding graph grows with vocabulary. 100k new words can add hundreds of MB to the graph in Kaldi. Fine for servers; annoying for local installs.
- **Everything else**: negligible until you're into hundreds of thousands of entries.

Recommendations for this workspace's use case (live desktop dictation)

For a pause-tolerant prompt dictator who wants custom words to work reliably in streaming mode:

1. **Tier the list**. Only the 20–50 words that *actually get mis-transcribed repeatedly* need to be in the biasing layer. Dumping a 2,000-word glossary in makes the model worse, not better.
2. **Use the deepest mechanism the tool offers**. If you're on a cloud API with both a phrase list and a custom class / pronunciation field, use the pronunciation field for genuine novel words (surnames, product names) and the phrase list for existing-but-rare words.
3. **Don't use LLM rewrite for streaming**. The latency cost is incompatible with the feel of live dictation. Reserve it for batch-style "record then paste" flows.
4. **For Whisper-based local stacks**, the `initial_prompt` is the single most useful lever — keep it under 150 words, lead with the most important terms, and refresh it per application context if possible.
5. **Keep a post-processing find-and-replace layer as a safety net** for deterministic fixes (casing, hyphenation, brand spelling). It's cheap and it cleans up what the acoustic layer can't distinguish anyway.

References

- OpenAI Whisper paper and `openai/whisper` decoder prompt handling — Whisper GitHub repo (`decoding.py`, `initial_prompt`).
- Google CLAS: Pundak et al., *Deep Context: End-to-End Contextual Speech Recognition*, Interspeech 2018.
- Google Cloud Speech-to-Text `SpeechAdaptation` and `CustomClass` — cloud.google.com/speech-to-text/docs/adaptation.
- Deepgram `keywords` / `keyterm` parameter docs — developers.deepgram.com.
- AssemblyAI `word_boost` docs — assemblyai.com/docs.
- AWS Transcribe Custom Vocabulary (IPA / SoundsLike / DisplayAs) — docs.aws.amazon.com/transcribe/latest/dg/custom-vocabulary.html.
- Azure Custom Speech pronunciation files — learn.microsoft.com/azure/ai-services/speech-service/how-to-custom-speech-test-and-train.
- NVIDIA Riva word-boosting and custom pronunciation — docs.nvidia.com/deeplearning/riva.
- Kaldi WFST / lexicon construction — kaldi-asr.org docs.
- Speechmatics `custom_dictionary` — docs.speechmatics.com.
- Related in this workspace: Why Whisper isn't built for live dictation, Interim results and stabilisation.

Topic: `dictation-formatting-built-in-vs-supplementary`

Prompt

Dictation formatting features — built-in model support vs supplementary post-processing

Asked: 14/04/26 **Slug:** `dictation-formatting-built-in-vs-supplementary` **Response:** ideas/dictation-formatting-built-in-vs-supplementary.md

The question

Several formatting behaviours are essential for dictation output to feel usable — if the STT model doesn't handle them, they require a second-pass processor. Enumerate these requirements, then identify for each requirement which local and cloud STT models provide it as built-in tooling versus which ones need a supplementary model. Where a supplementary model is required, identify which ones are typically used.

The core requirements to cover:

1. **Sentence boundary inference** — segmenting a continuous audio stream into discrete sentences.
2. **Punctuation prediction** — inserting commas, periods, question marks, exclamation marks in the right places.
3. **Filler-word removal** — stripping "um", "uh", "like", "you know", false starts.
4. **Paragraph breaks** — inserting paragraph separators based on pause length or topic shift.

For each: which engines do it natively, which don't, and what the standard drop-in supplementary models are.

Follow-up — 2026-04-14

Paragraph break detection seems to be the least implemented of the four formatting features across both cloud and local models. Is there a technical reason for this, or is it purely a frontend responsibility (emulating a blank line)?

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Dictation formatting features — built-in vs supplementary, across local and cloud models

Question: `../questions/dictation-formatting-built-in-vs-supplementary.md` **Written:** 14/04/26 **Stack:** Desktop live voice typing. Covers cloud streaming ASR (Google, Deepgram, AssemblyAI, Azure, AWS Transcribe, Speechmatics, Soniox, OpenAI `gpt-4o-transcribe` / `whisper-1`), local engines (Whisper variants, NVIDIA Parakeet / Canary / Riva, Moonshine, Vosk / Kaldi, wav2vec2), and common supplementary post-processors (Silero punctuation, `deepmultilingualpunctuation`, NeMo PnC, punctuator2, NLTK / PySBD, LLM rewrite passes).

TL;DR

There are **four formatting behaviours** that make the difference between dictation output you can send directly to an editor and output you need to clean up by hand. Most "modern" models — anything that came out of the post-2022 wave of seq2seq ASR (Whisper, Parakeet, AssemblyAI Universal, Deepgram Nova-2/3, Soniox, Speechmatics Ursa, `gpt-4o-transcribe`) — were trained on punctuated, capitalised data and therefore produce **sentence boundaries + punctuation natively as a side effect of decoding**. Older CTC-only stacks (classic Kaldi chains, Vosk's default models, raw wav2vec2) produce lowercase unpunctuated token streams and genuinely require a bolt-on punctuation model.

Almost no engine does **filler removal** or **paragraph breaks** well out of the box. Deepgram and AssemblyAI are the exceptions (both have native flags). Everyone else needs a supplementary pass — either a silence-gap heuristic you write yourself (for paragraphs) or an LLM rewrite (for filler removal that respects intent).

The practical summary:

- **Sentence boundaries + punctuation:** solved by the model for most cloud providers and modern local engines.
- **Filler removal:** usually a post-processor (LLM or word-list filter).
- **Paragraph breaks:** almost always a frontend heuristic you write yourself, informed by VAD silence timestamps.

The four formatting requirements, restated precisely

1. Sentence boundary inference

The segmentation of a continuous audio stream into discrete sentence-sized units. Downstream, this is what lets the UI capitalise the next word, emit a terminal punctuation mark, or trigger a sentence-level commit.

Worth separating two sub-problems that are often conflated:

- **Acoustic sentence segmentation** — detecting that a sentence *ended*, typically from a pause + final intonation contour. Any VAD-gated pipeline implicitly does this.

- **Textual sentence segmentation** — deciding where sentences begin and end in the emitted text, independent of pauses (a speaker can run two sentences together with no pause and still need a period between them).

Modern seq2seq models do the textual version as part of decoding. CTC models do not.

2. Punctuation prediction

Inserting `.` `,` `?` `!` `:` `;` at the right positions. Often bundled with **capitalisation** (together called "PnC" — punctuation and capitalisation). The hard cases are:

- Question marks — require semantic understanding, not just acoustic ones (many questions in English are not marked by rising intonation, e.g. "you're going where").
- Commas inside a clause — genuinely ambiguous; models vary wildly.
- Apostrophes / hyphens — usually handled by the tokenizer, not a separate prediction.

3. Filler-word removal

Stripping **disfluencies**: "um", "uh", "er", "mm", "hmm", "like" (when used as a filler), "you know", "I mean", plus partial words and false starts ("I was — I was going to"). This is genuinely hard because:

- You don't want to strip legitimate uses ("I like Python" ≠ filler).
- False starts require understanding that a span was replaced by a later span, not just deleting tokens.
- Whether to strip at all is a UX question. Medical / legal transcription typically *keeps* disfluencies. Dictation into a notes app wants them gone.

4. Paragraph breaks

Inserting `\n\n` between topically or rhythmically distinct passages. Two triggering heuristics dominate:

- **Silence-based** — a pause longer than some threshold (2–4 seconds is typical) is treated as a paragraph boundary.
- **Semantic** — topic shift detection, usually via an LLM or a dedicated segmentation model.

Silence-based is what almost every tool that supports paragraphs actually does. Semantic segmentation is rare in live pipelines because it adds latency.

Cloud models — what's built in

Provider / Model	Sentence boundaries	Punctuation	Filler removal	Paragraph breaks
Google Cloud Speech-to-Text (`chirp_2`, `chirp_3`, `latest_long`)	Native (from PnC)	Native — `enable_automatic_punctuation=true`	❌ Not native (Google V2 has limited "spoken punctuation" handling but no filler filter)	❌ Not native — you consume `resultEndOffset` silence gaps and segment yourself

Provider / Model	Sentence boundaries	Punctuation	Filler removal	Paragraph breaks
Deepgram (Nova-2 / Nova-3)	Native	Native — `punctuate=true`	✅ Native — `filler_words=true` returns them tagged, `filler_words=false` omits them; `smart_format=true` also strips most	✅ Native — `paragraphs=true` returns a `paragraphs` array with start/end and sentence-level children
AssemblyAI (Universal / Universal-2)	Native	Native — `punctuate=true`	✅ Native — `disfluencies=false` (default) removes them; `=true` keeps them	✅ Paragraphs via `/transcript/:id/paragraphs` endpoint (post-hoc), plus `auto_chapters` for topical segmentation
Azure Cognitive Services Speech	Native	Native in "Display" output (`ResultReason.RecognizedSpeech`) with `OutputFormat.Detailed`	⚠ Partial — "profanity masking" and "disfluency removal" exist but are documented mainly for conversation transcription, not dictation mode	❌ Not native — frontend heuristic
AWS Transcribe	Native	Native — on by default in real-time streams from ~2021 onward	❌ Not native	❌ Not native (has speaker segments but not paragraphs)
Speechmatics (Ursa / Enhanced)	Native	Native — `punctuation_overrides` config	✅ Native disfluency handling — `transcription_config.enable_partials` returns disfluency markers; full removal needs `operating_point=enhanced` + post-filter	⚠ Partial — exposes speaker changes and segment boundaries, paragraphs you compose yourself
Soniox (2-channel real-time)	Native	Native	✅ Native filler removal in `smart_format` mode	⚠ Partial — silence gaps exposed, you as-

Provider / Model	Sentence boundaries	Punctuation	Filler removal	Paragraph breaks
				semble paragraphs
OpenAI `gpt-4o-transcribe` / `gpt-4o-mini-transcribe`	Native	Native (model emits punctuated casedtext)	⚠ Inconsistent — sometimes removes “um”/“uh”, not prompt-controllable in the transcription endpoint	❌ Not native in streaming mode
OpenAI `whisper-1` (API)	Native	Native	❌ Not native — Whisper keeps most disfluencies in verbatim mode, strips some in “concise” variants	❌ Not native

What the cloud tier looks like overall

The two standouts for **built-in formatting coverage** are Deepgram and AssemblyAI — both expose all four requirements as simple boolean flags, and both are well-documented. If you want a single-API solution where you don’t write any post-processing, pick one of those.

Google, AWS, Azure, OpenAI give you sentence boundaries + punctuation for free and leave filler removal and paragraphs to you. For a custom live-typing frontend that already does its own pause tracking (for VAD-gated commit), paragraph insertion is trivial to add on top — the harder piece to bolt on is filler removal.

Local models — what’s built in

Engine / Model	Sentence boundaries	Punctuation	Filler removal	Paragraph breaks
OpenAI Whisper (`openai-whisper`, `faster-whisper`, `whisper.cpp`, all sizes)	Native (seq2seq decoder was trained on punctuated data)	Native	❌ Keeps disfluencies (verbatim bias); `--suppress_tokens` can mask filler IDs but it’s crude	❌ Not native (but VAD-gated pipelines that chunk by silence give you pseudo-paragraphs for free)
NVIDIA Parakeet-TDT 1.1B / 0.6B-v2	Native	Native — trained with PnC	❌ Not native	❌ Not native

Engine / Model	Sentence boundaries	Punctuation	Filler removal	Paragraph breaks
NVIDIA Canary-1B	Native	Native	❌ Not native	❌ Not native
NVIDIA Riva (ASR service)	Native	**Via a separate PnC model** — Riva ships PnC as a post-ASR component rather than fused in the acoustic model	❌ Not native	❌ Not native
Moonshine (tiny / base / small)	Native	Native (trained with PnC)	❌ Not native	❌ Not native
Distil-Whisper	Native (inherits from Whisper)	Native	❌ Not native	❌ Not native
Vosk (default small / large models)	❌ Not native — lowercase unpunctuated output	❌ Not native	❌ Not native	❌ Not native
Kaldi chain / nnet3 models (most pretrained)	❌ Not native	❌ Not native	❌ Not native	❌ Not native
wav2vec2 (CTC fine-tunes from Hugging Face, e.g. `facebook/wav2vec2-large-960h`)	❌ Not native — raw CTC, no casing, no punctuation	❌ Not native	❌ Not native	❌ Not native
Silero STT (community models)	Partial — some community variants ship PnC, many don't	Partial	❌ Not native	❌ Not native

The practical divide for local models

There's a clean split:

- **Anything seq2seq trained on modern web data after ~2022** (Whisper, Parakeet, Canary, Moonshine) gives you sentence boundaries + punctuation for free. Filler removal and paragraphs are always on you.

- **Anything CTC-based and trained before PnC was routine** (Vosk, raw wav2vec2, older Kaldi recipes) gives you none of the four requirements. You need a full post-processing chain.

Riva sits oddly in the middle — it's modern and high-quality, but NVIDIA ships PnC as a separate deployable component rather than baking it into the acoustic model, so you compose it yourself at the server layer.

Supplementary models — when you need a post-processor

For punctuation + casing on an unpunctuated stream

- **`deepmultilingualpunctuation`** (Oliver Guhr's model on Hugging Face, based on XLM-RoBERTa). Drop-in Python library. Supports 12+ languages. The most common pick for wav2vec2 / Vosk pipelines.
- **Silero punctuation + capitalisation** (`silero-models` on GitHub / PyTorch Hub). Tiny (~30MB), CPU-friendly, multilingual. Used widely in privacy-focused / offline setups.
- **NVIDIA NeMo punctuation & capitalisation** (`nemo_asr.models.PunctuationCapitalizationModel`). Heavier but higher quality; the natural choice if you're already on a NeMo/Riva stack.
- **punctuator2** (Tilk). Older (2016), still works for English. Light and fast but lower ceiling than transformer-based ones.
- **LLM rewrite pass** (local: Llama-3-8B-Instruct, Phi-3, Qwen2.5; cloud: `gpt-4o-mini`, `claude-haiku-4-5`, Gemini Flash). Highest quality but highest latency and cost. Better for batch/post-editing than streaming.

Rule of thumb: for a live typing UI, Silero is the lightest option that still produces readable output. For a post-commit "polish" pass, an LLM is the right tool.

For sentence segmentation from already-punctuated text

- **NLTK `sent_tokenize`** — Punkt model; good for English prose, confused by things like "Dr. Smith."
- **PySBD** — rules-based, handles edge cases (abbreviations, decimals) much better than NLTK.
- **spaCy** `Doc.sents` — requires a loaded model, slightly slower, but accurate on modern prose.

These are only relevant if you need explicit sentence spans (for sentence-level commits, citation, etc.) — the text itself already has the boundaries once punctuation is in.

For filler / disfluency removal

This is the hardest category because word-list approaches over-strip and model-based approaches need context.

- **Word-list filter** — regex against `\b(um|uh|er|hmm|mm|like|you know|I mean)\b` with hedges. Fast, brittle, over-strips legitimate uses of "like" and "you know".
- **Disfluency detection models**:
  - **`Hugging Face` disfluency models** — e.g. `bhadresh-savani/bert-base-uncased-emotion` variants fine-tuned on Switchboard disfluency tags. Small and local-runnable.

  - **AssemblyAI's `disfluencies` flag** — if you're already on AssemblyAI, this is the right tool. They trained on conversational data and handle false starts reasonably.
- **LLM rewrite with a prompt like "clean up disfluencies but preserve meaning"** — currently the only approach that reliably handles false starts ("I went to the — I went to the store" → "I went to the store") rather than just deleting tokens. GPT-4o-mini / Claude Haiku / Gemini Flash are all capable; local 7B-8B instruction-tuned models work too for simpler cases.

For a dictation tool where the user is *prompting* an AI (the common modern case), filler removal is often unnecessary — the downstream LLM ignores fillers anyway. Only strip them if the output goes directly to a human reader.

For paragraph breaks

- **Silence-gap heuristic** — the 5-line implementation everyone ships: emit `\n\n` whenever a pause exceeds N seconds (tunable, 2.5s is a reasonable default). Requires VAD or per-word timestamps from the ASR.
- **LLM rewrite** — prompt with "insert paragraph breaks at topic shifts, preserving wording". Higher quality but adds noticeable latency; better as a post-stop polish than a streaming operation.
- **Sentence-embedding topic shift** (e.g. `sentence-transformers/all-MiniLM-L6-v2` + cosine distance between adjacent sentence embeddings) — niche, mostly research-grade; rarely worth the complexity for a typing UI.

The honest recommendation: start with the silence-gap heuristic. Only move to an LLM rewrite if users specifically complain that paragraphs land in the wrong place.

Putting it together — what a "full coverage" pipeline looks like

Three concrete configurations that hit all four requirements, from lightest to heaviest:

Config A: Single-API cloud (lowest friction)

```
Deepgram Nova-3 (streaming)
  │
  ├─ punctuate=true          → punctuation + sentence boundaries
  ├─ filler_words=false      → filler removal
  ├─ smart_format=true       → ITN + casing + punctuation polish
  └─ paragraphs=true         → paragraph segmentation
```

One API call does all four. AssemblyAI is the equivalent with `punctuate` / `disfluencies=false` / `format_text` / `paragraphs`.

Config B: Local Whisper + silence heuristic + optional LLM polish

```
faster-whisper (small or medium)
  │
  ├─ native → sentence boundaries + punctuation
  ├─ VAD silence gaps > 2.5s → paragraph breaks (frontend code)
  └─ post-stop LLM pass (optional) → filler removal + polish
```

Adds a post-processor only for filler removal, and only when the user asks for it. The cleanest local-first setup in practice.

Config C: CTC engine (Vosk / wav2vec2) + full bolt-on chain

```
Vosk / wav2vec2 (streaming)
  │
  ├─ Silero punctuation    → punctuation + casing
  ├─ silence-gap heuristic → paragraph breaks
  └─ LLM rewrite pass       → filler removal + polish
```

Heaviest, most brittle, but also the most customisable. Only worth it if you have a hard constraint (GPU-free CPU inference, specific language support, air-gapped deployment) that rules out the seq2seq engines.

Bottom line

- **Sentence boundaries + punctuation** are solved at the model layer by anything modern. If your engine doesn't do them, the quickest win is to switch engines, not to add a post-processor.
- **Filler removal** is genuinely model-dependent. Deepgram and AssemblyAI handle it; everyone else leaves it to you. For most dictation-to-AI use cases, leave it alone — the LLM ignores fillers and over-stripping causes real meaning loss.
- **Paragraph breaks** are almost always the frontend's job, even when the API says it supports them. A silence-gap timer in the VAD layer produces better results than blind reliance on a cloud provider's `paragraphs` output, because *you* know what pause length means "new paragraph" for *your* user.

The surprising result from surveying these is how little the built-in coverage varies among engines aimed at dictation specifically (Deepgram, AssemblyAI, Soniox): they all cover the four requirements. The gap is between dictation-optimised products and general-purpose ASR APIs (Whisper, Google, AWS) that produce high-quality transcripts but expect you to assemble the formatting layer yourself.

Follow-up — 2026-04-14

Why paragraph break detection is the least implemented of the four

Short answer: it's not a frontend-vs-backend split so much as a **problem-definition** split. Sentence boundaries, punctuation, and filler removal all have **objective training targets** — there's a correct answer you can label in a dataset. Paragraph breaks don't. What counts as a paragraph is a stylistic and domain-specific judgment, and models don't learn it reliably because there's no consistent supervision signal to learn from.

Pulling that apart into the actual technical reasons:

1. No canonical ground truth in training data

The three "solved" features come with abundant labelled data:

- **Sentence boundaries / punctuation** — every piece of written text in the training corpus is a label. Books, Wikipedia, news, forum posts — they all have terminal punctu-

ation and sentence structure already baked in. The model learns PnC essentially for free from the web-scale text side of its training pairs.
- **Filler removal** — labelled disfluency corpora exist (Switchboard, Fisher, plus modern conversational datasets like AMI and ICSI) with explicit `[DISFLUENCY]` / `[FILLED_PAUSE]` / `[PARTIAL]` tags annotated by humans. Small, but consistent enough to train on.
- **Paragraph breaks** — the label is *author intent about visual formatting*. The same spoken passage could legitimately be one paragraph or three depending on genre (blog post vs chat message vs legal brief), on the author's personal style, and on the rendering surface (Twitter vs a Word document). You cannot collect a corpus where annotators agree on where paragraph breaks "should" go with anything like the agreement you'd get for sentence endings.

Models don't learn well from inconsistent labels. So the feature gets left out of the training objective, and it shows up as a gap at inference time.

2. The acoustic signal for paragraphs is weak

Pauses, intonation resets, and breath noise all correlate with paragraph breaks, but none are reliable:

- A 3-second pause can be "thinking about the next sentence" or "starting a new paragraph" or "phone notification distracted me" — same audio, different intent.
- Prosodic paragraph-ending cues (pitch reset, final lowering) exist in careful read speech but are largely absent in spontaneous dictation.
- Speakers often don't signal paragraph intent at all — they just produce a long run of connected sentences and decide visually later, when they see the text.

So even if you wanted a model to predict paragraph breaks acoustically, the signal is noisy enough that a simple silence threshold performs as well as a learned predictor, with none of the training cost.

3. Paragraphs are a rendering-surface concept, not a text concept

A paragraph is "a chunk of text separated from other chunks by whitespace". That definition only makes sense relative to a rendering layer. Streaming ASR emits a token stream — it has no concept of a rendered surface.

Compare:

- Sentences end in `.` — the punctuation is *in the text itself*, unambiguous across surfaces.
- Paragraphs end with a newline — but `\n\n` means "new paragraph" in Markdown, `<p>` in HTML, a new `<p:Paragraph>` run in a Word document, pressing **Enter twice** in a plain textbox. In some contexts (terminal prompts, single-line chat inputs) a newline means *submit*, not "new paragraph".

An ASR pipeline that doesn't know where its output is going has no principled way to emit the right break character. Most pipelines solve this by leaving paragraph breaks entirely to whatever owns the cursor — i.e. the frontend, which does know.

This is the kernel of truth in the "is it just a frontend thing?" intuition. Strictly, paragraph emission requires a **two-line handoff**:

1. The ASR or VAD layer tags "long silence here" (timestamp + duration).
2. The frontend decides what that means in the current context — `\n\n`, two Enter presses, a new message, nothing at all.

Step 2 genuinely has to be at the frontend because only the frontend knows what app has focus.

4. The live-streaming constraint

Paragraph break detection is least harmful as a *retrospective* decision — you can look at the whole transcript and decide where paragraphs go. But live dictation is irrevocably *prefix-only*: once you've emitted text, you can't go back and restructure it without a visible jump. So the model would have to decide "new paragraph starts here" in a single forward pass, with only the audio up to that point, and commit to the decision before knowing what comes next.

Contrast with punctuation: a streaming model can emit a period with reasonable confidence from a falling intonation and a short pause, because the downside of a wrong period is small (one mis-punctuated sentence). A wrong paragraph break is more jarring — a large visual artefact in the middle of what should be connected prose.

So even engines that *could* emit paragraph breaks live tend not to, because the cost of false positives is high and the user can add a break manually with one keystroke.

5. Users already have a mechanism

For sentence-level punctuation, the "manual" alternative (typing a period) is laborious and slow. Automation pays off.

For paragraph breaks, the manual alternative (press Enter twice, or say "new paragraph" as a command) is fast and reliable. Several dictation products — Dragon NaturallySpeaking historically, and Wispr Flow more recently — let the user say `"new paragraph"` or `"new line"` as an explicit command. That sidesteps the detection problem entirely, costs almost nothing in UX, and is more predictable than any automatic detector.

When an explicit user command gives you 100% accuracy for one spoken phrase per paragraph, the incentive to build a detection model collapses.

So: frontend thing, or deeper?

Both, but in a specific order:

- It's **deeper than a frontend quirk** — there are real training-data, signal-quality, and commit-latency reasons no one ships great automatic paragraph detection.
- And it's **also a frontend thing** — because the rendering surface genuinely has to decide what break character to emit, a frontend step is unavoidable even when the ASR does signal silence gaps.

The typical production pattern, across every dictation tool that handles paragraphs at all:

```
ASR / VAD → "pause of N ms at timestamp T" → frontend rule:
  if N > 2500ms  → emit \n\n  (or the surface-appropriate break)
  else if N > 800ms → emit \n    (soft break, rare)
  else            → no break
```

Plus an explicit voice command `"new paragraph"` as an override. That's it. The entire feature lives in ~20 lines of frontend code, which is why no backend model vendor bothers to own it properly.

The honest rephrasing of your observation: paragraph break detection is “the least implemented” because it’s **the one feature where a trivial frontend heuristic works as well as anything a model could realistically learn**, and the rendering-surface dependency means it has to live on the frontend anyway. So the model vendors skip it and leave it to whoever owns the cursor.

Topic: `focus-loss-during-dictation`

Prompt

Focus loss during dictation

Asked: 14/04/26 **Slug:** `focus-loss-during-dictation` **Response:** `ideas/focus-loss-during-dictation.md`

The question

When using live voice typing that's supposed to insert text at the cursor position, it's very easy — while waiting for the transcription to arrive — to move into another workflow context (switch windows, click into something else).

The standard solution is to also place the text on the clipboard. But what other alternatives are there? For example: when you return to the original window, the text is pasted where it was originally intended to be.

What's the design space here?

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Handling focus loss between dictation start and transcript arrival

Question: `questions/focus-loss-during-dictation.md` **Written:** 14/04/26 **Stack:** Desktop live voice typing (real-time speech-to-text injecting into the focused window). Author's primary target is Ubuntu 25.10 / KDE Plasma / Wayland, but the patterns generalise to X11, macOS, and Windows.

TL;DR

Cursor-injection dictation has an inherent race: the user starts speaking with one window focused, but by the time the transcript is ready to inject, focus may have moved. The standard mitigation is to also place the text on the clipboard so nothing is lost. Better designs go further — capture the *target window + caret* at dictation start, then either re-inject when the user returns to that window, or surface the pending text in a non-modal overlay the user can commit deliberately. On Wayland, what's actually implementable is sharply constrained by the compositor's input/focus security model, and that should drive the choice of pattern.

Background

A live voice typing tool's job is to take audio and put text where the cursor is. The "where the cursor is" part is the unstable bit:

- ASR has latency (often hundreds of ms; seconds for longer utterances or chunked decoding).
- The user does not freeze while waiting. They alt-tab to read something, click a notification, scroll a doc, switch to a chat window to grab a quote.
- When the transcript arrives, the focused window is no longer the one the user was talking *to*. The tool's synthetic keystrokes land in the wrong place — sometimes harmlessly (a search box), sometimes destructively (a terminal, an IDE shortcut binding, a chat that auto-sends on Enter).

This is not an ASR problem. It is a UX/state-management problem at the boundary between the dictation tool and the windowing system. Every serious live-typing tool has to take a position on it.

The patterns

Pattern 1 — Inject blindly into whatever is focused now

The naive default. Tool listens, transcribes, calls `xdotool type` / `ydotool type` / `wtype` / Windows `SendInput` / macOS Accessibility into the currently focused window.

- **Pros:** trivial to implement; works fine for users who hold still.
- **Cons:** silently destructive when focus has moved. The user has no signal that anything went wrong until they look at the wrong window and find a sentence pasted into it.
- **Verdict:** unacceptable as a sole strategy. Usable only if combined with a strong "you must hold focus" affordance (push-to-talk + visible recording overlay anchored to the target window).

Pattern 2 — Clipboard fallback (the "standard mitigation")

Inject *and* copy to the clipboard. If the keystrokes go to the wrong window, the user can undo and paste manually into the right one.

- **Pros:** zero loss of content; one extra Ctrl+V recovers. Implementable on any platform without privileged APIs.
- **Cons:** clobbers the user's existing clipboard contents (a real cost — they may have had something important copied). Recovery is manual: user must notice, undo the wrong injection, switch windows, paste. The wrong-window injection still happens.
- **Variants worth considering:**
  - **Clipboard-only mode** (no injection at all): the tool never types, it only copies; the user always pastes themselves. Eliminates the wrong-window problem at the cost of one extra keystroke per utterance. Some users prefer this as the *default* once they've been burned a few times.
  - **Clipboard ring / save-and-restore:** snapshot the existing clipboard at dictation start, place the transcript on the clipboard for N seconds or until next clipboard event, then restore. Reduces collateral damage. `clipnotify` / `wl-paste --watch` make this tractable.

Pattern 3 — Capture-target, defer-inject

At the moment dictation *starts*, capture:

- The focused window's identifier (X11 window ID, Wayland — see caveats; macOS AXUIElement; Windows HWND).
- Where possible, the caret position within that window (rarely retrievable cross-app; usually you settle for "the window").

Then, when the transcript is ready:

- If the captured window is still focused: inject normally.
- If not: do *not* inject into the wrong window. Instead, hold the transcript in a pending state and either:
  - **Re-focus the original window automatically and inject** (most aggressive — may yank the user out of what they're now doing; usually a bad idea).
  - **Inject when the user next focuses the original window** (passive; the transcript "lands" the moment they return). This is the option the question asks about and it's the most respectful of user intent.
  - **Show a non-modal overlay** ("Transcript ready — click here, press hotkey, or refocus original window to insert") and let the user pick.

This is the strongest pattern and the one most worth building toward. It treats the *target* as part of the dictation session, not just "wherever focus happens to be".

Pattern 4 — Anchored overlay / inline preview

Show the streaming transcript in a floating overlay anchored near the original caret position. Nothing is committed to the underlying app until the user explicitly accepts (Enter, hotkey, click). Focus changes during dictation are harmless because nothing is being typed into anything yet.

- **Pros:** removes the race entirely; user sees what they said before it goes anywhere; trivial editing/correction surface.
- **Cons:** breaks the "transparent typing" illusion — feels like a separate tool, not a keyboard replacement. Adds a commit step. Anchor positioning is hard on Wayland.
- **Where it shines:** longer utterances, dictation into apps with no undo, anything safety-critical.

Pattern 5 — Confirmation-on-mismatch

Hybrid. Inject only if the focused window matches the captured target. On mismatch, fall back to overlay or clipboard with a notification ("Focus changed — transcript saved to clipboard / pending in tray"). Cheap to implement, sharply reduces silent failures.

What's actually implementable on Wayland

The platform constrains the design space more than the design space constrains the platform.

- **X11:** every part of patterns 3/4/5 is feasible. `xdotool getactivewindow` gives you a stable window ID at start; you can re-check it later, you can warp focus back, you can inject with `xdotool type --window <id>` directly into a target window even when it isn't focused (with caveats per app).

- **Wayland (KDE/Plasma, GNOME):** by design, no client can globally enumerate windows or know which client owns focus. There is no portable equivalent of `getactivewindow`. Synthetic input via `wtype` requires a virtual-keyboard protocol the compositor must allow; `ydotool` works at the uinput level but injects into *whatever is focused now*, which is exactly the problem pattern 3 is trying to solve.
  - KWin (KDE) exposes some of this via D-Bus / KWin scripting, but it's KDE-specific.
  - The XDG desktop portals do not currently provide a "remember target window and re-inject later" primitive. This is a real gap.
  - Practical consequence: on Wayland, **pattern 4 (overlay) is often the only honest option** for the deferred-inject case, because you can't reliably distinguish "same window" from "different window" later. The overlay sidesteps the whole problem by never injecting at all until the user explicitly commits.
- **macOS:** Accessibility APIs give you AXUIElement references to text fields; with permission, you can read and write them directly. Patterns 3 and 4 are both clean.
- **Windows:** UI Automation + HWND tracking make patterns 3 and 5 straightforward.

A recommended layered default

For a tool that has to work across desktops, layer the patterns:

1. **Always** copy the transcript to the clipboard (pattern 2), with a clipboard save-and-restore variant if feasible. This is the floor — content is never lost.
2. **Capture the target window at dictation start** where the platform allows (X11, macOS, Windows). On Wayland, capture whatever the compositor will give you (sometimes nothing).
3. **At injection time**, check whether focus is still on the captured target.
   - Same target → inject inline.
   - Different target → do **not** inject. Show a non-modal overlay/notification ("Transcript ready — refocus or click to insert"). Inject automatically when the captured target regains focus, with a short timeout (e.g. 30 s) after which the transcript is left only on the clipboard and the pending state is dropped.
4. **Offer pattern 4 (anchored overlay with explicit commit)** as a per-session or per-app mode, for users who want zero surprise injection.

This gives the user three escape hatches — automatic re-injection on return, clipboard paste, overlay click — and removes the silent-wrong-window failure mode entirely.

Verification

A focus-loss strategy is working if all of the following hold during deliberate testing:

- Start dictation in window A, immediately switch to window B, finish speaking. **No text appears in B.**
- Return to window A. The transcript appears at the caret (or the user can trigger it with one action).
- Clipboard contains the transcript as a fallback regardless.
- If a pre-existing clipboard value was present, it is either preserved (save-and-restore) or the user has been warned.
- On Wayland, where target tracking is unavailable, the tool degrades to the overlay/clipboard path rather than blindly injecting into B.

Caveats / things that can go wrong

- **Auto-injection on return can still fire into the wrong place** if the user returned to window A but moved the caret elsewhere within A (e.g. clicked a different field). Mitigation: also verify the focused *widget* matches, where the platform exposes that; otherwise prefer the overlay/click-to-commit path.
- **Re-focus warping** (forcing window A back to the front to inject) is hostile UX and prone to fighting the user's current action. Avoid except as an explicit user choice.
- **Clipboard managers** (KDE Klipper, CopyQ, etc.) will record every transcript as a clipboard history entry unless the tool tags clipboard writes as transient. KDE's Klipper respects the X-KDE-Klipper-Selection-Manager hint; check the equivalent for the user's clipboard manager.
- **Password fields** should never receive auto-injection. Detect where possible (macOS AX, Windows UIA expose this; Linux generally does not) and refuse to inject; force the overlay path.
- **Terminals** interpret some characters specially (Ctrl-sequences, multi-line paste behaviour). Bracketed paste mode helps. Many users will want a terminal-specific mode that only ever uses the clipboard.
- **Wayland compositor differences:** what works on KWin won't work on Mutter or Sway. Don't promise pattern 3 cross-compositor without per-compositor implementations.
- **Latency-vs-correctness:** the longer the deferred-inject window, the more likely the user has moved on mentally. A 30 s pending overlay is reasonable; a 5 minute one is just clutter.

References

- `wtype` — Wayland virtual-keyboard typing utility: https://github.com/atx/wtype
- `ydotool` — uinput-based input injection (Wayland-compatible, root/uinput required): https://github.com/ReimuNotMoe/ydotool
- `xdotool` — X11 keyboard/mouse/window automation: https://www.semicomplete.com/projects/xdotool/
- KWin scripting / D-Bus interface (KDE Plasma window introspection): https://develop.kde.org/docs/plasma/kwin/
- XDG Desktop Portal — Global Shortcuts and RemoteDesktop portals (closest thing to a cross-compositor input API): https://flatpak.github.io/xdg-desktop-portal/
- nerd-dictation — minimal local dictation tool, useful reference for the inject-on-finalise pattern: https://github.com/ideasman42/nerd-dictation
- Talon Voice — production voice-control system, exemplar of pattern 4 (overlay + explicit commit) and target-aware behaviour: https://talonvoice.com/

Topic: `handy-inference-vs-typing-delay`

Prompt

When Handy prints the transcript over 10–20 seconds, is that real-time inference or an artificial frontend delay?

Asked: 14/04/26 **Slug:** `handy-inference-vs-typing-delay` **Response:** `ideas/handy-inference-vs-typing-delay.md`

The question

Explore the determinants of the speed of inference in Handy, which records in one shot and then prints the transcript to the screen. For a few paragraphs of speech, that print stage can take 10–20 seconds.

Is that delay artificial — created by the way the frontend chooses to print words to the screen — or is the inference output being displayed in real time, so that as each word appears it reflects speech being transcribed live?

Related

- `batch-vs-chunked-inference-accuracy` — the accuracy side of batch-on-stop inference.
- `streaming-vs-batch-injection` — separates the recognition-mode axis from the injection-mode axis.
- `inference-cadence-and-sentence-entry` — the three intervals that govern how live-feeling dictation feels.

Follow-up questions

- 14/04/26 — What is “set clipboard, auto-fire Ctrl+V, done” called, and how is it implemented on Wayland with tools like ydotool?
- 14/04/26 — In Handy’s “Paste Method” dropdown, does “paste” mean automatic injection or does it require manual Ctrl+V?

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Handy's 10–20 second print delay is almost entirely keystroke-injection pacing, not inference

Question: `questions/handy-inference-vs-typing-delay.md` **Written:** 14/04/26 **Stack:** Handy (open-source desktop dictation, Rust + Tauri) running Parakeet-TDT or Whisper locally. Ubuntu 25.10 / KDE / Wayland. Local GPU inference.

TL;DR

What you're watching is **not** streaming inference. Handy (like most "push-to-talk → transcribe-on-release" tools) is architecturally a **batch** pipeline: audio in → one offline inference pass → finished transcript string → typed out character-by-character into the focused window. For a few paragraphs on a modern GPU, Parakeet-TDT finishes inference in **~0.5–3 seconds** total. The remaining 10–20 seconds you observe is the **keyboard-injection layer** — the tool deliberately paces synthetic keystrokes at ~5–30 ms per character to avoid dropped events in target apps. On a 1500-character transcript at 15 ms/char, that's ~22 s of visible "typing". So yes, **the delay is essentially artificial**, and the mechanism is pacing of the input-synthesis layer, not live transcription.

Background

Handy follows a well-known pattern for desktop dictation:

1. Hold PTT key → record audio to a buffer.
2. Release PTT key → pass the full buffer to an offline ASR model (Parakeet-TDT, Whisper, Moonshine-offline).
3. Model returns a finished transcript string.
4. Tool synthesises keystrokes that type that string into whatever window has focus.

Steps 2 and 4 are what the user experiences as "the delay". They are **independent stages** that can each contribute latency, and it matters which one dominates because the fixes are completely different.

The user's intuition — "am I watching the model think, or am I watching the UI deliberately feed characters slowly?" — is the right axis to separate. They have different scaling behaviour:

- **Inference time** scales roughly with audio duration (GPU compute per second of audio).
- **Injection time** scales with character count (keystroke count × per-keystroke delay).

A quick back-of-envelope test distinguishes them: speak a long, slow monologue (lots of audio, few words) vs. a short, information-dense burst (little audio, many words). If the delay tracks audio duration, it's inference. If it tracks character count, it's injection.

Stage 1: the inference itself

Handy supports multiple backends, but the practical defaults are:

Model	Engine / runtime	Approx RTF on consumer GPU	Approx wall time for 60 s of audio
Parakeet-TDT 0.6B v2	ONNX / `parakeet-rs` / NeMo	~0.02–0.05× real-time (i.e., 20–50× faster than real-time) on RTX 3060+	~1.2–3 s
Whisper large-v3 (ggml via whisper.cpp)	CPU: 0.3–0.8× RTF. GPU (CUDA): 0.05–0.15× RTF.	~4–50 s on CPU, ~3–9 s on GPU	
Whisper medium (whisper.cpp)	CPU ~0.2× RTF, GPU ~0.03×	~2–12 s	
Moonshine-base (ONNX)	~0.05× RTF on CPU, faster on GPU	~1–3 s	

RTF = "real-time factor" = processing_time / audio_duration. Smaller is faster.

Parakeet-TDT on a GPU is the "effectively instantaneous" tier — you dictate for a minute, and the model hands back the full transcript in 1–3 seconds. A "few paragraphs" of spoken content is maybe 30–60 s of audio, which puts inference comfortably under 3 s on GPU, under ~8 s on a reasonable CPU-only build.

If you are seeing 10–20 s before the first character appears, and nothing appears until the end, that could be inference. But if characters start appearing quickly and the "delay" is characters rolling out across the screen, inference is almost certainly already done.

How to confirm inference time directly

Handy logs inference duration to its console / log file (Handy's dev builds print this; release builds may require turning on verbose logging). Alternatively, run the model under the same runtime outside Handy:

```
# Parakeet via nemo (or parakeet-mlx / parakeet-rs)
time python -c "import nemo.collections.asr as nemo_asr; m =
nemo_asr.models.ASRModel.from_pretrained('nvidia/parakeet-tdt-0.6b-v2');
print(m.transcribe(['/tmp/sample.wav']))"

# Whisper via whisper.cpp
time ./main -m models/ggml-medium.bin -f /tmp/sample.wav
```

Compare the `real` time to the audio length. If it's <10% of the audio length, inference isn't your bottleneck.

Stage 2: the injection — this is almost certainly the real culprit

Desktop dictation tools have to choose an input-synthesis strategy. On Linux/Wayland specifically, the options are:

Mechanism	How it works	Speed characteristics
Synthetic keystrokes via virtual keyboard (`ydotool`, `wlrctl`, `enigo` with `uinput`)	Tool emits KEY_DOWN/KEY_UP events per character via `uinput`. Each character = 1–3 kernel events + a short sleep.	Paced. Typically 5–30 ms/char.
Input method / IME injection (IBus, Fcitx protocol)	Tool pretends to be an IME, commits a string in one shot.	Nearly instantaneous; complex to implement; some apps ignore IME.
Clipboard + paste	Transcript → clipboard → synthesise Ctrl+V.	Instant once the clipboard is set.
X11 XTest / XSendEvent	On X11 only. Fast, but Wayland sessions don't expose it for arbitrary windows.	Instant-ish, but X11-only.

Most cross-platform dictation tools, including Handy, default to the "synthesise keystrokes one at a time" path. The reason is reliability — it works in every focused app (terminals, browsers, IDEs, chat boxes, address bars) without per-app quirks. The cost is that it is **paced**.

Why it's paced at all

Without a delay between keystrokes, you hit several failure modes:

1. **Dropped events** — target apps with expensive per-keystroke work (IDEs running syntax analysis, terminals with bracketed-paste handlers, web apps with oninput listeners) drop characters if they arrive faster than the event loop can process.
2. **Auto-completion interference** — editors with autocomplete or snippet engines interpret mid-stream characters and inject unwanted text.
3. **Modifier-key sequencing** — capital letters, accented characters, and non-ASCII glyphs need SHIFT held, dead-key compositions, or Unicode code-point entry. These require multi-event sequences that can't reliably run back-to-back without a hold time.
4. **`uinput` syncing** — the kernel batches events; too-rapid submission drops some devices.

So every mature tool settles on a **per-character delay**, usually configurable. Typical defaults:

Tool	Default per-key delay
`ydotool`	~12 ms
`xdotool type --delay`	default 12 ms
`enigo` (Rust crate, used by many Tauri apps including Handy)	no built-in delay, but applications using it typically add a sleep
AutoHotkey `Send` modes	`SendInput` ~0 ms, `SendEvent` 10 ms, `SendPlay` variable

The math of the visible delay

At 15 ms/character:

Transcript length	Visible typing time
200 chars (~40 words)	3 s
500 chars (~100 words)	7.5 s
1000 chars (~200 words, a short paragraph)	15 s
1500 chars (~300 words, a few paragraphs)	22.5 s
3000 chars (~600 words, a page)	45 s

This is the shape that matches the user's observation exactly. "A few paragraphs → 10–20 s" is the injection layer doing its paced thing, not the model thinking.

So is it "artificial"?

It's **designed**, not accidental — but from the user's point of view, yes: the delay between "inference complete" and "all characters on screen" exists purely because the tool chose keystroke synthesis with pacing as its injection strategy. Three things are true simultaneously:

1. **You are not watching the model transcribe live.** Inference finished seconds ago; the string is sitting in memory waiting to be typed.
2. **The pacing is deliberate and reasonable by default.** Removing it would cause dropped characters and mangled output in many target apps.
3. **It is still the dominant contributor to the perceived latency** for any transcript longer than a sentence. On a GPU with Parakeet, inference is 1–3 s and injection is 10–20 s.

What you can do about it

If the goal is to make the output *appear* faster without rewriting Handy:

Switch injection mode to paste

Paste is instant. Many dictation tools expose a "paste instead of type" setting, sometimes called "fast mode" or "clipboard mode".

- Pros: transcript appears in one go, ~0 ms after inference.
- Cons: overwrites the clipboard (mitigable with save/restore); some apps block programmatic Ctrl+V; loses the "typing character-by-character at the cursor" UX that matches keyboard input.

Check Handy's config for `paste` / `inject_mode` / similar; if absent, it's a reasonable feature request.

Lower the per-keystroke delay

If Handy exposes a keystroke delay (or uses `enigo` / `ydotool` with a configurable interval), drop it. Going from 15 ms → 3 ms per character cuts a 20-second "typing" to 4 seconds. Watch for dropped characters in your target apps; dial it back up if you see them.

Use an IME-based injection

Fcitx/IBus commit-string injection puts the full transcript into the IME's composition buffer in one go. This is how Japanese/Chinese input methods work natively, and it's the cleanest mechanism on Wayland. Not many cross-platform tools implement it (complexity, per-desktop wiring), but if Handy ever grows an IME backend, this is the path to near-instant injection without a clipboard round-trip.

Or: switch the entire pattern to streaming

If you want to actually watch words appear as they are transcribed (not as they are typed out of a finished string), that's a **different architecture** — streaming ASR (Deepgram, AssemblyAI Universal-Streaming, Parakeet-TDT-streaming, Moonshine-streaming) where interim results arrive every ~200–300 ms during dictation. See `whisper-vs-streaming-asr-for-dictation` and `inference-cadence-and-sentence-entry` for that shape.

Verification — quick test you can run

1. **Time-to-first-character test.** Dictate a long passage and watch the time from PTT release to the first character landing. If that gap is short (<3 s) but the full transcript takes 15 s to finish typing, injection is the bottleneck. If time-to-first-character is long and then the rest lands quickly, inference is.
2. **Character-count scaling.** Dictate (a) 30 s of slow speech vs. (b) 10 s of very dense speech of roughly the same word count. If both take similar total "print" time, it's injection. If (a) takes ~3× longer, it's inference.
3. **Log inspection.** Run Handy with verbose logging (or check its log file) for an "inference time" or "transcribe ms" line. That number is almost always much smaller than the observed total delay.

References

- `ydotool` default delay and config: github.com/ReimuNotMoe/ydotool
- `xdotool type --delay` default: man.archlinux.org/man/xdotool.1
- `enigo` Rust crate (commonly used by Tauri desktop apps for key injection): github.com/enigo-rs/enigo
- Parakeet-TDT 0.6B v2 model card (RTF benchmarks): huggingface.co/nvidia/parakeet-tdt-0.6b-v2
- `whisper.cpp` benchmarks: github.com/ggerganov/whisper.cpp#benchmarks
- Moonshine: github.com/usefulsensors/moonshine
- Wayland input synthesis background — `wlroots` virtual-keyboard protocol: wayland.app/protocols/virtual-keyboard-unstable-v1
- Related topics:
  - `batch-vs-chunked-inference-accuracy` — why a batch pipeline like Handy can be a deliberate accuracy choice.
  - `streaming-vs-batch-injection` — the injection axis on its own.
  - `inference-cadence-and-sentence-entry` — what the three "live-feel" intervals are for true streaming systems.
  - `focus-loss-during-dictation` — why paced injection is a liability when focus shifts mid-type.

Caveats

- Exact defaults depend on the Handy version and which backend is compiled in (Parakeet vs Whisper vs Moonshine) — run the time-to-first-char test on your own build to confirm the split.
- The 5–30 ms/char range is typical of `uinput`-style keystroke synthesis; some builds use shorter delays with batching or longer delays for reliability. If Handy exposes a config key for injection speed, its value is the definitive answer.
- On X11 (not Wayland), XTest-based injection can be much faster (sub-millisecond per keystroke) because events go straight to the X server rather than through `uinput`. If you run Handy under Xwayland or an X11 session, the pacing math above changes.
- "Inference in 1–3 s" assumes a GPU backend. CPU-only whisper.cpp on Whisper-medium/large can reach inference times that genuinely contribute 5–15 s to the observed delay.

Follow-up — 2026-04-14: one-shot automatic injection on Wayland

Question: ydotool paced typing feels like the worst of both worlds — you don't get live streaming, and you still wait through a fake typing animation. Manual paste isn't acceptable either. What is "set the clipboard, fire Ctrl+V automatically, done" called, and how do you implement it on Wayland?

Names for the pattern

There isn't one universal term; these are the ones used interchangeably in dictation tooling:

- **Paste-based injection** / **paste injection** — most common name in open-source STT tools.
- **Clipboard injection** / **clipboard paste** — same thing, viewed from the clipboard side.
- **Auto-paste** / **programmatic paste** — user-facing label (often a config toggle: `use_paste = true`, `fast_mode`, `instant_mode`).
- **Bulk injection** — generic term contrasting with keystroke-by-keystroke synthesis.
- **Commit-string injection** — specifically the IME route (next section); distinct from clipboard paste because it doesn't touch the clipboard at all.

If you grep a dictation tool's source or config for any of these terms, you're looking at the same mechanism.

Two real mechanisms on Wayland

(1) Clipboard + synthetic Ctrl+V — the ubiquitous path.

This is what almost every "fast mode" dictation tool actually does. Three steps:

```
# 1. Put the transcript on the clipboard
wl-copy "<transcript>"

# 2. Synthesise Ctrl+V into the focused window
ydotool key 29:1 47:1 47:0 29:0   # LCTRL down, V down, V up, LCTRL up
#   or, with key names: ydotool key ctrl+v
```

```
# 3. (Optional) Restore previous clipboard contents after a short delay
```

You already have `ydotool` installed — add `wl-clipboard` (`sudo apt install wl-clipboard`) and you have all the pieces. Total injection time: ~10–50 ms regardless of transcript length.

Gotchas that make this "not quite free":

- **Clipboard pollution.** The user's previous clipboard contents get overwritten. Robust tools save the prior clipboard with `wl-paste`, then restore it ~200 ms after pasting.
- **Primary vs clipboard selection.** Wayland, like X11, distinguishes the clipboard (`Ctrl+V`) from the primary selection (middle-click). `wl-copy -p` sets the primary; bare `wl-copy` sets the clipboard. Use the clipboard; it's what Ctrl+V consumes.
- **Terminals and bracketed paste.** Terminals that enable bracketed paste mode will wrap the injected text in `ESC[200~…ESC[201~`, which is usually desired (suppresses control-character interpretation). Some terminals or shells with bracketed paste disabled will interpret newlines and tabs in the transcript as input.
- **Apps that block programmatic Ctrl+V.** Rare on Linux, but some Electron/web apps listen for the `paste` event's `isTrusted` flag and reject synthetic events. Fallback is per-app.
- **ydotool daemon must be running.** `ydotoold` needs to be a running service with your user in the `input` group (or it needs `uinput` access). If `ydotool key ctrl+v` is silent, check `systemctl --user status ydotoold`.
- **Non-latin / mixed-script content.** The clipboard carries a UTF-8 string — no keycode mapping needed, no layout issues, no dead-key composition problems. This is a real advantage over keystroke synthesis: emoji, Hebrew, accented characters all Just Work.
- **Focus race.** If focus moves between the `wl-copy` call and the `ydotool` Ctrl+V, the paste lands in the wrong window. Same risk as paced typing, but now a single-shot mistake instead of a partial one. See `focus-loss-during-dictation`.

(2) IME commit-string via Fcitx / IBus — the "proper" way, rarely implemented.

Wayland's native mechanism for an external process to put text into a focused editor is the input-method / text-input protocol pair (`zwp_input_method_v2` ↔ `zwp_text_input_v3`). An IME calls `commit_string`, and the app receives the text as a single atomic input event — no keystrokes, no clipboard.

- **Pros:** no clipboard pollution; no paste-event quirks; works in any text-input-v3-aware app regardless of key-layout; indistinguishable from a real IME like Fcitx typing Japanese.
- **Cons:** you have to implement (or piggyback on) Fcitx5 or IBus as an IME addon. Some apps still don't implement `zwp_text_input_v3` correctly — GTK4 and Qt6 do; Electron is patchy; terminals vary.
- **In practice:** almost no open-source dictation tool for the Linux desktop ships an IME backend. The path of least resistance has been clipboard-paste, and it's "good enough" for most users.

You can prototype the IME route by writing a small Fcitx5 addon that receives a transcript over a Unix socket and calls `commit_string`, but it's a weekend project and you get roughly the same user-visible result as wl-copy+Ctrl+V with fewer quirks.

What to look for in Handy

Based on the pattern every similar tool follows, look for one of these in Handy's config (usually `~/.config/handy/config.toml` or similar):

- `output_mode`, `injection_mode`, `typing_mode` — a string/enum with values like `"type"` / `"paste"` / `"clipboard"`.
- A boolean like `use_paste` / `paste_on_commit` / `fast_mode` / `instant_inject`.
- A per-character delay (`type_delay_ms`, `keystroke_delay_ms`) — if present and there's no paste mode yet, dropping it to 1–2 ms gives you near-instant typing at the risk of dropped events.

If none of those exist, Handy hasn't exposed the choice and a small patch (set clipboard → send Ctrl+V → restore clipboard) is the minimum viable change. Filing it as a feature request is reasonable; most Linux-facing STT tools have landed this feature under a name like "paste mode" after enough users asked.

Tools that already ship this

For reference — these all implement some variant of auto-paste as the default or an option:

- **nerd-dictation** (Blender-author's tool) — `--output SIMULATE_INPUT_TYPE=PASTE` uses the clipboard path.
- **Whispering** (open-source) — "Paste mode" toggle.
- **Talon Voice** — output strategies include clipboard paste.
- **Dragon for Linux** (there isn't one, but on macOS, all modern STT tools default to paste).
- **Vibe** / **Whisper-keyboard** / many GNOME extensions — clipboard paste is the near-universal default for batch-mode tools.

Recommendation

For your stack (Ubuntu 25.10, KDE Wayland, ydotool already present): the one-line answer is **"paste-based injection via `wl-copy` + synthesised Ctrl+V"**, and you can validate the UX immediately with a shell pipeline before touching Handy's source:

```
# Swap Handy's typing call for this, in spirit:
TEXT="<whatever Handy just transcribed>"
OLD_CLIP="$(wl-paste 2>/dev/null)" || OLD_CLIP=""
printf '%s' "$TEXT" | wl-copy
ydotool key ctrl+v
sleep 0.3
printf '%s' "$OLD_CLIP" | wl-copy
```

That's the whole pattern. When someone says "paste mode" / "fast mode" / "instant injection" in the dictation-tool ecosystem, this is what they mean. The IME-commit route is the same outcome through a cleaner protocol, but it's not something you wire up in an afternoon.

Follow-up — 2026-04-14: Handy's "Paste Method" setting, decoded

Question: In Handy's settings there's a "Paste Method" dropdown with options *Direct*, *Clipboard (Ctrl+V)*, *Clipboard (Ctrl+Shift+V)*, *Clipboard (Shift+Insert)*, *None*, *External Script*. Does "paste" here mean automatic injection, or does it require the user to press Ctrl+V manually?

Handy's "Paste Method" is the **umbrella name for the output stage**, not a setting specifically about manual pasting. Mildly confusing naming — "Output Method" would have been clearer. Per the official docs, only **one** of the six options requires user intervention:

Option	Mechanism	Automatic?
Clipboard (Ctrl+V)	Sets clipboard, then synthesises Ctrl+V. Default on macOS and Windows.	Yes
Clipboard (Ctrl+Shift+V)	Same, with Ctrl+Shift+V — for apps that intercept plain Ctrl+V.	Yes
Clipboard (Shift+Insert)	Same, with Shift+Insert — recommended for Windows Terminal / PowerShell.	Yes
Direct	Types character-by-character with no clipboard. This is the paced-typing path described in the main response. Caveat: doesn't respect non-US keyboard layouts (AZERTY, QWERTZ garble).	Yes, but paced
None	Copies to clipboard only; user hits Ctrl+V themselves.	**No — manual**
External Script (Linux only)	Runs a user-provided script; transcript passed on stdin.	Custom

Two companion settings matter on Linux:

- **Typing Tool**: picks the backend used to *simulate the paste keystroke* for clipboard-based methods — `wtype`, `kwtype`, `dotool`, `ydotool`, `xdotool`, or `Auto`. Only relevant when Paste Method is one of the Clipboard variants or Direct.
- **Clipboard Handling**: `Don't Modify Clipboard` (saves and restores prior clipboard contents) vs `Copy to Clipboard` (leaves the transcript on the clipboard for reuse).

Applied recommendation for Daniel's stack (Ubuntu 25.10 / KDE / Wayland)

1. **Paste Method** → `Clipboard (Ctrl+V)`. This is the auto-paste pattern from earlier in this note — one-shot, layout-safe, instant regardless of transcript length.
2. **Typing Tool** → `ydotool` (explicit) or `Auto`. `wtype` also works on wlroots but KDE/Kwin support for `wtype` has historically been patchy; `ydotool` via its daemon is the reliable path.
3. **Clipboard Handling** → `Don't Modify Clipboard` if you rely on clipboard managers or multi-item clipboards; `Copy to Clipboard` if you want the transcript to persist for re-paste.
4. Leave `Direct` aside — it's the slow one, and on a non-US layout it would also garble characters.

Observed issues worth flagging

- `cjpais/Handy#692`: `Direct` mode repeats/breaks text in terminals when dictating into Claude Code / Codex. Switching to any Clipboard variant fixes it — further evidence that Direct's per-character keystroke synthesis fights terminals' input handling.
- `cjpais/Handy#439`: `Direct` doesn't respect non-US keyboard layouts.

Both arguments for preferring a Clipboard paste method on Linux regardless of the latency framing.

Topic: `inference-cadence-and-sentence-entry`

Prompt

Inference cadence and sentence entry for pause-for-thought dictators

Asked: 14/04/26 **Slug:** `inference-cadence-and-sentence-entry` **Response:** `ideas/inference-cadence-and-sentence-entry.md`

The question

Inference splitting and sentence entry, for someone who pauses for thought while dictating.

I find it distracting when words appear **exactly** as I speak them (maybe just a neurotype thing), so I like the display to be **lagging — but not too lagging**, because then you have to wait. That's a point of UI friction worth documenting: if inference happens all in one pass, I have to wait for it to finish before I can (for example) hit enter to send the message in an AI conversation.

There's a middle ground. Deepgram, I think, gets this really right. Questions:

- What are the common actual intervals (partial cadence, endpointing, utterance-end)?
- What interval becomes a particularly useful point at which **sentence and paragraph boundaries** are easy to detect, even when someone pauses for thought while dictating?

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Inference cadence and sentence entry — finding the UX sweet spot for pause-for-thought dictators

Question: `questions/inference-cadence-and-sentence-entry.md` **Written:** 14/04/26 **Stack:** Desktop live dictation for someone who pauses mid-sentence to think. Distracted by words-as-you-speak; also frustrated by fully-batched wait-for-the-end rendering.

TL;DR

The "distracting vs laggy" axis is actually **three tunable intervals**, not one:

1. **Partial cadence** — how often interim results redraw on screen (typically 100–300 ms).

2. **Finalisation threshold** — how long of silence before a segment is committed as immutable final (can be ~10 ms up to seconds).
3. **Utterance-end threshold** — how long of silence before the system declares "you're done" (the one that lets you press Enter) — typically **1000–2500 ms**.

Deepgram's UX feels right because it **decouples** these. Most tools conflate them. The sweet spot for a pause-for-thought dictator is: partials at ~250–300 ms with **stabilised / immutable** semantics, finals fired quickly on short silences, and an utterance-end signal around **1.5–2 seconds** — long enough to survive a thinking pause, short enough not to feel laggy when you actually stop.

Background

Every streaming ASR system emits two event types over the wire: **interim** results (mutable, redrawable) and **final** results (committed, not revised). Vendors layer a third signal on top — **utterance-end** — that tries to say "the speaker is finished for now," independent of when any particular token was finalised.

The UX problem is that these three intervals interact, and most apps expose only one knob. You get "too live" when partials redraw aggressively with every token. You get "too laggy" when a single batch-on-endpoint runs, you wait for it, and only then can you hit Enter. The systems that feel best are the ones that let the three signals run on different clocks.

The three intervals, with numbers

Signal	What it controls	Typical value	User-facing feel
Partial cadence	How often interim text redraws	100–300 ms	"Liveness" — lower = more flicker, higher = more lag
Finalisation silence	Silence before `is_final=true` on a segment	10 ms – few sec	How quickly text "locks in"
Utterance-end silence	Silence before "done speaking" event	1000–2500 ms	How long a pause-for-thought you can take before the system commits

Partial cadence — per vendor

Vendor	Documented cadence	Partial semantics
Deepgram Nova-3	~100–300 ms	Mutable — will rewrite until final. `smart_format` improves finals.
AssemblyAI Universal-Streaming	~300 ms	**Immutable** — partials, once emitted, don't retract.
Google Cloud STT v2	~100–200 ms (observational)	Mutable.

Vendor	Documented cadence	Partial semantics
Azure Speech	~200–500 ms (observational)	Mutable `Recognizing` events.
Speechmatics	Continuous; `max_delay` caps finals (default **10 s**, min 2 s)	Mutable; `max_delay_mode` trades latency vs punctuation.
OpenAI Realtime / `gpt-4o-transcribe`	Sub-second token deltas, not formally spec'd	Token-level — semantics differ from CTC/RNN-T partials.

The **immutable-partials** pattern (AssemblyAI) is the explicit UX countermeasure to word-by-word flicker. Deepgram achieves a similar feel by emitting slightly lagged, stabilised partials rather than retracting aggressively — in practice a ~250–300 ms cadence with minimal mutation.

Endpointing and utterance-end — per vendor

- **Deepgram** distinguishes two things:
  - `endpointing` — ms of silence before `is_final=true` fires on a segment. Default **10 ms** (i.e. finalize ASAP).
  - `utterance_end_ms` — emits a separate `UtteranceEnd` event after this much non-speech. Recommended **1000–2000 ms**. This is the one you wire up for "OK, hit Enter now."
- **AssemblyAI** uses model-based end-of-turn rather than pure silence: `end_of_turn_confidence_threshold`, `min_end_of_turn_silence_when_confident` (~160 ms default), `max_turn_silence` (~2400 ms default).
- **Google Cloud** — `single_utterance` mode ends on silence; threshold not user-exposed.
- **Azure** — `SpeechServiceConnection_EndSilenceTimeoutMs` (default ~500 ms).
- **OpenAI Realtime** — `turn_detection.silence_duration_ms` (default 500 ms) controls server VAD turn boundaries.

Industry silence-threshold convention

These aren't standardised, but there's a broad consensus across vendors and VAD libraries:

Silence duration	Typical interpretation
< 300 ms	Within-utterance pause (don't segment)
500–1000 ms	Sentence boundary
1500–2500 ms	Turn / paragraph boundary / "done speaking"
> 3000 ms	Stop recording

Silero VAD typical hangover is 100–500 ms; WebRTC VAD runs at 10/20/30 ms frames with app-level hangover usually 200–800 ms.

Why Deepgram feels right

Deepgram's API cleanly separates the three signals:

- Partials stream at ~250–300 ms with stabilisation that avoids per-word retraction — **present enough to feel live, lagged enough not to strobe**.

- `endpointing` defaults to 10 ms, so segments commit as immutable finals as soon as silence begins — the text "settles" behind the partial cursor.
- `utterance_end_ms` is a **separate event** you can set to 1500–2000 ms. That means: the text has already finalised, *and* you get a distinct signal for "press Enter now" after the user's pause-for-thought grace period.

This decoupling is the key. Most consumer tools fire a single "done" event that bundles finalise + commit + send, which forces you to pick between "too eager" and "too laggy". Deepgram lets you pick different values for each.

Sentence and paragraph boundary detection

All major vendors combine a **punctuation/capitalization model** (applied on finals) with **silence-based segmentation**. No vendor advertises a semantic sentence-boundary model separate from punctuation.

- Deepgram `smart_format` + `paragraphs=true` — paragraph breaks use long silences + punctuation signals.
- AssemblyAI `format_text` — punctuation on finals.
- Google `enable_automatic_punctuation` — same pattern.

Practical mapping for a pause-for-thought dictator:

Silence	Action
150–400 ms	Ignore — pause within thought. Do nothing.
400–800 ms	Sentence-ending punctuation candidate (punctuation model decides `.` / `?` / `,`).
1500–2000 ms	Utterance-end — safe to commit and signal "done".
2000–3000 ms	Paragraph boundary — insert `\n\n`.
> 3000 ms	Stop recording / release PTT equivalent.

The 1500–2000 ms window is the critical one. Too short (say 800 ms) and a thinking pause looks like "done". Too long (say 3000 ms) and the user drums their fingers after they actually stopped.

The UI friction points, explicitly named

1. **Strobing partials** — text redrawing word-by-word with retractions as the ASR revises. Fixed by: immutable partials (AssemblyAI), stabilised partials (Deepgram), or UI-side lag buffer (render partials ~200 ms behind arrival).
2. **Batch-on-endpoint wait** — a batch model (Whisper-large) running only after you stop talking, blocking the Enter-to-send moment. Fixed by: streaming partials running live + final re-score in parallel; or separate utterance-end event so the UI commits before the re-score returns.
3. **Premature commit on thinking pause** — a too-short utterance-end threshold cuts you off mid-thought. Fixed by: raise `utterance_end_ms` to 1500–2000 ms; optionally make it adaptive (longer after conjunctions, shorter after periods).
4. **Conflated endpoint and commit** — a single event triggers finalisation *and* send. Fixed by: separate "text is final" from "user is done" — two events, not one.

Recommendation

For Daniel's prototype (and for any live-typing UI serving pause-for-thought users), the target configuration:

- **Partial redraw cadence**: ~250 ms, with either immutable-partial semantics or a UI-side 200 ms stabilisation buffer.
- **Segment finalisation**: aggressive — ~10–200 ms, so text "locks in" behind the live cursor quickly.
- **Utterance-end for Enter-to-send**: **1500 ms** default, user-configurable 1000–2500 ms. Crucially, a **separate event** from finalisation.
- **Paragraph break heuristic**: 2000–3000 ms silence inserts `\n\n`.
- **Visual cue**: render partials in a slightly dimmer / italicised style until final — gives the eye something to latch onto without the "flickering-cursor" jitter.

Deepgram's `interim_results=true` + `endpointing=10` + `utterance_end_ms=1500` is the reference config for this user profile. For local, the equivalent is a streaming ASR (Parakeet-TDT or Moonshine-streaming) plus a separate app-side Silero VAD timer that fires the "done" event at 1500 ms independently of the ASR's own segment finalisation.

References

- Deepgram streaming API — `endpointing`, `utterance_end_ms`, `smart_format`, `paragraphs`: developers.deepgram.com/docs/understanding-end-of-speech-detection
- AssemblyAI Universal-Streaming end-of-turn params: assemblyai.com/docs/speech-to-text/universal-streaming
- Google Cloud STT v2 interim results: cloud.google.com/speech-to-text/v2/docs
- Speechmatics `max_delay`: docs.speechmatics.com
- OpenAI Realtime `turn_detection`: platform.openai.com/docs/guides/realtime
- Silero VAD: github.com/snakers4/silero-vad
- Related topics:
  - `vad-for-live-typing` — VAD building blocks for the timers above.
  - `partial-transcript-rewriting` — the mechanics of interim results and stabilisation.
  - `streaming-vs-batch-injection` — how these intervals map to injection strategy.
  - `batch-vs-chunked-inference-accuracy` — why you may still want a final re-score after utterance-end.

Caveats

- Default numbers (Deepgram `endpointing=10`, Speechmatics `max_delay=10s`) are documented.
- Google / Azure / OpenAI partial cadences above are observational and undocumented — treat as ballparks, not spec.
- AssemblyAI end-of-turn defaults are current as of 2025 and subject to change.
- The "200–400 ms feels live, >800 ms feels laggy" rule is anecdotal industry folklore — no peer-reviewed HCI number confirms it.

Topic: `live-typing-models-saas-and-local`

Prompt

Leading STT models for live typing — SaaS/API and locally runnable

Asked: 14/04/26 **Slug:** `live-typing-models-saas-and-local` **Response:** `ideas/live-typing-models-saas-and-local.md`

The question

Leading models for the live typing use-case, in two categories:

1. **SaaS / API access**
2. **Locally runnable**

For (2) include Moonshine and NVIDIA Parakeet (etc). Check what Handy offers for a list. Note some commonalities and points of difference in architecture.

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Leading STT models for live typing — SaaS and local

Question: `questions/live-typing-models-saas-and-local.md` **Written:** 14/04/26 **Stack:** Desktop live dictation — real-time speech-to-text feeding a cursor in arbitrary apps. Hardware-agnostic but Daniel's box is Ubuntu 25.10 / KDE on a GPU workstation.

TL;DR

For **live typing**, the two things that actually matter are (a) whether the model **emits partial hypotheses quickly enough to feel live** (sub-300 ms tick), and (b) whether its **final output is accurate enough that you don't have to correct afterwards**. That split maps cleanly onto an architecture axis:

- **Transducer-family** (RNN-T, TDT) and **streaming CTC** — built for low-latency partials. Dominant in SaaS streaming APIs and in Parakeet / Moonshine-streaming locally.
- **Encoder-decoder / attention** (Whisper, Canary, gpt-4o-transcribe) — attention over the full utterance gives top-tier accuracy but is inherently **batch-after-endpoint**; streaming is faked by re-running on a rolling window.

Live-typing tools almost always bolt **VAD → streaming ASR → endpointed final rescore** together. The "best" model depends on which of the three you're willing to compromise on.

Background

A live-typing pipeline has five stages: mic capture → VAD/endpointing → ASR partials (typed on-screen as they stabilise) → ASR final (committed on utterance end) → injection into focused window. The ASR stage is what this doc is about.

Two orthogonal choices drive model selection:

1. **SaaS vs local** — latency floor, privacy, cost per minute, offline capability.
2. **Streaming vs batch-on-endpoint** — whether you want text to appear as you speak, or only once you stop.

Handy is a useful reference point because its model registry reflects current production-grade open-weight options — and it ships multiple architectures side by side so users can swap.

SaaS / API — the streaming leaders

All of these stream over WebSocket (or equivalent) and emit **interim results** with a **final** on endpoint. Architecture families below are based on public vendor characterisation; proprietary specifics are often undisclosed.

Vendor / model	Architecture family	Streaming behaviour	Notes
Deepgram Nova-3	End-to-end transformer, CTC-family (historical; not vendor-confirmed)	WebSocket, interim + finals, endpointing built in	Low P50 latency (~300 ms partial), strong English, word-level timestamps, keyterm boosting.
AssemblyAI Universal-Streaming (2025)	Transducer-family, purpose-built for streaming	"Immutable" partials (don't rewrite), clean end-of-turn finals	Designed around agent/voice-UI use. Different partial semantics from Deepgram.
Speechmatics	Proprietary (historically CTC-based, "Ursa"/"Flow")	WebSocket, partials + finals	Strong accent/ESL robustness; batch-quality finals while still streaming.
Google Cloud STT v2 (Chirp 2)	Encoder + CTC-style decoding for streaming models (USM-derived)	`interim_results: true`	Wide language coverage; latency varies by region.
Azure Speech	Transformer-based (undisclosed specifics)	SDK/WebSocket with partial hypotheses	Tight Windows/M365 integration,

Vendor / model	Architecture family	Streaming behaviour	Notes
			continuous recognition mode.
OpenAI `gpt-4o-transcribe` / `gpt-4o-mini-transcribe`	Encoder-decoder (GPT-4o family)	Streaming via Realtime API, delta tokens	Newer offering; accuracy is LLM-grade but partials are decoder tokens, not ASR alignments — semantics differ from CTC/RNN-T partials.
OpenAI `whisper-1` (legacy)	Encoder-decoder (Whisper)	**Not streaming — batch only**	Listed for completeness; not viable for live typing against the API.

Common pattern: all of these let you stream raw PCM up, receive `is_final` flags on result objects, and most support **endpointing hints** (silence thresholds, utterance-end triggers). Differences that matter for a live-typing UX:

- **Immutable vs rewritten partials** — AssemblyAI's Universal-Streaming prefers partials that don't move once emitted; Deepgram will rewrite aggressively until the final. Rewriting feels smart on screen but complicates injection if you type partials directly.
- **Endpoint control** — Deepgram and AssemblyAI expose explicit VAD/utterance-end parameters; Google/Azure are more opaque.
- **Word timings vs token deltas** — transducer/CTC APIs give word-aligned timestamps; GPT-4o-transcribe gives token deltas without alignment.

Locally runnable — what Handy ships

Handy's model registry (`src-tauri/src/managers/model.rs`) is a current, actively maintained list of local STT options grouped by engine. As of April 2026:

Whisper family (via `whisper-rs` / whisper.cpp, GGML)

- `small` (465 MB), `medium` q4_1 (469 MB), `turbo` (large-v3-turbo, 1549 MB), `large` (large-v3 q5_0, 1031 MB), `breeze-asr` (Taiwanese Mandarin fine-tune, 1030 MB).
- **Encoder-decoder**. Not streaming-native — for live typing you chunk with overlap or wait for endpoint.

NVIDIA Parakeet family (via `transcribe-rs`, ONNX int8)

- `parakeet-tdt-0.6b-v2` (English, 451 MB), `parakeet-tdt-0.6b-v3` (25 EU + ru/uk, 456 MB) — **marked `is_recommended: true`** in Handy.
- **FastConformer encoder + TDT (Token-and-Duration Transducer) decoder.** Transducer-family → native streaming, very low-latency partials. Licence: **CC-BY-4.0** (commercial use with attribution).

Moonshine family (Useful Sensors)

- `moonshine-base` (EN, 55 MB, non-streaming), and V2 streaming variants: `moonshine-tiny-streaming-en` (31 MB), `moonshine-small-streaming-en` (99 MB), `moonshine-medium-streaming-en` (192 MB).
- **Encoder-decoder** (transformer), but tiny: tiny ≈ 27M params, base ≈ 61M params. Licence: **MIT**. Paper: arXiv:2410.15608.
- Handy exposes **two distinct engine types** — `Moonshine` (batch) and `MoonshineStreaming` — because the V2 streaming variants use a different runtime contract.

NVIDIA Canary family (via `transcribe-rs`)

- `canary-180m-flash` (en/de/es/fr, 146 MB), `canary-1b-v2` (25 EU languages, 691 MB).
- FastConformer-based. Designed for multilingual accuracy over raw streaming latency.

Others

- `sense-voice-int8` (zh/en/ja/ko/yue, 152 MB) — multilingual, tuned for Asian languages.
- `gigaam-v3-e2e-ctc` (Russian CTC, 151 MB) — pure CTC, language-specific.
- `cohere-int8` (multilingual, 1708 MB) — Cohere's ASR entry.

Runtime split

- **Whisper** → `whisper-rs` (whisper.cpp bindings), GGML format, GPU-accelerated where available.
- **Everything else** (Parakeet, Moonshine, Canary, SenseVoice, GigaAM, Cohere) → `transcribe-rs` (same author), **ONNX Runtime**, int8 quantised, CPU-optimised.
- VAD: **Silero** via `vad-rs`. Audio I/O: `cpal`. Input injection: `rdev` + `xdotool`/`wtype`/`dotool` fallbacks on Linux.

Architecture commonalities and differences

All production STT models today share the same front end: **log-mel features → neural encoder** (Conformer/FastConformer/Transformer). Where they diverge is **the decoder**, and that's what determines streaming behaviour.

Decoder family → streaming behaviour

Family	Examples	Streaming?	Partial-result feel
CTC (Connectionist Temporal Classification)	Speechmatics (historical), Google Chirp streaming, GigaAM	Native streaming; frame-synchronous	Partials stabilise quickly but over-generate repeats before final collapse.
RNN-T / Transducer / TDT	Parakeet-TDT, AssemblyAI Universal-Streaming, most "real-time" SaaS	Native streaming; frame-synchronous with label budget	Clean partial growth, strong rewriting semantics; latency-accuracy tunable.

Family	Examples	Streaming?	Partial-result feel
Encoder-decoder (attention)	Whisper, Canary, Moonshine, gpt-4o-transcribe	**Not natively streaming** — attends over full encoder states	Batch-quality finals; "streaming" is simulated via chunking + overlap or rolling-window re-inference.
Hybrid / multi-head	Some Nova-generation systems, various research models	Varies	Vendor-specific.

The practical implications for a live-typing UI

- **If you want text to appear during the utterance**: pick a **transducer or streaming CTC** model (Parakeet-TDT, Moonshine-streaming, Deepgram, AssemblyAI). These are what makes the "words appear as you speak" UX feel natural.
- **If you commit on endpoint only**: **encoder-decoder models are fine, often better** — Whisper-turbo or gpt-4o-transcribe will beat most streaming models on final accuracy. This matches the pattern most push-to-talk tools use (Handy's default is push-to-talk + Parakeet-v3).
- **Model size is not the main axis** — Moonshine-tiny (31 MB) streaming beats Whisper-small (465 MB) for live partials despite being ~15× smaller, because its decoder is designed for the job. Size mostly drives vocabulary/multilingual coverage and final-form accuracy, not streaming fitness.

Licensing, briefly

- **Whisper** — MIT.
- **Moonshine** — MIT.
- **Parakeet (TDT 0.6B)** — CC-BY-4.0 (attribution required for commercial use).
- **Canary** — CC-BY-4.0.
- **SaaS APIs** — per-minute pricing, no model weights exposed.

Recommendation (for Daniel's prototype)

Given the desktop live-typing goal and a GPU workstation:

1. **Local baseline for push-to-talk**: `parakeet-tdt-0.6b-v3`. It's Handy's recommended default for a reason — transducer streaming, small enough to run on CPU, near-Whisper-large accuracy on English. Commercial licence is clean.
2. **Local baseline for true streaming partials**: `moonshine-small-streaming-en` or `moonshine-medium-streaming-en`. MIT licence, purpose-built for streaming, tiny enough that latency is trivial.
3. **Quality ceiling for batch-on-endpoint**: Whisper `turbo` (large-v3-turbo) locally, or `gpt-4o-transcribe` via API for the absolute top of the accuracy envelope — accept that partials will be fake-streamed (decoder token deltas).
4. **SaaS fallback for the live UX**: Deepgram Nova-3 or AssemblyAI Universal-Streaming depending on whether you prefer rewriting partials or immutable partials. Both are the reference implementations of "good streaming partial UX".

A pragmatic architecture is **Parakeet-streaming for partials + Whisper-turbo for end-of-utterance re-score** — that's the pattern explored in `batch-vs-chunked-inference-accuracy`.

References

- Handy source — model registry: `src-tauri/src/managers/model.rs`
- Handy runtime: `transcribe-rs`, `whisper-rs`, `vad-rs`
- Moonshine paper: arXiv:2410.15608
- Parakeet-TDT-0.6B-v3: huggingface.co/nvidia/parakeet-tdt-0.6b-v3
- Deepgram Nova-3 docs: developers.deepgram.com
- AssemblyAI Universal-Streaming: assemblyai.com/blog
- Related topics in this repo:
  - `whisper-vs-streaming-asr-for-dictation` — why Whisper isn't built for live dictation.
  - `batch-vs-chunked-inference-accuracy` — accuracy gap between streaming and end-of-utterance inference.
  - `partial-transcript-rewriting` — interim results and stabilization.

Topic: `local-stt-builtin-vad`

Prompt

Local STT engines with built-in VAD (or equivalent silence-handling architecture)

Asked: 14/04/26 **Slug:** `local-stt-builtin-vad` **Response:** `ideas/local-stt-builtin-vad.md`

The question

Which local speech-to-text engines have built-in VAD — or an alternative architectural mechanism that serves the same purpose — such that they **will not hallucinate inputs the user didn't speak** when the user pauses for thought? Either the engine has VAD bundled, or its architecture is such that audio captured during silent thinking periods is never sent through the transcription model.

Related:

- `ideas/vad-for-live-typing.md` — the general VAD pipeline story.
- `ideas/local-stt-inference-engines-gpu.md` — engine × vendor × backend table.

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Local STT engines and silence handling — which ones have built-in VAD, which need it bolted on, and which sidestep the problem architecturally

Question: `questions/local-stt-builtin-vad.md` **Written:** 14/04/26 **Stack:** Linux desktop (Ubuntu 25.10 / KDE / Wayland), local inference, live dictation use case where the user pauses mid-thought for seconds at a time.

TL;DR

For a pause-for-thought dictator on a local stack, the question "will it hallucinate during my thinking silences?" has three possible answers depending on how the engine is built:

1. **Built-in VAD / endpointer** — the engine itself gates silence before the model sees it, or the model emits an explicit end-of-utterance token. **Vosk, NeMo streaming**

Conformer/FastConformer, sherpa-onnx / sherpa-ncnn, Wenet, Riva. These are safe by default.

2. **Bundled-but-optional VAD** — the engine is a Whisper-family wrapper that ships a VAD under a flag. Off by default on some, on by default on others. **faster-whisper (`vad_filter=True`), whisper.cpp (`--vad-model`), WhisperX, whisper-streaming, Whispering, nerd-dictation, Handy.** Safe *if you enable the flag.*
3. **No VAD, no endpointer, will hallucinate on silence** — raw Whisper in any form with VAD disabled, Moonshine in its base form, raw `openai-whisper` Python package, Silero STT. These require an external VAD in front of them or they will emit "Thank you for watching" / "." / "Bye!" / translated-language noise during your pauses.

The alternative architecture that **sidesteps VAD entirely** is **push-to-talk** (hold a key while speaking) — the mic is hardware-gated by the user, so there is no silence to hallucinate on. This is why Talon, nerd-dictation (in PTT mode), many Whisper-keyboard tools, and every "hold Fn to dictate" macOS/Windows built-in work reliably without any VAD logic: the user *is* the VAD.

Background — what "silence hallucination" is and why it matters for dictation

Whisper and Whisper-family models were trained on 30-second windows of audio scraped from YouTube, podcasts, and similar sources. A large fraction of those clips contain speech throughout. When Whisper is fed a 30-second window that is actually silence or low-level background noise, it has no "no-speech" class to fall back on — it decodes into whatever token sequence is statistically most likely, which turns out to be things like:

- `Thank you for watching.`
- `Subscribe to the channel.`
- `.` (a single period, repeated)
- `Bye!`
- Entire sentences in Welsh, Korean, or Portuguese.
- The previous utterance, repeated (a failure mode called "repetition loop").

This is **well-documented** (Koenecke et al., "Careless Whisper," FAccT 2024) and **reproducible** — feed `faster-whisper` 10 seconds of room tone with `vad_filter=False` and you will get hallucinated text.

For a dictator who **pauses to think mid-sentence**, this is catastrophic:

- Every thinking pause becomes a potential insertion of garbage text into the focused window.
- If the injector commits partials as they arrive, the garbage gets typed *into the user's document*.
- Correction workflow has to assume any given span could be model hallucination rather than user speech — which destroys trust in the tool.

So "does this engine prevent silence hallucinations?" is one of the single most important questions when choosing a local STT stack for live typing. It sits above model accuracy, above latency, above GPU story.

Engines that handle silence *architecturally*

These engines either do not use a Whisper-style batch decoder, or they have an explicit endpointing / VAD stage that prevents silence from reaching the acoustic model.

Vosk (Kaldi under the hood)

- **Mechanism**: Kaldi's decoder has a built-in silence model and endpointing logic. `KaldiRecognizer.AcceptWaveform(chunk)` returns `True` when an utterance boundary is detected; `PartialResult()` and `FinalResult()` give you text only when there was actual speech.
- **Silence behaviour**: no hallucination. Silence produces empty strings. Pausing mid-dictation is safe by default — the decoder just holds the partial and waits.
- **Tradeoffs**: Kaldi acoustic models are older than Whisper; accuracy on casual conversational English is noticeably lower than Whisper-large. But it is bulletproof on silence.
- **Best fit**: a "no-hallucination-ever" constraint matters more than peak WER.

NVIDIA NeMo — streaming Conformer / FastConformer / Parakeet streaming

- **Mechanism**: RNN-T / CTC streaming models with an integrated endpointing head that emits probabilities per frame. The decoder only outputs tokens when the frame is classified as speech with sufficient confidence.
- **Silence behaviour**: genuine non-emission on silence. No hallucination.
- **Caveat**: the *offline* Parakeet TDT models (the ones most commonly benchmarked for their sub-Whisper WER) are **batch** and do not have the same protection. The streaming variants (`stt_en_fastconformer_hybrid_large_streaming_multi`, etc.) are the ones with the endpointing behaviour.
- **Best fit**: serious live dictation on NVIDIA hardware; willing to run NeMo/Riva.

sherpa-onnx / sherpa-ncnn (k2-fsa project)

- **Mechanism**: wraps streaming RNN-T Zipformer models plus an explicit Silero-VAD ONNX pass. The recognizer API has `is_endpoint(stream)` that returns `True` after configurable silence.
- **Silence behaviour**: silence is gated by Silero VAD; the acoustic model sees only speech segments; endpointing flushes finals on pauses.
- **Tradeoffs**: excellent on-device footprint (runs well on Raspberry Pi, Android, WASM); English models are solid but not Whisper-class on hard domains.
- **Best fit**: low-latency live typing, especially on modest hardware or when you need a single binary with no Python.

Wenet

- **Mechanism**: CTC/AED hybrid with built-in endpointing. Similar story to NeMo — the streaming decoder emits tokens only when there's classified speech.
- **Best fit**: research / Chinese-English bilingual dictation; less common than NeMo in English-first setups.

NVIDIA Riva (self-hostable)

- Same streaming Conformer family as NeMo, packaged as a Triton-based service. Two-pass endpointing is built in.
- **Best fit**: multi-user / long-running daemon; overkill for a single-user desktop tool.

Silero STT

- Ships from the same authors as Silero VAD, so the model and VAD are designed together. English models are CTC-based with explicit handling of non-speech.
- **Caveat**: Silero STT is not commonly used for English desktop dictation — the maintained English models are older, and the Whisper ecosystem has eclipsed it. Worth being aware of but not a first choice in 2026.

Engines that ship with VAD bundled but *optional*

These are Whisper-family wrappers. They do not have architectural silence handling — Whisper itself cannot emit "no speech" reliably — but they ship a VAD wrapper. If the flag is on, silence is gated before Whisper ever sees it. If the flag is off, you get hallucinations.

faster-whisper (CTranslate2)

- Flag: `transcribe(audio, vad_filter=True, vad_parameters=dict(min_silence_duration_ms=500))`.
- VAD model: **Silero VAD** (bundled in the package since v0.9).
- **Default**: `vad_filter=False` — i.e. off by default. **This is the single most common source of "why is my Whisper hallucinating" bug reports.** Always turn it on for live dictation.
- Once enabled, hallucination on silence is almost entirely eliminated.

whisper.cpp

- Flag: `./main --vad --vad-model models/silero-v5.1-small.onnx ....`
- VAD model: Silero, loaded as an ONNX file.
- **Default**: off. Must be enabled explicitly.
- The `stream` example (the sliding-window live-ish mode) has its own energy-based gating that is cruder than Silero; for a real live-typing tool use the Silero ONNX flag.

WhisperX

- Silero VAD (or pyannote VAD, configurable) is used for **segment boundaries** — it's integral to how the pipeline works, not optional.
- Silence doesn't produce hallucination output because silent regions aren't fed to Whisper.
- Caveat: WhisperX is batch-oriented (diarization, word-level alignment) — not the natural choice for live typing.

whisper-streaming (ufal)

- VAD-gated chunks by default. The reference implementation uses Silero VAD (or WebRTC VAD as fallback) to decide when to flush to Whisper.
- Well-suited to pause-for-thought dictation out of the box.

Handy

- Desktop Whisper-keyboard app; uses VAD gating on its hands-free mode.
- Push-to-talk mode sidesteps VAD entirely.
- See `ideas/live-typing-models-saas-and-local.md` for where it sits in the landscape.

nerd-dictation (Linux)

- Built on Vosk, so inherits Vosk's architectural silence handling — no Whisper in the default path.
- Even in its optional Whisper-backend modes, VAD gating is present.
- Push-to-talk is the canonical usage.

Whispering, Dictation, WisprFlow, SuperWhisper, MacWhisper, Aqua, Wispr Flow

- All are Whisper-family desktop tools with VAD baked into their pipeline. They would be unusable without it, so it's not optional at the user level.
- Push-to-talk is typically the default UX; hands-free mode when offered is VAD-gated.

Engines that do NOT handle silence — will hallucinate without external VAD

Reference `openai-whisper` (the original Python package)

- No VAD. `whisper.transcribe("audio.wav")` will hallucinate on silence.
- Has a `no_speech_threshold` (default 0.6) and `logprob_threshold` — these are **per-30-second-segment heuristics**, not a real VAD. They reduce but do not eliminate silence hallucinations, and they're unreliable on short clips.
- **Only safe for live dictation if you put a VAD in front of it yourself.**

Moonshine (Useful Sensors, base form)

- Small streaming-friendly encoder-decoder. No built-in VAD or endpointer.
- The reference demo apps pair it with Silero VAD; without that pairing it will hallucinate on silence like Whisper does.
- Good choice for live typing **only if you wire Silero in front.**

Distil-Whisper

- Smaller, faster Whisper distillation. Inherits Whisper's lack of VAD exactly.
- Same fix: pair with Silero, or run it via faster-whisper with `vad_filter=True`.

Parakeet TDT (the offline / batch variants)

- High-accuracy NVIDIA NeMo models, but the offline variants are batch and do not endpoint. They can and do hallucinate on long silent regions.
- For live typing, pick the **streaming** Parakeet / FastConformer variants, not the offline TDT ones.

Canary (NVIDIA)

- Multilingual offline model. Same story as Parakeet TDT — no streaming endpointer; needs external VAD for live use.

The alternative architecture — push-to-talk makes VAD almost unnecessary

A fundamentally different way to prevent silence hallucinations: **don't capture silence in the first place.**

- User holds a hotkey (or foot pedal, or push-button) while speaking.
- Mic is opened on keydown, closed on keyup.
- Whatever audio is captured is (by definition) intended speech.
- Thinking pauses happen with the key released — no audio captured, nothing to hallucinate on.

This is the reason **Talon**, **nerd-dictation (in PTT mode)**, **most Whisper-keyboard tools**, **macOS Dictation's "Fn Fn" hold-to-dictate**, and **every foot-pedal medical dictation setup for the last 40 years** works without sophisticated silence handling. The user is providing the VAD with their finger.

Tradeoffs relative to hands-free + VAD:

- **Pro**: zero hallucination risk, zero VAD tuning, zero false commits on coughs / keyboard noise / TTS playing in another window.
- **Pro**: extremely simple to implement. One global hotkey + `sounddevice.InputStream` context manager.
- **Con**: requires a hand on a key. Breaks accessibility use cases where hands-free is the whole point.
- **Con**: can't do true long-form dictation without holding the key indefinitely; usually paired with a toggle mode as an escape hatch.

For a pause-for-thought dictator, push-to-talk + release-on-pause is arguably the **correct default** — you think with the key up and speak with the key down. The "pause for thought" problem becomes a non-problem because silence is never in the capture buffer.

See `ideas/voice-dictation-hotkey-count-tradeoffs.md` for the ergonomic side of this.

Quick reference table

Engine	Silence handling	What to do for safe live typing
Vosk	Built-in (Kaldi endpointer)	Use as-is
NeMo streaming Conformer / FastConformer	Built-in endpointing head	Use streaming variants, not offline TDT
sherpa-onnx / sherpa-ncnn	Built-in (Silero + endpoint API)	Use as-is
Wenet	Built-in CTC endpointing	Use as-is
Riva	Built-in two-pass endpointing	Use as-is

Engine	Silence handling	What to do for safe live typing
Silero STT	Trained with VAD-aware data	Viable but ecosystem is thin
faster-whisper	Optional (Silero, off by default)	`vad_filter=True` always
whisper.cpp	Optional (Silero ONNX, off by default)	`--vad --vad-model silero...onnx`
WhisperX	VAD integral to pipeline	Use as-is (but batch, not live)
whisper-streaming	VAD-gated by default	Use as-is
Handy / Whispering / WisprFlow / etc.	VAD baked in	Use as-is
nerd-dictation	Vosk-backed, or VAD-gated Whisper	Use as-is
`openai-whisper` (reference)	None	Add Silero VAD in front, or switch to faster-whisper
Moonshine (base)	None	Add Silero VAD in front
Distil-Whisper	None (inherits Whisper)	`vad_filter=True` via faster-whisper
Parakeet TDT (offline)	None	Switch to streaming FastConformer, or add VAD
Canary	None	Add VAD

Recommendation for this workspace

For a pause-for-thought live-typing prototype on Linux:

1. **If push-to-talk is acceptable UX**: skip the VAD debate entirely. Bind a hotkey, open the mic only while held, feed whatever you captured to any local Whisper. Hallucinations become a non-issue.
2. **If hands-free is required**: pick an engine with **architectural** silence handling — **sherpa-onnx** (lightweight, streaming, native endpointing) or **faster-whisper with `vad_filter=True` always on** (best WER, Silero bundled). Do not use raw `openai-whisper`, raw Moonshine, or offline Parakeet TDT for hands-free dictation.
3. **Do not rely on Whisper's `no_speech_threshold` alone.** It's a heuristic, not a VAD. It will leak silence hallucinations into live output.
4. **Expose VAD sensitivity as a UX setting**, not a buried config. Silence-end threshold (500–800 ms typical) directly shapes how long a thinking pause can be before the system commits prematurely.

References

- Koenecke et al., "Careless Whisper: Speech-to-Text Hallucination Harms" (FAccT 2024) — https://dl.acm.org/doi/10.1145/3630106.3658996
- Silero VAD — https://github.com/snakers4/silero-vad

- faster-whisper VAD filter — https://github.com/SYSTRAN/faster-whisper
- whisper.cpp VAD flag — https://github.com/ggerganov/whisper.cpp
- Vosk API (endpointing via `AcceptWaveform`) — https://github.com/alphacep/vosk-api
- NVIDIA NeMo streaming ASR — https://docs.nvidia.com/deeplearning/nemo/user-guide/docs/en/main/asr/models.html
- sherpa-onnx (streaming + VAD + endpointing) — https://github.com/k2-fsa/sherpa-onnx
- Moonshine — https://github.com/usefulsensors/moonshine
- Companion topic: `ideas/vad-for-live-typing.md`
- Companion topic: `ideas/local-stt-inference-engines-gpu.md`
- Companion topic: `ideas/whisper-vs-streaming-asr-for-dictation.md`

Topic: `local-stt-inference-engines-gpu`

Prompt

Local STT inference engines and GPU acceleration (NVIDIA vs AMD)

Asked: 14/04/26 **Slug:** `local-stt-inference-engines-gpu` **Response:** `ideas/local-stt-inference-engines-gpu.md`

The question

The **inference side of things on local**: if I'm using an AMD GPU, or an NVIDIA GPU, which inference engines provide for acceleration?

Include a table for the **arch split** — which engine supports which vendor's GPU, and via which backend.

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Local STT inference engines — GPU acceleration on NVIDIA vs AMD

Question: `questions/local-stt-inference-engines-gpu.md` **Written:** 14/04/26 **Stack:** Desktop live dictation on Linux (Ubuntu 25.10 / KDE). Focus: local inference, GPU-accelerated where possible. Hardware assumption — either an NVIDIA GPU (CUDA) or an AMD GPU (RDNA2/3/4 via ROCm, HIP, or Vulkan).

TL;DR

Picking a **model** (`live-typing-models-saas-and-local`) is only half the decision — you also pick an **inference engine**, and that's what determines whether your GPU actually gets used.

- **NVIDIA side is boring and easy**: almost every engine has a first-class CUDA path. cuBLAS/cuDNN for general work, TensorRT for squeezed-out latency, ORT's CUDA/TensorRT execution providers for ONNX models.
- **AMD side is where the architecture decisions bite**. Three distinct paths exist — **ROCm/HIP** (closest to CUDA parity, but fragile), **Vulkan compute** (cross-vendor, shipped by whisper.cpp), and **CPU fallback** (sometimes faster than fighting ROCm). Engine by engine, exactly one of those three is usually viable.

- **Short version for AMD users**: `whisper.cpp` with the Vulkan backend is the least painful GPU path for Whisper-family. For Parakeet/Moonshine/Canary you're usually on **ONNX Runtime CPU** unless you rebuild ORT against ROCm.

Background

A local STT stack has three layers:

1. **Model** — the weights (Whisper, Parakeet, Moonshine, …) and their architecture.
2. **Runtime / inference engine** — what actually executes the graph (whisper.cpp, ONNX Runtime, PyTorch, CTranslate2, NeMo, sherpa-onnx…).
3. **Backend / execution provider** — how the runtime reaches the hardware (CUDA, HIP, Vulkan, CPU SIMD).

Model and runtime are usually coupled (Parakeet runs in NeMo or ONNX; Moonshine ships ONNX + Keras + MLX; Whisper runs basically everywhere). Runtime and backend are more loosely coupled — but not as loosely as vendor marketing suggests, especially on AMD.

The engines — what they are and what they accelerate on

whisper.cpp / whisper-rs

- **Runtime**: C/C++ on GGML; GGUF model format.
- **NVIDIA**: `GGML_CUDA=1` (cuBLAS, optional cuDNN).
- **AMD**: two options — `GGML_HIP=1` (requires ROCm toolchain) **or** `GGML_VULKAN=1` (cross-vendor, works without ROCm).
- **CPU**: heavily tuned — AVX/AVX2/AVX512, ARM NEON.
- **Models**: Whisper family only (and Distil-Whisper ports).
- **Streaming**: the `stream` example does sliding-window + VAD — it's *pseudo-streaming*, not true streaming ASR.
- **Why it matters**: **the Vulkan backend is the least-painful GPU path on AMD Linux**. Upstream ships Vulkan binaries; no ROCm install required.

faster-whisper (CTranslate2)

- **NVIDIA**: CUDA + cuDNN; uses cuBLAS.
- **AMD**: **no official ROCm/HIP path**. CTranslate2's GPU path is CUDA-only — on AMD you're on CPU.
- **CPU**: excellent — int8 / int8_float16 quantisation, typically faster than reference Whisper on equivalent hardware.
- **Models**: Whisper family + Distil-Whisper.
- **Streaming**: batch-oriented; word timestamps and batched inference from v1.0.
- **Verdict**: best Whisper runtime on NVIDIA. On AMD, use CPU or switch to whisper.cpp.

WhisperX

- Thin layer over faster-whisper + wav2vec2 alignment + pyannote diarization.
- Inherits faster-whisper's CUDA-only GPU situation. pyannote is PyTorch, so ROCm-PyTorch *should* work for that stage — **unverified**.
- Batch-only; aimed at timestamped transcripts, not live typing.

openai-whisper (reference PyTorch)

- **NVIDIA**: CUDA via PyTorch wheels.
- **AMD**: ROCm via PyTorch ROCm wheels — works on RDNA2/3; RDNA4 is landing in PyTorch 2.5+/ROCm 6.2+ (**verify per release**).
- Reference implementation — slow compared to whisper.cpp or faster-whisper.

ONNX Runtime (ORT)

This is the quiet workhorse — it's what Handy uses under the hood for Parakeet/Moonshine/Canary/etc via `transcribe-rs`.

- **NVIDIA EPs**: `CUDAExecutionProvider`, `TensorrtExecutionProvider`, cuDNN-backed.
- **AMD EPs**: `ROCmExecutionProvider` (HIP), `MIGraphXExecutionProvider`. Windows-only: `DmlExecutionProvider` (DirectML).
- **No upstream Vulkan EP** — community forks exist but aren't production-grade.
- **Gotcha**: ROCm EP requires **building ORT from source against a matching ROCm version** — pip wheels don't include it. This is the single biggest friction point for local Parakeet/Moonshine on AMD.

NVIDIA NeMo

- PyTorch + NeMo toolkit; exportable to ONNX / TensorRT.
- **NVIDIA-only** in the official stack (CUDA/cuDNN/TensorRT).
- On AMD: export to ONNX and run via sherpa-onnx or an ORT-ROCm build — NeMo itself won't run on AMD officially.
- Models: Parakeet (CTC/RNN-T/TDT), Canary, FastConformer, Citrinet.
- RNN-T / TDT variants support **true streaming**.

sherpa-onnx (k2-fsa)

- Built on ONNX Runtime + custom C++ decoding (FST/LG graphs).
- **NVIDIA**: CUDA EP.
- **AMD**: inherits ORT — ROCm EP if you've rebuilt ORT; otherwise CPU.
- Models: Whisper, Moonshine, Parakeet (exported), SenseVoice, Zipformer, Paraformer.
- **Strongest true-streaming support of the OSS engines** — streaming Zipformer and streaming Paraformer are designed for it.

vLLM / TensorRT-LLM

- vLLM has experimental Whisper/audio support; TensorRT-LLM has Whisper examples.
- NVIDIA-only in practice. vLLM has a ROCm path for LLMs, but **Whisper on ROCm is unverified**.
- No broadly adopted local equivalent of `gpt-4o-transcribe` — that model isn't released. Open analogues are Parakeet-TDT and Canary, not vLLM-deployed.

Moonshine's own runtimes

- Useful Sensors ship **ONNX**, **Keras/TF**, **MLX** (Apple Silicon), and a PyTorch reference.
- No dedicated CUDA runtime — acceleration is whatever the chosen backend does (ONNX → ORT EPs; Keras → TF GPU).
- Tiny enough (27M–~200M params) that CPU is often fine for live typing.

transcribe-rs (cjpais — used by Handy)

- Rust wrapper over ONNX Runtime plus whisper.cpp for Whisper models.
- EPs: CPU by default; CUDA available via `ort` crate features; **ROCm EP is not wired up in the default Handy build** — verify per release.
- **Practical upshot for Handy on AMD**: Whisper models can use Vulkan (via whisper.cpp); Parakeet/Moonshine/Canary/etc run on CPU unless you rebuild.

The arch split — engine × vendor × backend

Engine	Runtime / format	NVIDIA path	AMD path	CPU path	Models
whisper.cpp / whisper-rs	GGML	CUDA (cuBLAS, cuDNN)	**HIP (ROCm)** or **Vulkan** ✅	AVX/AVX2/AVX512, NEON	Whisper (+ Distil)
faster-whisper	CTranslate2	CUDA + cuDNN	**CPU only** (no ROCm)	int8 / int8_f16 — very fast	Whisper (+ Distil)
WhisperX	faster-whisper + pyannote	CUDA	CPU (pyannote-on-ROCm unverified)	via faster-whisper	Whisper + alignment/diarization
openai-whisper	PyTorch	CUDA	ROCm PyTorch (RDNA2/3 OK; RDNA4 2.5+)	Yes, slow	Whisper
ONNX Runtime	ONNX	CUDA EP, TensorRT EP	ROCm EP, MIGraphX EP — **rebuild required**	Default CPU EP, OpenVINO EP	Any ONNX-exported model
NVIDIA NeMo	PyTorch + NeMo	CUDA / cuDNN / TensorRT	**Not officially supported**	Yes	Parakeet, Canary, Fast-Conformer, Citrinet
sherpa-onnx	ONNX Runtime + C++ decoder	CUDA EP	ROCm EP (rebuild)	Default CPU	Whisper, Moonshine, Parakeet, SenseVoice, Zipformer, Paraformer
vLLM / TensorRT-LLM	Custom	CUDA / TensorRT	Unverified for Whisper on ROCm	No	Whisper (experimental)
Moonshine runtimes	ONNX / Keras / MLX / PyTorch	via chosen backend (CUDA)	via chosen backend	Yes, often sufficient	Moonshine only

Engine	Runtime / format	NVIDIA path	AMD path	CPU path	Models
			(ROCm-ORT if built)		
transcribe-rs (Handy)	ONNX + whisper.cpp	CUDA (ort feature)	Whisper → Vulkan/HIP via whisper.cpp; others → CPU	Yes — default	Whisper, Parakeet, Moonshine, Canary, SenseVoice, GigaAM, Cohere

Commonalities and points of difference

Common ground:

- Every engine has a working **CPU** path. For small streaming models (Moonshine-tiny/small, Parakeet-TDT-0.6B int8), CPU is often enough for live typing — the GPU question matters more for Whisper-large/turbo batch finals.
- Every engine with NVIDIA support uses some combination of **CUDA + cuBLAS + cuDNN**; the optional TensorRT layer is where you squeeze latency but also where breakage lives.
- **ONNX is the lingua franca for non-Whisper local STT.** If the model isn't Whisper, odds are you're going through ORT — directly (NeMo export, Moonshine), or via a wrapper (sherpa-onnx, transcribe-rs).

Where they diverge:

- **GGML world** (whisper.cpp + friends) has a Vulkan backend. The ONNX world does not. That's the single biggest practical AMD-vs-NVIDIA asymmetry.
- **Streaming-native engines** are sherpa-onnx, NeMo (for RNN-T/TDT models), and Moonshine's own runtimes. Everything else fakes streaming via chunking.
- **Bundled-vs-BYO model** — whisper.cpp and faster-whisper ship with one model family. ORT/sherpa-onnx/transcribe-rs run anything you can export.

ROCm reality check on consumer AMD

- **RDNA3** (7900 XTX/XT, 7800/7700/7600) — supported since ROCm 5.7, but consumer SKUs often need `HSA_OVERRIDE_GFX_VERSION` set for PyTorch to recognise the device.
- **RDNA4** (RX 9070 series) — ROCm 6.3+/6.4+ required; **verify against the current ROCm release notes**. PyTorch ROCm wheels lag ROCm itself by 1–2 minor versions.
- **RDNA2** and older — legacy path; patchy support.

If you're on AMD and want GPU-accelerated Whisper today without fighting ROCm, the pragmatic answer is: **whisper.cpp with `GGML_VULKAN=1`**. Everything else is either CPU (which is often fine), or a ROCm build project.

Recommendation

Given the live-typing goal and the two vendor cases:

Vendor	Whisper-family (batch-on-endpoint)	Streaming partials (Parakeet/Moonshine)
NVIDIA	`faster-whisper` on CUDA (best throughput) or `whisper.cpp` CUDA (simpler)	`sherpa-onnx` with CUDA EP, or Handy (`transcribe-rs`) with CUDA feature
AMD	`whisper.cpp` with **Vulkan**	`transcribe-rs` / `sherpa-onnx` on **CPU** (usually fine for Moonshine-small and Parakeet-TDT-0.6B int8) — ROCm-ORT rebuild only if you need the headroom

For Daniel's current setup this lines up cleanly with Handy: Whisper uses GPU via whisper.cpp, Parakeet-TDT-0.6B-v3 runs on CPU at int8 — no ROCm required.

References

- whisper.cpp: github.com/ggml-org/whisper.cpp — see `Vulkan` and `HIP` sections of the README.
- faster-whisper: github.com/SYSTRAN/faster-whisper; CTranslate2: github.com/OpenNMT/CTranslate2
- ONNX Runtime execution providers: onnxruntime.ai/docs/execution-providers — ROCm EP build instructions live under the "Build for inferencing" guide.
- NVIDIA NeMo: github.com/NVIDIA/NeMo
- sherpa-onnx: github.com/k2-fsa/sherpa-onnx
- Moonshine runtimes: github.com/usefulsensors/moonshine
- Handy / transcribe-rs: github.com/cjpais/Handy, github.com/cjpais/transcribe-rs
- Related topics:
  - `live-typing-models-saas-and-local` — the model-side counterpart to this doc.
  - `whisper-vs-streaming-asr-for-dictation` — why streaming engines matter for live typing.

Topic: `partial-transcript-rewriting`

Prompt

What is the dynamic-rewriting display in tools like Deepgram called?

Asked: 14/04/26 **Slug:** `partial-transcript-rewriting` **Response:** `ideas/partial-transcript-rewriting.md`

The question

Certain live dictation tools — Deepgram is a good example as implemented — print the dictated text dynamically onto the screen as you speak, and rewrite it incrementally. Some text appears, then based on subsequent audio the sentence boundaries are re-inferred and the on-screen text is updated in place.

What is this methodology called? Is it a frontend feature, a backend feature, or a mixture? How does it actually work?

Related

- `whisper-vs-streaming-asr-for-dictation` — deeper architectural background on partial vs. final hypotheses in streaming ASR.
- `streaming-vs-batch-injection` — how (and whether) those rewritten partials should be pushed into a focused window.

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

The dynamic-rewriting display: interim results, stabilization, and where the work happens

Question: `../questions/partial-transcript-rewriting.md` **Written:** 14/04/26 **Stack:** Streaming ASR services (Deepgram, AssemblyAI Universal-Streaming, Google Cloud Speech-to-Text streaming, Azure Speech, Soniox, Speechmatics, NVIDIA Riva, Apple Dictation) feeding a desktop client. Distinct from the *recognition* question covered in `whisper-vs-streaming-asr-for-dictation` — this is about the *display contract*.

TL;DR

The pattern doesn't have a single canonical name; the field uses several interchangeable terms:

- **Interim results** — Google's term, the most common in API docs.
- **Partial transcripts / partial hypotheses** — Deepgram, AssemblyAI, most academic papers.
- **Non-final results** — Azure, Apple `SFSpeechRecognitionResult.isFinal == false`.
- **Unstable hypothesis** + **stable prefix** — Google's finer-grained variant, also seen in research as "stabilization".
- **Live caption** / **live transcript** — the user-facing UI label.
- **Incremental ASR** — the academic umbrella term for the whole class of behaviours.

Mechanically it is **mostly backend, with a thin frontend rendering contract.** The streaming ASR server emits a continuous sequence of `(transcript, is_final, [stability])` events over a WebSocket. Each interim event *replaces* (not appends to) the current in-progress segment in the UI. When the server flips `is_final = true`, that segment is committed and a new in-progress segment begins. The client's job is small but not trivial: maintain a `committed_text + in_progress_text` buffer, replace `in_progress_text` on each interim event, and concatenate to `committed_text` on each final event.

The "sentence boundaries are re-inferred" effect comes from two places: the acoustic model revising its earlier guess as more audio arrives (backend), and the post-processing chain — punctuation, casing, ITN, disfluency removal — being re-run on the growing partial each time (backend). The client almost never runs ML; it runs a string-replace loop.

Background

A streaming ASR session is a stateful, full-duplex connection (WebSocket or gRPC bidi stream). The client sends raw audio frames upward at the capture rate (typically 16 kHz PCM in 20–250 ms chunks). The server sends transcript events downward at its own cadence, decoupled from the audio cadence — usually one event every 100–500 ms once speech is detected.

Each event carries:

- **`transcript`** — the model's current best guess for the *current in-progress utterance*. Not cumulative across utterances.
- **`is_final`** (boolean) — whether this is the locked version of the utterance. Once true, no further events will revise this segment.
- **`stability`** or **`confidence`** (float, vendor-dependent) — how much the model thinks this partial will change.
- Optionally **word-level timings, alternatives (n-best), speaker labels.**

Google's API additionally splits each interim result into a **stable prefix** (the leading words the model is now confident about) and an **unstable suffix** (the trailing words still subject to revision). Other vendors don't expose this split explicitly — you get one transcript string and have to treat the whole thing as mutable.

The UX behaviour the user describes — text appearing, growing, occasionally rewriting earlier words and resnapping sentence boundaries — falls naturally out of this contract.

What the methodology is called

There is no single industry-standard term. The taxonomy below maps the names you'll encounter to the same underlying behaviour.

API / vendor terminology

Vendor / system	Interim event	Locked event	Stability hint
Google Cloud Speech-to-Text	`interim_results`	`is_final = true`	`stability` (0.0–1.0) + `result_end_time` for stable prefix
Deepgram	`is_final = false`	`is_final = true` + `speech_final` for end-pointed segment	`confidence`; `interim_results` flag toggles emission
AssemblyAI Universal-Streaming	`PartialTranscript`	`FinalTranscript`	none surfaced; engine retunes silently
Azure Speech	`Recognizing` event	`Recognized` event	none surfaced
Apple `SFSpeechRecognizer`	`isFinal = false`	`isFinal = true`	none
Speechmatics	`AddPartialTranscript`	`AddTranscript`	none
Soniox	`is_final = false` tokens	`is_final = true` tokens	per-token finalisation (very granular)
NVIDIA Riva	`is_final = false`	`is_final = true`	`stability`
Web Speech API (browser)	`event.results[i].isFinal = false`	`isFinal = true`	none

Conceptual / academic terminology

- **Incremental ASR** — the umbrella term in the speech research literature for any system that emits hypotheses before the utterance is complete.
- **Online ASR** — broader, also covers streaming without incremental emission (i.e. "produce one transcript at the end with low latency").
- **Partial / final hypothesis discipline** — how a system manages the lifecycle of a guess from "first emitted" to "locked".
- **Stabilization** / **prefix stability** — the property that early parts of the transcript stop changing as more audio arrives. A high-stability system rewrites less; a low-stability system rewrites more but may end with better accuracy.
- **Re-decoding** / **lattice rescoring** — the actual mechanism by which an earlier guess gets revised: the model (or a downstream LM) re-evaluates the n-best lattice with new acoustic context.
- **Token revision** / **deletion-and-rewrite** — the visible effect when a previously-emitted token is replaced by a different one. Some systems are *non-revisable* (CTC-style, append-only) and some are *revisable* (RNN-T with re-scoring, attention-based with restart).
- **Right-context delay** — the engineering knob that trades latency for stability: how much future audio the model is allowed to peek at before emitting a token.

For the desktop-dictation reference you're building, the cleanest names to standardise on are probably **interim results** (for the events) and **stabilization** (for the rewriting behaviour as it appears to the user).

Frontend or backend?

It is a **mixture, weighted heavily toward the backend.** The split is roughly:

Backend does (≈90% of the work)

1. **Streaming acoustic decoding** — the model consumes audio chunks and produces token-level hypotheses with monotonic alignment (RNN-T, streaming CTC, streaming Conformer; see `whisper-vs-streaming-asr-for-dictation` for the architectural detail).
2. **Hypothesis revision** — as new audio arrives, the joint network's beam search re-evaluates the n-best hypotheses for the current utterance. An earlier "their" can flip to "they're" once a following verb disambiguates it. This is the source of the visible rewriting.
3. **Post-processing chain re-run on each partial** — punctuation/casing model, inverse text normalisation ("twenty twenty six" → "2026"), disfluency removal, entity formatting. These run on the partial transcript each time it's emitted, which is why sentence boundaries snap into different positions as the partial grows.
4. **Endpointing** — a separate VAD or a learned end-of-utterance head decides "the speaker is done with this segment, flip to final." This is what causes the in-progress text to lock and a new in-progress segment to begin.
5. **Event emission cadence** — the server decides when to emit an interim result. Some vendors emit on every model step (~80–160 ms); others throttle to ~500 ms; some only emit when the transcript actually changes.

Frontend does (≈10% — but easy to get wrong)

1. **Maintain two buffers**: `committed` (everything from past `is_final = true` events, concatenated) and `in_progress` (the latest interim transcript for the *current* utterance, replaced wholesale each event).
2. **Render `committed + in_progress`** on each event. Style `in_progress` differently (lighter colour, italic, underline, subtle background) to signal mutability.
3. **On `is_final`**: append `in_progress` to `committed`, clear `in_progress`, ready for the next utterance.
4. **Cursor / caret positioning** — keep the caret at the end of `committed + in_progress` so the user sees text growing toward them.
5. **Optional: client-side stabilization heuristic** — if the vendor doesn't provide a stable-prefix hint, the client can compute a longest-common-prefix across the last N partials and only animate / rewrite the suffix beyond that prefix. This dampens visible jitter at the cost of a small extra delay.
6. **Optional: smoothing / debouncing** — coalesce rapid-fire partials so the UI doesn't repaint at 20 Hz. A 60–80 ms render budget is usually right.
7. **Edge cases**: handle out-of-order events (rare on WebSocket but possible on lossy transports); handle the connection dropping mid-utterance (commit `in_progress` as-is or discard).

Where the line moves

Some products move *more* work to the frontend deliberately:

- **Two-pass clients**: render the raw partial immediately for responsiveness, then run a small client-side rewriter (punctuation, capitalisation) to clean it up before commit. Reduces server post-processing latency.
- **Custom stabilization**: the client tracks how often each token has been revised and only renders tokens that have been stable for k consecutive partials. Trades a bit of latency for much less visible rewriting.
- **Predictive rendering**: the client runs a tiny on-device LM that predicts the next 1–2 words from the current partial and renders them ghosted, then confirms or rewrites when the server catches up. Rare in practice — too easy to hallucinate.

And some products move *more* work to the backend:

- **Pre-stabilized output**: the server only emits an interim result when its stable-prefix has grown. The client gets an append-only stream and never has to rewrite. AssemblyAI's Universal-Streaming and Soniox's per-token finalisation lean this way.
- **Server-side rendering of the styled HTML**: rare for dictation, common for live-caption broadcast tools.

How a single utterance looks on the wire

Concrete trace for the spoken phrase *"the meeting is at four PM tomorrow"*, talking to a generic streaming ASR endpoint with `interim_results=true` and `smart_format=true`. Times are illustrative.

```
t=0.00s  client opens WebSocket, starts streaming 16-bit PCM @ 16kHz
t=0.30s  → interim   "the"                              is_final=false
t=0.45s  → interim   "the meeting"                      is_final=false
t=0.62s  → interim   "the meeting is"                   is_final=false
t=0.80s  → interim   "the meeting is at"                is_final=false
t=1.05s  → interim   "the meeting is at 4"              is_final=false  ← ITN kicks
in
t=1.25s  → interim   "the meeting is at 4 PM"           is_final=false
t=1.55s  → interim   "the meeting is at 4 PM tomorrow"  is_final=false
t=1.80s  → final     "The meeting is at 4 PM tomorrow." is_final=true   ←
punctuation + casing on commit
t=2.10s  endpointer fires; segment closed
```

Things to notice:

- The same prefix ("the meeting is at") is re-emitted in each interim event. The client replaces, not appends.
- `"four"` becomes `"4"` mid-stream — that's the ITN model running on the partial each time and changing its mind once it sees the following "PM".
- Capitalisation of `"The"` and the trailing period only land on the final event. Many vendors only run the heaviest post-processing on the final utterance to keep interim latency low; cheaper post-processing (number formatting) runs on every partial.
- A revision can also delete: if the user's next word disambiguates an earlier homophone, an interim result can shorten or restructure the prefix.

Why it's done this way

The design is a compromise between three competing goals:

- **Perceived latency** — the user wants to see something within ~250 ms of speaking. Waiting for a finalised, post-processed transcript would feel sluggish.
- **Final accuracy** — the model gets better answers when it sees more right-context. Forcing it to commit early hurts WER.
- **UI stability** — the user wants the text to stop wiggling so they can read it. Constant rewriting is distracting.

Interim results + stabilization is the standard compromise: emit a fast, mutable guess for responsiveness, lock it once enough right-context has arrived, and visually distinguish the two states. The exact tuning (how often to emit, how much to revise, when to flip to final) is what differentiates products and what the various stability/endpointing knobs control.

Implications for an "ideal" desktop live-typing UI

A few design choices follow directly from the contract:

- **Render committed and in-progress text in different visual styles.** Users learn very quickly to ignore the unstable region while reading the stable one. Without the visual cue, every partial revision looks like a bug.
- **Never inject the in-progress region into a destination application.** Only commit on `is_final`, or on a client-side stability heuristic (e.g. "this prefix has been stable for 400 ms"). See `streaming-vs-batch-injection` for the injection-policy implications.
- **Tune endpointing aggressively for dictation.** Default endpointing thresholds are tuned for meeting transcription (long pauses tolerated). Dictation users want short pauses to commit, so they can think between phrases without losing the segment.
- **Respect the two-buffer model in your data structures.** A single mutable string is hard to reason about; `committed: string + in_progress: string` makes the contract explicit and matches the API event shape.
- **Provide a manual "commit now" affordance.** Some users want to flush the in-progress segment without waiting for the endpointer. A keyboard shortcut that forces `in_progress → committed` is cheap to add and high-value.
- **Consider stabilization debouncing if the vendor's partials are jittery.** A 200–400 ms longest-common-prefix dampener removes a lot of visible noise without hurting perceived latency.

Caveats

- **"Deepgram does X" is a moving target.** Vendor APIs evolve; check current docs before relying on specific event field names. The high-level contract (interim → final, replace not append) is stable; the field names and stability metrics are not.
- **Web Speech API is not a peer to commercial APIs.** Browser implementations vary wildly in stability behaviour; Chrome's implementation (which proxies to Google) is the most production-like.
- **Some embedded / on-device models are append-only.** Apple's older on-device dictation, several CTC-only open-source models, and some keyboard voice-input modes don't revise — they emit once and commit. The "rewriting" UX is specific to revisable systems (RNN-T, attention with restart, server-side beam search).

- **Stabilization metrics are not comparable across vendors.** Google's `stability` and Deepgram's `confidence` measure different things and aren't on the same scale.
- **Translation streaming behaves differently.** When a streaming ASR is paired with a streaming MT model, the rewriting can be much more aggressive (whole clauses reordered) because translation needs more right-context than transcription. The same display contract applies but the visual jitter is higher.
- **Local Whisper "streaming" wrappers fake this contract.** They run batch Whisper on rolling VAD chunks and synthesise interim/final events from the chunk boundaries. The rewriting feel is shallower and the latency floor is higher; see `whisper-vs-streaming-asr-for-dictation`.

References

- Google Cloud Speech-to-Text — interim results and stability: https://cloud.google.com/speech-to-text/docs/basics#interim-results
- Deepgram — interim results and `is_final` / `speech_final`: https://developers.deepgram.com/docs/interim-results
- AssemblyAI Universal-Streaming — partial vs. final transcripts: https://www.assemblyai.com/docs/speech-to-text/universal-streaming
- Azure Speech SDK — `Recognizing` vs `Recognized` events: https://learn.microsoft.com/azure/ai-services/speech-service/get-started-speech-to-text
- Apple `SFSpeechRecognitionResult.isFinal` documentation: https://developer.apple.com/documentation/speech/sfspeechrecognitionresult/1649463-isfinal
- Soniox — per-token finalisation: https://soniox.com/docs/
- Speechmatics — partial vs final transcript messages: https://docs.speechmatics.com/rt-api-ref
- Web Speech API `SpeechRecognitionResult.isFinal`: https://developer.mozilla.org/docs/Web/API/SpeechRecognitionResult/isFinal
- Background on incremental ASR (Schlangen & Skantze, *A General, Abstract Model of Incremental Dialogue Processing*): https://aclanthology.org/E09-1081/
- FastEmit — training-time technique for low-latency streaming emission: https://arxiv.org/abs/2010.11148

- Related in this workspace:
  - `whisper-vs-streaming-asr-for-dictation` — the model architecture that makes revisable partials possible.
  - `streaming-vs-batch-injection` — what to do with interim results when feeding a focused application.

Topic: `pause-tolerant-dictation-profile-and-stack`

Prompt

Pause-tolerant dictation — codify this user profile and recommend an STT stack

Ideas: `ideas/pause-tolerant-dictation-profile-and-stack.md` **Asked:** 14/04/26

Given the user profile below, produce a specification for the ideal live-typing STT stack and recommend concrete models and tooling (cloud and local).

User profile

- **Long thinking pauses** mid-dictation, commonly 3–20 seconds. Requires **absolute certainty** that pauses do not trigger hallucinated content — applies equally to cloud and local models. Hard requirement, not a preference.
- **Output should be post-processed**: inferred paragraph breaks, sentence structure, removal of filler words (“um”, “uh”, “you know”) and false starts.
- **Primary workload**: general desktop typing, with heavy emphasis on dictating prompts to AI agents — precision and nuance matter, especially for editing instructions where a misheard word changes the meaning of an edit.
- **Framing**: STT is a flow-state productivity tool, not just transcription. The goal is to minimise friction between thought and instruction.
- **Preferred frontend UX**: periodic text output at roughly a **~20-second cadence**. Too-real-time rendering creates a “being watched” pressure that breaks flow. A ~20 s cadence provides reassurance that inference is working without turning typing into a performance.

What the answer should contain

1. A named user archetype and a codified checklist of requirements.
2. A derived technical specification — pause tolerance, post-processing pipeline, injection cadence, accuracy bar.
3. A cloud recommendation table (Deepgram, AssemblyAI, Speechmatics, OpenAI, ElevenLabs) with the specific knobs that matter for this profile.
4. A local recommendation (Whisper-family with VAD, Parakeet/Canary, Moonshine, whisper-streaming).
5. Frontend tools whose UX already matches this profile.
6. A concrete recommended architecture (the “ideal spec” as a pipeline).
7. Anti-patterns to avoid.

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

The pause-tolerant prompt-dictator — a codified user profile and the STT stack that matches it

Question: `questions/pause-tolerant-dictation-profile-and-stack.md` **Written:** 14/04/26
Stack: Desktop live dictation on Ubuntu 25.10 / KDE / Wayland, primary workload is dictating prompts to AI agents. Thinking pauses of 3–20 s are routine. Hallucinations during silence are an absolute no. Preferred UX is batched commits at ~20 s cadence, not token-streaming.

TL;DR

The right architecture for this user is **push-to-talk (or hold-to-dictate) with batched commits at a ~15–20 s cadence**, fed into an ASR model with proven non-hallucinating silence behaviour, then polished by an LLM pass that removes disfluencies and inserts paragraph breaks before injection. The **top cloud pick is Deepgram Nova-3** (with `smart_format=true`, `endpointing` decoupled from `utterance_end_ms`, or simply PTT-gated so endpointing is moot). The **top local pick is NVIDIA Parakeet-TDT 0.6B v2 via NeMo** (streaming variant for hands-free, offline variant under PTT) because it has no Whisper-style silence-hallucination failure mode. Raw Whisper is only acceptable under faster-whisper with `vad_filter=True` + strict `no_speech_threshold` / `logprob_threshold` / `compression_ratio_threshold` settings.

The user profile, codified

Archetype name: **the pause-tolerant prompt-dictator**.

Defining traits, as a checklist:

- [ ] **Thinks mid-utterance.** 3–20 s pauses are routine, not exceptional. Any system that treats >2 s of silence as “done speaking” is incompatible.
- [ ] **Zero tolerance for hallucinated content.** A fabricated “Thank you for watching” or a repeated prior phrase inside a prompt to an AI agent is worse than a missed word — it actively corrupts the instruction.
- [ ] **Wants post-processed output.** Raw ASR with `um`s, false starts, and wall-of-text monologue is unacceptable. Output should read like edited prose: paragraphs, punctuation, disfluencies removed.
- [ ] **Precision-sensitive workload.** The primary consumer of the transcript is an LLM acting on editing instructions. A misheard “remove” vs “reword” changes the outcome. Justifies paying for a top-tier model.
- [ ] **Flow-state tool, not transcription tool.** The goal is to lower the activation energy between a thought and its delivery to an agent. Anything that makes the user aware of the tool itself (flickering partials, token-by-token reveal, premature commits) is a regression.
- [ ] **Batched visible output at ~20 s cadence.** Token-streaming is distracting. Fully silent until end-of-session is also wrong — the user wants reassurance that inference is making progress. A commit every 15–20 s hits that middle.

Derived specification

Translating the profile into concrete technical requirements:

1. Long-pause tolerance — hard requirement

Two valid architectures, pick one:

- **(a) Sidestep endpointing entirely** — push-to-talk or hold-to-dictate. The mic is hardware-gated by the user's finger. Silences during thought happen with the key released, so they are never in the audio buffer. This is the cleanest solution and what this profile strongly suggests.
- **(b) If hands-free is required** — use an engine with **architectural non-emission on silence** (Parakeet streaming, NeMo FastConformer, sherpa-onnx, Vosk) and set the utterance-end threshold to ≥ 20 s. Deepgram `utterance_end_ms` goes well above default and can be tuned high; Speechmatics `max_delay` caps finals rather than defining an utterance boundary, so utterance-end is app-side anyway.

Hallucination risk vectors to neutralise:

- Whisper's well-documented silence-hallucination behaviour (Koenecke et al., FAccT 2024). Mitigations, in order of defensive strength:
  1. VAD gating in front of the model (Silero VAD; `vad_filter=True` in faster-whisper).
  2. `no_speech_threshold` ≥ 0.6 (default), `logprob_threshold` ≥ -1.0, `compression_ratio_threshold` ≤ 2.4. These are heuristics, not a VAD — do not rely on them alone.
  3. `condition_on_previous_text=False` — breaks the repetition-loop failure mode.
  4. Switch to a non-Whisper model (Parakeet, FastConformer, Nova-3) that doesn't have the training-distribution silence problem in the first place.
- Cloud streaming endpoints can also emit spurious finals during long pauses on some providers; Deepgram Nova-3 and Speechmatics have the cleanest empirical profile here as of April 2026.

2. Post-processing pipeline — two valid shapes

Architecture A — **native formatter only** (cheaper, lower latency, less flexible):

```
mic → ASR with built-in formatter → injection
         (Deepgram smart_format / AssemblyAI format_text+disfluencies
          / Speechmatics punctuation_overrides)
```

Architecture B — **ASR + LLM polish pass** (better prompt-grade output):

```
mic → ASR (raw text + optional native formatter)
    → LLM reformatter (disfluency removal, paragraph inference,
                       light prose cleanup — NOT rewriting content)
    → injection
```

For this user, **Architecture B is the right default**. Native formatters handle punctuation and casing, but they do not reliably remove "you know", "like I said", false starts, or mid-sentence topic swerves — and they don't insert paragraph breaks on semantic content changes. A small LLM pass gets you there.

Candidate polish models:

Model	Hosted/local	Latency for ~300 words	Notes
`gpt-4o-mini` / `gpt-4.1-mini`	Hosted	1–3 s	Cheap; reliable at "clean this transcript without changing meaning".
`claude-haiku-4`	Hosted	1–3 s	Strong at preserving intent; good for instruction-grade transcripts.
Local `Qwen2.5-7B-Instruct`	Local (GPU)	1–4 s	Runs alongside STT on a 16 GB+ card; privacy win.
Local `Llama-3.1-8B-Instruct`	Local (GPU)	1–4 s	Similar.
Local `gemma-3-4b-it`	Local (CPU/GPU)	2–5 s	Lightweight fallback.

Key prompt engineering rule for the polish stage: **instruct the LLM not to rewrite, only to remove disfluencies and add paragraph breaks**. Otherwise it will "improve" prompts and change their meaning, which is the exact failure mode this user cannot tolerate.

3. Injection cadence — batched at ~15–20 s

This is **not** token-streaming. It is also not "fully silent until PTT release". It is periodic commits of well-formed prose.

Implementation options:

- **PTT commit-on-release** (simplest). Works for bursts up to the user's natural utterance length. For longer dictations it violates the "reassurance that inference is working" criterion.
- **PTT with periodic flush every 15–20 s** while the key is held. App-side timer; on tick, slice the audio buffer at the last VAD-detected pause, run inference + polish on the slice, inject, and continue recording on the remainder. This is the architecture that directly matches the profile.
- **Speechmatics `max_delay` ≈ 15–20 s** — caps how long the model will wait before committing a final. Limits apply (documented ceiling is typically 10 s on some plans as of April 2026; check current docs).
- **Deepgram with manual buffering**: let partials stream, but ignore them for UI purposes; only commit to the focused window on utterance-end events, or on an app-side 20 s timer.
- **Whisper/Parakeet chunked batch inference**: slice audio into 15–20 s windows (aligned to VAD-detected silence boundaries to avoid mid-word cuts), run batch inference on each, inject.

This is a generalisation of the Handy "print delay" pattern documented in `handy-inference-vs-typing-delay.md` — but shifted up the pipeline. Handy paces the *keystrokes* of an

already-done transcript. This profile wants pacing at the *inference* boundary, so each commit is a genuinely fresh chunk of finalised, polished text.

4. Accuracy / nuance bar

Prompt dictation demands a higher bar than casual typing. A transcript going to an AI agent is a *specification*, and a single misheard token can invert an instruction. This justifies:

- Largest-tier models: **whisper-large-v3** / **Nova-3** / **Universal-2** / **Parakeet-TDT 0.6B v2** / **Scribe**. Do not use `whisper-tiny`, `whisper-base`, or `distil-small` for this workload.
- Domain vocabulary boosting where available: Deepgram `keywords` / Speechmatics `custom_dictionary` / AssemblyAI `word_boost`. Seed with tool names, library names, and recurring proper nouns.

Recommended cloud stack

All prices / parameter defaults are April 2026; verify against current vendor docs before committing.

Vendor / model	Pause tolerance	Post-processing	Injection cadence knob	Hallucination-on-silence	Approx $/hr	Fit for this profile
Deepgram Nova-3	Excellent — `utterance_end_ms` decoupled from `endpointing`; set to ≥ 20000 or ignore in favour of PTT	`smart_format=true`, `paragraphs=true`, `filler_words=false` (or `=true` then LLM-strip)	App-side buffer + commit on `UtteranceEnd` or 20 s timer	Low on silence; streaming non-emission is well-behaved	~$0.43–0.58	**Top pick.** Cleanest separation of "final" and "done".
AssemblyAI Universal-Streaming (Universal-2)	Good — `max_turn_silence` tunable, immutable partials mean no flicker	`format_turns=true`, `disfluencies=false`, `punctuate=true`	Commit on end-of-turn events	Low; immutable-partials semantics reduce retraction artefacts	~$0.37	Strong alternative; end-of-turn model is opinionated and may not reach 20 s — app-side timer is safer.
Speechmatics Enhanced	Good — `max_delay` caps finals (commonly up to 10 s; beyond that,	`punctuation_overrides`, `enable_entities`, diarisation optional	`max_delay` + app buffer	Low-hallucination profile; explicit silence handling	~$0.30–0.50	Good for long-form; nuance model competitive with Nova-3

Vendor / model	Pause tolerance	Post-processing	Injection cadence knob	Hallucination-on-silence	Approx $/hr	Fit for this profile
	app-side batching)					on UK / accented English.
OpenAI `gpt-4o-transcribe` (Real-time)	Mixed — token-level streaming; defaults to 500 ms	Built-in; fluent, sometimes over-edits	`turn_detection=null` to disable server VAD; commit client-side	Low in practice, but semantics differ from CTC/RNN-T; occasional over-formalisation	~$0.60/hr (audio input pricing)	Use with `turn_detection=null` + PTT to sidestep; otherwise endpointing is too eager.
OpenAI `whisper-1` (non-Realtime)	Batch only	Simple punctuation	Not applicable (batch)	Inherits Whisper's silence-hallucination risk	~$0.36	Acceptable under PTT commit-on-release; not for streaming.
ElevenLabs Scribe v1	Batch only	Good formatting, disfluency handling	Not applicable (batch)	Non-streaming; no live hallucination pattern	~$0.40	Strong for post-hoc polish; **not** a live dictation engine.

Cloud recommendation: Deepgram Nova-3 + `smart_format` + app-side 20 s buffer, fronted by PTT. The PTT lets you treat `utterance_end_ms` as irrelevant; the app-side buffer gives you the 20 s cadence; `smart_format` handles punctuation, and a cheap LLM polish pass removes disfluencies.

Deepgram knobs that matter (as of April 2026):

```
model=nova-3
smart_format=true
punctuate=true
paragraphs=true
filler_words=false
diarize=false
interim_results=false      # we don't want partials on the wire for this profile
endpointing=10             # irrelevant under PTT, leave at default
utterance_end_ms=20000     # fallback if running hands-free
keywords=<project vocab list>
```

Recommended local stack

Privacy, cost, and offline operation all favour local. Daniel's hardware is documented in `local-stt-inference-engines-gpu.md` — see that topic for which engine × backend combinations are viable on this machine.

Engine / model	Silence handling	Accuracy tier	Best usage for this profile
Parakeet-TDT 0.6B v2 (offline, via NeMo)	No architectural silence handling; safe under PTT	State-of-the-art English WER on LibriSpeech / AMI	**Top pick under PTT.** ~20–50× real-time on a modern GPU; chunk into 15–20 s windows.
FastConformer streaming (NeMo)	Built-in endpointing; no hallucination on silence	Very good, slightly below offline Parakeet	**Top pick hands-free.** Use for live streaming + long-pause tolerance.
faster-whisper large-v3	VAD via Silero, optional (`vad_filter=True`)	State-of-the-art Whisper	Acceptable with strict config (below). Still slower than Parakeet.
whisper.cpp large-v3	`--vad --vad-model silero-v5.1-small.onnx`	Same as above	Best for CPU-only or when CTranslate2 isn't available.
WhisperX	VAD integral to pipeline	Same as Whisper	Batch-oriented; fine for post-hoc polish passes, not live.
whisper-streaming (ufal)	VAD-gated by default	Whisper WER	Designed for long-form streaming; matches this profile if you must use Whisper in a streaming shape.
Moonshine-base	No built-in VAD — must pair with Silero	A tier below Whisper-large	Small, fast, PTT-friendly; not the first choice when accuracy matters for prompts.
sherpa-onnx (Zipformer)	Built-in VAD + endpointing	Solid, below Whisper-large on hard domains	Great ultra-low-latency fallback; not the top pick for nuance.
Canary (NeMo)	No streaming endpointer	Strong multilingual	Only under PTT with VAD in front.

Safe `faster-whisper` config for this profile:

```python
from faster_whisper import WhisperModel

model = WhisperModel("large-v3", device="cuda", compute_type="float16")

segments, info = model.transcribe(
    audio,
    vad_filter=True,
    vad_parameters=dict(
        min_silence_duration_ms=500,
        threshold=0.5,
    ),
    no_speech_threshold=0.6,
    log_prob_threshold=-1.0,
    compression_ratio_threshold=2.4,
    condition_on_previous_text=False,   # critical: breaks repetition loops
    beam_size=5,
    temperature=(0.0, 0.2, 0.4),        # fallback ladder; first pass greedy
    word_timestamps=False,
    language="en",
)
```

The `condition_on_previous_text=False` line is the single most important setting for this user. It is Whisper's main silence-hallucination mitigation beyond VAD gating.

Local recommendation: Parakeet-TDT 0.6B v2 under push-to-talk, batched at ~20 s windows, polished by a local Qwen2.5-7B-Instruct pass, injected via Fcitx IME commit-string. No Whisper quirks to manage; inference is fast enough that the 20 s cadence is inference-done-in-parallel rather than inference-blocking.

See `local-stt-builtin-vad.md` for the wider silence-handling landscape.

Recommended frontends that match this UX

Existing desktop tools ordered by how well they match the pause-tolerant prompt-dictator profile:

Tool	Pattern	Fit for this profile
Handy (Rust/Tauri, open source)	PTT → batch transcribe → keystroke-paced injection	**Strong match.** Paced typing *visually* approximates the batched-commit cadence. Use Parakeet backend. See `handy-inference-vs-typing-delay.md`.
Superwhisper / MacWhisper (macOS)	Hold-to-record → batch transcribe → paste	Strong match on macOS only. PTT sidesteps endpointing entirely.
Wispr Flow / WhisprFlow	Cursor-level dictation with LLM polish built in	Matches the "LLM polish pass" requirement out of the box. Closed-source; cloud-only.

Tool	Pattern	Fit for this profile
Aqua Voice	“Writes like you would” — polish-oriented	Same category as Wispr Flow; closed-source / cloud.
nerd-dictation (Linux)	PTT, Vosk backend by default, optional Whisper	Strong PTT match; needs LLM polish bolted on for the paragraph-inference requirement.
whisper-typer / whisper-keyboard (various)	PTT → Whisper → inject	Adequate if paired with a VAD-gated Whisper build.
Speechnote, Numen, Talon	Hands-free with their own VAD logic	Mixed — Talon is PTT-friendly, others are hands-free and would need their endpointing pushed out to 20 s.
Generic Deepgram/AssemblyAI demo UIs	Token-streaming overlays	**Poor fit.** Exact “being watched” pattern this user wants to avoid.

Framing: push-to-talk + batched commit is the cleanest match. Streaming tools with configurable utterance-end ≥ 20 s are acceptable. Hands-free streaming with short endpointing (default 500–2000 ms) is fundamentally incompatible with this profile.

Recommended architecture — the spec

Putting it all together:

```
┌──────────────────────────────────────────────────────────────────────┐
│  PTT hotkey (hold to dictate, with optional toggle for long-form)    │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Audio capture — sounddevice.InputStream, 16 kHz mono, ring buffer   │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Silero VAD pass — label frames; used only to find safe cut points   │
│  for chunk boundaries (never drops audio inside a chunk).            │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Chunk-and-commit scheduler:                                         │
│  - Every 15–20 s of elapsed PTT time, OR on PTT release,             │
│    slice audio at nearest VAD silence boundary.                      │
│  - Pass the completed slice to ASR; keep recording the remainder.    │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
```

```
│  ASR — Parakeet-TDT 0.6B v2 (local, GPU)  OR                        │
│        Deepgram Nova-3 (cloud, smart_format)                        │
│  Returns raw text (or lightly formatted text).                      │
└─────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌─────────────────────────────────────────────────────────────────────┐
│  LLM polish pass — Qwen2.5-7B-Instruct local  OR  gpt-4.1-mini      │
│  System prompt: "Remove disfluencies and false starts. Insert       │
│  paragraph breaks where the topic shifts. Do NOT rewrite, reorder,  │
│  or change meaning. Preserve technical terms exactly."              │
└─────────────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
┌─────────────────────────────────────────────────────────────────────┐
│  Injection — Fcitx IME commit-string (instant, Wayland-clean) or    │
│  clipboard+paste fallback. Per-commit; not character-paced.         │
└─────────────────────────────────────────────────────────────────────┘
```

Latency budget per 20 s chunk (local stack, modern NVIDIA GPU):

Stage	Budget
VAD scan for cut point	<50 ms
Parakeet-TDT inference on 20 s audio	0.5–1.5 s
Qwen2.5-7B polish pass on ~60 words	1–3 s
IME commit injection	<50 ms
Total wall time from cut to injection	**~2–5 s**

So a user who has been dictating for 20 s sees a polished paragraph land 2–5 s later, then continues speaking. The *feel* is one new paragraph every 15–25 s, which is the target cadence.

For cloud, swap Parakeet for Nova-3 (0.5–2 s) and Qwen for `gpt-4.1-mini` (1–2 s); budget is similar.

Anti-patterns for this user

Things to avoid, specifically for the pause-tolerant prompt-dictator:

1. **Short utterance-end timers (< 2 s).** Will commit during thought pauses. Either push endpointing to ≥ 20 s, or use PTT so endpointing is moot. Deepgram `utterance_end_ms=1500`, AssemblyAI `max_turn_silence=2400` — both too aggressive for this profile.
2. **Vanilla Whisper (`openai-whisper`) on long audio without VAD.** Silence-hallucinations ("Thank you for watching", "Subscribe", repeated phrases, drifts into Welsh). Catastrophic for prompt instructions. Always use faster-whisper with `vad_filter=True` + the strict config above, or switch to Parakeet.
3. **Token-by-token streaming overlays.** The exact "being watched" pressure the user wants to avoid. Disable `interim_results` on Deepgram; don't render partials in the UI.

4. **Hands-free VAD-only toggling without a PTT fallback.** A cough, keyboard noise, or a family member's voice triggers capture. For a prompt-dictator, any false capture is a corrupted instruction.
5. **LLM polish passes with vague prompts.** "Clean up this transcript" will paraphrase the user's prompt and change its meaning. Prompt the polish stage narrowly: *remove disfluencies, add paragraph breaks, preserve wording*.
6. **Character-paced keystroke injection** on long transcripts. The 15–20 s-per-paragraph cadence plus another 10–20 s of character-paced typing compounds into an unacceptable delay. Use IME commit-string or clipboard+paste, not paced `ydotool`/`enigo`. See `handy-inference-vs-typing-delay.md`.
7. **Mixing `condition_on_previous_text=True` with Whisper in chunked mode.** Guarantees repetition-loop failures eventually. Always `False` for this workload.
8. **Using `whisper-tiny` / `distil-small` to "speed things up".** The accuracy loss matters more for prompts than for casual typing; save the cycles somewhere else.

References

- Deepgram — `smart_format`, `endpointing`, `utterance_end_ms`, `paragraphs`, `filler_words`: developers.deepgram.com/docs/understanding-end-of-speech-detection
- AssemblyAI Universal-Streaming — end-of-turn parameters, immutable partials: assemblyai.com/docs/speech-to-text/universal-streaming
- Speechmatics — `max_delay`, `punctuation_overrides`, `enable_entities`: docs.speechmatics.com
- OpenAI Realtime — `turn_detection.silence_duration_ms`, `gpt-4o-transcribe`: platform.openai.com/docs/guides/realtime
- ElevenLabs Scribe: elevenlabs.io/docs/capabilities/speech-to-text
- NVIDIA Parakeet-TDT 0.6B v2: huggingface.co/nvidia/parakeet-tdt-0.6b-v2
- NVIDIA NeMo streaming ASR (FastConformer, Canary): docs.nvidia.com/nemo-framework/user-guide/latest/nemotoolkit/asr/models.html
- faster-whisper (`vad_filter`, `condition_on_previous_text`, thresholds): github.com/SYSTRAN/faster-whisper
- WhisperX: github.com/m-bain/whisperX
- whisper-streaming (ufal): github.com/ufal/whisper_streaming
- Moonshine: github.com/usefulsensors/moonshine
- Silero VAD: github.com/snakers4/silero-vad
- Koenecke et al., "Careless Whisper: Speech-to-Text Hallucination Harms" (FAccT 2024): dl.acm.org/doi/10.1145/3630106.3658996
- Related topics in this workspace:
  - `inference-cadence-and-sentence-entry` — the three-interval model this spec sits on top of.
  - `whisper-vs-streaming-asr-for-dictation` — why Whisper isn't built for live dictation.
  - `local-stt-builtin-vad` — which local engines are safe on silence, which aren't.
  - `vad-for-live-typing` — the VAD building blocks.
  - `live-typing-models-saas-and-local` — the wider SaaS/local landscape.
  - `handy-inference-vs-typing-delay` — why paced injection differs from paced inference.
  - `local-stt-inference-engines-gpu` — which engines run on this machine's GPU.
  - `cursor-dictation-vs-clipboard-stt-adoption` — injection strategy tradeoffs.

Caveats

- Vendor pricing, parameter defaults, and model names are accurate to the best of my knowledge **as of April 2026** but move fast. Check Deepgram / AssemblyAI / Speechmatics / OpenAI docs before committing.
- `max_delay` ceilings on Speechmatics have historically been 10 s on most plans; pushing to 20 s may require enterprise tier or app-side batching.
- The “20 s cadence” target is a starting point, not a magic number. Tune between 10 s (tighter reassurance) and 30 s (longer chunks, better LLM polish context) based on subjective feel.
- The LLM-polish stage is the largest remaining risk: it can subtly rewrite the user’s intent. Narrow prompting + occasional manual review are the mitigations. Consider logging both raw ASR and polished output side by side during the prototype phase so drift is visible.
- Parakeet-TDT model sizes and availability on Hugging Face may differ from NeMo distributions; cross-check before downloading.
- Context7 MCP was rate-limited at the time of writing, so vendor-specific parameter names above draw from existing cross-referenced topics in this workspace rather than a fresh documentation pull. Verify parameter names against live docs before building.

Topic: `streaming-vs-batch-injection`

Prompt

Streaming injection vs batch transcription on stop

Asked: 14/04/26 **Slug:** `streaming-vs-batch-injection` **Response:** `ideas/streaming-vs-batch-injection.md`

The question

Some live-typing implementations keep an open audio stream and dump text at the cursor position incrementally — inference is happening on the fly and the screen updates as you speak.

What is *that* design choice called, as opposed to the one I'm currently using? My current setup runs NVIDIA Parakeet as essentially an asynchronous job: it waits for me to toggle the microphone off, then transcribes everything in one go and the full text appears at the cursor all at once.

What's the terminology for each side of this distinction, and where does each pattern sit in the wider design space?

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Streaming injection vs utterance-final injection in live dictation

Question: `questions/streaming-vs-batch-injection.md` **Written:** 14/04/26 **Stack:** Desktop live voice typing (real-time speech-to-text injecting into the focused window). NVIDIA Parakeet (RNNT/CTC/TDT family) used here as the example offline model; the same axis applies to Whisper, Vosk, Moonshine, faster-whisper, etc.

TL;DR

Two orthogonal axes are at play. **Axis 1** is the *recognition mode*: **streaming ASR** (the model emits hypotheses as audio arrives) vs **offline / batch ASR** (the model needs the whole utterance before it produces output). **Axis 2** is the *injection mode*: **incremental / streaming injection** (text is typed at the cursor as hypotheses arrive) vs **utterance-final / commit-on-stop injection** (the full transcript is typed once, after dictation ends). The pattern you're describing — text appearing on the fly — is **streaming ASR + incremental injection**, often marketed as "live dictation" or "real-time dictation". Your current

Parakeet setup is **offline ASR + utterance-final injection**, sometimes called **commit-on-stop**, **toggle-and-transcribe**, or just **batch dictation**.

Background

Desktop dictation tools differ along two axes that beginners often conflate:

1. **What the speech recognizer can do.** Some ASR systems are designed to consume audio as a stream and emit incremental hypotheses (partial tokens that may be revised, then "stabilised" as final). Others are designed to consume a complete audio segment and emit one finalized transcript. The first is *streaming*; the second is *offline* (also called *batch*, *non-streaming*, or *full-context*).
2. **What the dictation tool does with the output.** Even a streaming recognizer can be wired to a "wait until the user stops, then type everything" UX. Conversely, an offline recognizer can be made to feel more "live" by chunking audio at VAD boundaries and committing one chunk at a time. The injection policy is a separate decision from the recognizer's capability.

So there are four cells in the matrix, not two. Most tools sit in one of three of them; the fourth is rare in practice.

The terminology

Recognition mode (Axis 1)

- **Streaming ASR** — the model emits incremental hypotheses as audio frames arrive. Architectures designed for this: streaming RNN-T, streaming Conformer-CTC, monotonic chunkwise attention, Moonshine, streaming variants of NVIDIA Parakeet (Parakeet-RNNT and Parakeet-TDT have streaming configurations; the popular `parakeet-tdt-0.6b-v2` checkpoint is offline-only). Also sometimes called **online ASR** or **incremental ASR**.
- **Offline ASR** — the model needs the full utterance (or at least a long enough chunk) before it produces output. Whisper (the original OpenAI release) is offline by design. Most "highest WER score on Open ASR Leaderboard" models are offline. Synonyms: **batch ASR**, **non-streaming ASR**, **full-context ASR**.

Inside the streaming family, two sub-terms matter:

- **Partial hypothesis** (or **partial token**, **non-final result**) — the model's current best guess for what's been said so far. Mutable; will be replaced as more audio arrives.
- **Final hypothesis** (or **stable token**, **finalized result**) — the model has committed; this part of the transcript will not be revised.
- **Endpointing** — the act of deciding "the user has finished a segment." Usually VAD-driven (silence threshold) or model-internal. The output of endpointing is a *finalization event* that flips partials into finals.

Injection mode (Axis 2)

- **Incremental injection** (also: **streaming injection**, **live commit**, **type-as-you-speak**) — the dictation tool types into the focused window as soon as the recognizer emits tokens (usually finalized ones; some tools also display partials in an overlay without committing them to the underlying app).

- **Utterance-final injection** (also: **commit-on-stop**, **toggle-and-transcribe**, **deferred injection**, **batch injection**, sometimes simply **post-hoc dictation**) — nothing is typed until the user signals "done" (toggle off, release PTT key, long silence). The full transcript is then injected in one operation.
- **Chunked / segment injection** — a middle ground: the tool segments the audio at VAD pauses, transcribes each segment offline, and injects each segment as it completes. From the user's perspective this *looks* live, but under the hood it's batch-on-each-chunk. This is how a lot of "live" Whisper-based tools (e.g. wispr-style frontends) actually work.

The four-cell matrix

	Incremental injection	Utterance-final injection
Streaming ASR	True real-time dictation. Apple Live Dictation, Google Live Caption + dictation, Talon Voice (in dictation mode), Dragon NaturallySpeaking.	Streaming under the hood but batched at the UI — used when the developer wants stable text only. Less common; generally a wasted capability.
Offline ASR	Approximated via VAD chunking — appears live, isn't really. Many Whisper-based tools (Whispering, MacWhisper streaming mode, some `nerd-dictation` configs).	Toggle-and-transcribe. **This is your current Parakeet setup.** Also: most "press hotkey, speak, release, get text" tools built on Whisper-large or Parakeet-TDT-0.6B.

What you're actually using vs what you're describing

- **Your current setup (Parakeet, toggle-mic-off, full transcript appears at once):** offline ASR + utterance-final injection. The widely-used names are **commit-on-stop** or **toggle-and-transcribe**. NVIDIA's offline Parakeet checkpoints (e.g. `parakeet-tdt-0.6b-v2`) are designed for this; they hold state-of-the-art positions on the Open ASR Leaderboard precisely because they get to see the whole utterance before deciding.
- **The "text dumps on the fly" pattern you're describing:** streaming ASR + incremental injection. Marketing usually calls this **live dictation** or **real-time dictation**. From an engineering perspective the distinguishing feature is that the recognizer emits *finalized tokens incrementally* (as opposed to one big finalization at the end), and the injector commits them as they finalize.

Tradeoffs between the two

This is the part that matters for your "ideal UI" spec.

Where commit-on-stop wins

- **Higher accuracy.** Offline models see the whole utterance and can use bidirectional context. Whisper-large and Parakeet-TDT-0.6B both routinely beat their streaming counterparts on WER by 1–4 points absolute. For technical dictation, that gap is significant.
- **Cleaner punctuation and capitalization.** Offline post-processing has the full sentence to work with. Streaming models often emit punctuation late or have to revise it.

- **Simpler injection.** One `wtype`/`xdotool`/SendInput call. No need to manage cursor position drift, no risk of injecting partials into apps that auto-complete or auto-format on each keystroke (think: IDE intellisense, Slack's `@` mentions, terminal readline).
- **No mid-utterance corrections visible to the user.** The user doesn't see the model "change its mind" about earlier words.
- **Plays well with focus capture.** Because injection is a single discrete event, the focus-loss mitigations from `focus-loss-during-dictation` are easier to apply.

Where streaming + incremental injection wins

- **Perceived latency.** Even if total time-to-final-text is similar, watching the words appear feels dramatically faster and more responsive. This is the single biggest reason users prefer it for short utterances.
- **Self-correction loop.** The user can see they were misheard and stop / repeat without waiting for the full utterance to commit.
- **Long-form dictation feels less terrifying.** A 90-second commit-on-stop session producing a wall of text at once is psychologically heavier than the same content appearing as you go.
- **Better fit for hands-free / VAD-gated modes.** When there's no explicit "stop" signal, you have to commit incrementally.

Where the cracks show in each

- **Streaming + incremental injection** breaks badly when partials get committed into apps that react to each keystroke. Typing partials into a Slack message that auto-sends on Enter, or into a terminal where each character could be a command, is dangerous. The defense is to only commit *finals*, never *partials*, into the underlying app — and show partials only in an overlay if at all.
- **Commit-on-stop** breaks when the user expects to be able to start *acting* on early parts of what they said before they're done speaking — and it makes long utterances feel like they're stuck.

A note on Parakeet specifically

NVIDIA Parakeet is not inherently offline. The family includes:

- **Offline checkpoints** (e.g. `parakeet-tdt-0.6b-v2`, `parakeet-rnnt-1.1b`) — designed for batch transcription, what you're using.
- **Streaming checkpoints / configurations** — Parakeet-RNNT and Parakeet-TDT both have streaming variants in the NeMo framework, with configurable chunk sizes (typically 80 ms / 480 ms / 1040 ms lookahead). These give you partial + final hypotheses on a stream.

If you want to move from commit-on-stop to incremental injection while staying on Parakeet, the path is: switch to a streaming-configured Parakeet model in NeMo, consume the partial/final event stream, and only commit finals into the focused window. Expect a small accuracy regression compared to the offline checkpoint and a meaningful engineering jump in the injection layer.

Recommended framing for the workspace's "ideal UI" spec

When cataloguing tools and patterns, treat the two axes as independent and tag each tool with both:

- Recognition mode: `streaming` | `offline` | `chunked-offline` (VAD-segmented offline)
- Injection mode: `incremental-finals` | `incremental-with-partials-overlay` | `utterance-final` | `chunked-segment`

Most user-visible behaviour is explained by the *injection* axis; most accuracy and latency tradeoffs come from the *recognition* axis. Keeping them separate avoids the common confusion of calling any tool with a Whisper backend "not real-time" — when in fact it can feel quite real-time with the right chunking and injection policy.

Caveats / things that can go wrong

- **"Streaming" is overloaded.** Some vendors use it to mean "the API accepts a stream of audio bytes" without committing to incremental output. Check whether the API actually emits partial/final events or just one final result at the end.
- **Whisper is not streaming.** Tools that present "streaming Whisper" are doing chunked-offline with VAD. That's fine, but it's not streaming ASR in the technical sense and inherits offline's inability to revise earlier hypotheses.
- **Partial-vs-final discipline matters.** A surprising number of "live dictation" tools commit partials directly. This produces the unsettling effect of watching words rewrite themselves in your editor and can corrupt undo history.
- **Endpointing thresholds are the hidden UX dial.** Too-aggressive VAD finalization splits sentences mid-thought and loses long-range context (worse punctuation, worse capitalization). Too-lax VAD makes the tool feel laggy. Most "this tool feels wrong" complaints trace back here.
- **Latency is end-to-end, not just model.** Audio capture buffer + VAD lookahead + model RTF + injection round-trip all stack. Streaming ASR with a 1 s lookahead is not faster than offline ASR with a 200 ms RTF on a 3 s utterance.

References

- NVIDIA NeMo ASR documentation — streaming and offline configurations: https://docs.nvidia.com/deeplearning/nemo/user-guide/docs/en/main/asr/intro.html
- Parakeet model card on Hugging Face (offline TDT-0.6B-v2): https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2
- Streaming Conformer-CTC / Conformer-Transducer reference (NeMo): https://docs.nvidia.com/deeplearning/nemo/user-guide/docs/en/main/asr/models.html#streaming
- Open ASR Leaderboard (offline-leaning benchmark): https://huggingface.co/spaces/hf-audio/open_asr_leaderboard
- Moonshine — designed for streaming on-device ASR: https://github.com/usefulsensors/moonshine
- Talon Voice — illustrates incremental injection done well: https://talonvoice.com/
- nerd-dictation — example of offline + utterance-final injection on Linux, using Vosk/Whisper backends: https://github.com/ideasman42/nerd-dictation
- Whispering / WhisperX-based desktop tools — examples of chunked-offline approximating streaming UX.

Topic: `stt-enthusiastic-industry-adoption`

Prompt

Industries that adopted speech-to-text enthusiastically through recognition of its effectiveness

Asked: 14/04/26 **Slug:** `stt-enthusiastic-industry-adoption` **Response:** `ideas/stt-enthusiastic-industry-adoption.md`

The question

If we were to point to one industry which has adopted speech-to-text enthusiastically — not because it was mandated or almost necessary for workflow (as maybe is the case with medical transcription), but even if that was the introduction to it, achieved a second wind through sheer recognition of its effectiveness — what would be some cases we could point to and their levels of adoption?

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Industries that adopted STT enthusiastically through sheer recognition of effectiveness

Question: `questions/stt-enthusiastic-industry-adoption.md` **Written:** 14/04/26 **Stack:** Industry-agnostic — adoption patterns, not a technical stack question.

TL;DR

The cleanest archetypes of STT adoption driven by effectiveness rather than mandate are **podcasting/content editing (Descript)** and **B2B sales intelligence (Gong, Chorus.ai)**. Both started from zero mandated use, hit near-saturation in their respective professional cohorts within a few years, and are widely cited as transformative rather than merely useful. **Journalism/qualitative research** and **AI-forward software development** are strong secondary cases — the latter still early but on a steep curve.

Background

The user's framing distinguishes two adoption modes:

1. **Mandated or near-mandated** — medical transcription, legal depositions, call-center compliance recording, accessibility use. STT was adopted because the workflow effectively required it.
2. **Effectiveness-driven** — practitioners could have kept working without STT but chose it once they recognized it as genuinely better. This is the more interesting signal: it filters out regulatory/ergonomic coercion and reveals where STT *earned* its place.

A "second wind" case is a field where (1) introduced the technology but (2) is what sustained and deepened it. Several industries fit cleanly; a few sit in the borderland.

Strongest cases

Podcasting & content editing — the archetype

Catalyst: Descript (founded 2017, shipped transcript-based editing ~2018–2019). Prior waveform editors (Audition, Audacity, Pro Tools) were fully functional — nobody needed transcript-based editing.

What changed: Editing audio/video by editing the transcript is qualitatively different, not incrementally better. Deleting a word in a transcript deletes the audio. Rearranging paragraphs rearranges the timeline. Filler-word removal went from minutes-per-minute-of-audio to near-instant. Show-note generation became a transcription side-effect.

Adoption level: Near-saturation among indie podcasters and a substantial share of YouTube creators. The category now includes Descript, Riverside, Adobe Podcast (Enhance + transcript editing), Captions, Opus Clip, Submagic. Descript alone reported over a million users by 2022 and has grown since. The behavior — "I edit on the transcript" — is now the default expectation among new entrants.

Why it's the archetype: Zero prior mandate. Pure recognition of effectiveness. Viral adoption through peer demonstration.

B2B sales — the enterprise archetype

Catalyst: Gong (founded 2015) and Chorus.ai (2015, acquired by ZoomInfo 2021). Call recording existed for compliance and QA; bolting on STT + analysis was the genuine innovation.

What changed: Sales leaders discovered they could coach reps systematically, identify deal risk from linguistic signals ("procurement," "legal review"), mine objection patterns, and benchmark talk-time ratios — all from transcribed calls. Revenue teams treated it as ROI-positive within a quarter.

Adoption level: Gong hit a reported $7.25B valuation in 2021 and is effectively standard kit in B2B SaaS sales orgs above ~50 reps. The category (conversation intelligence) is now a line item in sales tech stacks alongside CRM and sales engagement.

Why it counts: No regulator required this. It spread because sales leaders measured the lift and kept paying.

Journalism & qualitative research — the professional-workflow case

Catalyst: Otter.ai (2016), Rev.com's AI offering, Trint (2014). Accuracy crossed a usability threshold around 2018–2020 with transformer-based ASR.

What changed: Reporters who previously paid $1–2/minute for human transcription migrated to automated services at a tenth the cost, with acceptable accuracy for most interviews. Academic qualitative researchers (NVivo, Atlas.ti users) adopted STT-assisted coding for focus groups and interviews. Note that this was partially a cost-driven migration from an existing mandated workflow (transcription was already required for coding and quotation) — but the *enthusiasm* came from the speed and turnaround, not the cost alone.

Adoption level: Otter claims tens of millions of registered users. Rev.com remains the dominant transcription brand. In academic qualitative methods, STT-assisted coding is mainstream rather than niche; the 2010s PhD thesis workflow of hand-transcribing interviews is now unusual.

Edge case: This sits between mandated and effectiveness-driven — transcription was always required, but STT reframed it from a budget item to a turnaround feature.

Software development (AI-forward subset) — the emerging case

Catalyst: Whisper (OpenAI, 2022) drove cost and latency down far enough that mainstream developer dictation tools became viable: Wispr Flow, Superwhisper, Aqua Voice, Handy, MacWhisper, Talon (older, accessibility-origin but now broader).

What changed: Developers working with LLM coding assistants (Claude, Cursor, Copilot chat) discovered that *prompts* are a verbose, prose-heavy form of input where typing is the bottleneck. Dictation is often 3–4× faster than typing for long prompts. No prior mandate — pure effectiveness recognition, and notably a reversal of the historical stereotype that "developers don't dictate."

Adoption level: Still early. Wispr Flow reported strong growth through 2024–2025; Aqua Voice, Superwhisper, and Handy each have active communities. Not yet mainstream — concentrated among AI-forward developers and prompt engineers. Growth curve is steep.

Why it's noteworthy: This is the cohort that previously resisted STT hardest (keyboard-centric, noisy open offices, code syntax doesn't dictate well). The shift happened specifically because LLM prompting changed what developers *type* — from syntax to prose.

Borderline and mandated-origin cases

These don't fit the "pure effectiveness" framing but are worth noting:

- **Legal drafting** — Dragon NaturallySpeaking built a generation of dictation habits in litigation practice. Genuine enthusiasm exists among brief-writers, but the category never had a "second wind" equivalent to Descript — it has been a sustained, quiet loyalty rather than a breakout.
- **Call centers / customer service** — real-time transcription and sentiment analysis are widely deployed but typically mandated top-down for QA, compliance, and coaching. Adoption is high; enthusiasm from frontline agents is mixed.
- **Medical transcription** — the user's explicit counter-example. Ambient clinical documentation (Nuance DAX, Abridge, Suki) is experiencing a second wind specifically around *effectiveness* — physicians report reclaimed time and reduced burnout. Arguably this *is* a second-wind case, with the mandated era (dictation-to-typist workflows) giving way to enthusiastically adopted ambient scribing.
- **Accessibility** — genuine effectiveness and enthusiastic adoption within the community, but the population is defined by necessity, so it doesn't fit the framing.

The clearest answer

If forced to name one: **Descript in podcasting**. It satisfies every criterion — no prior mandate, recognized effectiveness, rapid viral adoption, near-saturation in its cohort, and a genuinely new interaction pattern that couldn't exist without STT.

The enterprise equivalent is **Gong in B2B sales**. The emerging equivalent — worth watching for this research — is **AI-assisted coding with dictation-first prompting**, because it directly maps onto the live-typing UX questions in the rest of this workspace.

References

- Descript: https://www.descript.com
- Gong: https://www.gong.io
- Chorus.ai (ZoomInfo): https://www.zoominfo.com/solutions/conversation-intelligence
- Otter.ai: https://otter.ai
- Rev: https://www.rev.com
- Wispr Flow: https://wisprflow.ai
- Superwhisper: https://superwhisper.com
- Aqua Voice: https://withaqua.com
- Nuance DAX / Dragon Ambient eXperience: https://www.nuance.com/healthcare/ambient-clinical-intelligence.html
- Abridge: https://www.abridge.com

Topic: `vad-for-live-typing`

Prompt

VAD (voice activity detection) for live typing

Asked: 14/04/26 **Slug:** `vad-for-live-typing` **Response:** `ideas/vad-for-live-typing.md`

The question

VAD (voice activity detection) for live typing — to what extent can it be incorporated into live typing? Do ASR models (like Whisper, etc.) do VAD natively without requiring an external VAD model?

And if pairing a separate VAD with an ASR model is needed, what are the typical inference engines that let you run the two in the kind of parallel succession required for live dictation to work?

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

VAD for live typing: what's native, what's bolted on, and how to wire them together

Question: `questions/vad-for-live-typing.md` **Written:** 14/04/26 **Stack:** Linux desktop (Ubuntu 25.10 / KDE / Wayland), local + cloud ASR, Python/Node prototypes feeding text into the focused window.

TL;DR

VAD is a small, cheap classifier that says "voice / not voice" on short audio frames (10–30 ms). It is essential for hands-free live typing because it tells the system **when to start sending audio to the ASR**, **when an utterance has ended**, and **when to commit the final transcript and release the cursor**.

- **Whisper has no real VAD.** It hallucinates on silence and has no concept of "is the user still talking." Anything Whisper-based that behaves nicely (whisper.cpp, faster-whisper, WhisperX, whisper-streaming, whispering) is using an **external VAD** in front of it — almost always **Silero VAD** or **WebRTC VAD**.
- **Streaming ASR services do have native endpointing** (Deepgram, AssemblyAI, Soniox, Speechmatics, Google Cloud STT, NVIDIA Riva). They expose it as `endpointing_ms`, `utterance_end_ms`, `vad_events`, etc. This is genuinely native — the same model emits both partial tokens and an end-of-utterance signal.

- **For local pipelines, the standard recipe is Silero VAD → faster-whisper / NeMo / Vosk**, glued together in Python with `sounddevice` / `pyaudio` for capture and asyncio (or two threads + a queue) to keep VAD running while the ASR processes the previous chunk.

The "parallel succession" is not really parallelism in the GPU sense — it's a **producer/consumer pipeline**: VAD runs continuously on the mic stream and emits "speech start" / "speech end" events; the ASR consumes finalized speech segments. Single-process Python is fine; a worker thread per stage is the typical shape.

Background — why VAD matters specifically for live typing

In a live-typing UI you need to answer four operational questions, in order:

1. **Should I open the mic at all?** (push-to-talk solves this; hands-free needs a wake mechanism, often a hotkey or the VAD itself.)
2. **Has the user actually started speaking?** — needed so the overlay can flip from "listening" to "transcribing" without flicker on background noise.
3. **Has the user paused mid-sentence, or finished?** — the difference between "stabilize the partial and keep the stream open" vs. "commit the transcript, inject it, release focus."
4. **Has the room gone quiet long enough that I should close the mic?** — for battery, privacy indicator, and to stop the streaming-ASR meter running.

Pure ASR doesn't answer (2)–(4) reliably. Confidence scores and partial-token churn are noisy proxies. A dedicated VAD does answer them, cheaply (Silero VAD is ~1 MB and runs on CPU at hundreds of times realtime).

This is why every serious dictation tool — Talon, nerd-dictation, WisprFlow, MacWhisper, SuperWhisper, the Whisper-based Linux tools, even Apple/Google's built-in dictation — has a VAD or endpointer somewhere in the pipeline. The only difference is whether it's a separate component or fused into the ASR.

What "native VAD" actually means in different ASR systems

Whisper (OpenAI, and all derivatives) — no native VAD

Whisper is a **batch encoder–decoder** trained on 30-second windows. It has no notion of voice activity:

- It will happily decode pure silence and emit hallucinated text ("Thank you for watching", "Subscribe to the channel" — these are real, well-documented hallucinations from YouTube training data).
- Its `no_speech_prob` is a per-segment heuristic, not a frame-level VAD. It's unreliable below ~1 second of audio.
- It does not produce partials or endpointing events. There is no streaming protocol.

Every "live Whisper" tool wraps it with an external VAD that:

1. Gates the recorder so silence is never sent to the model.
2. Buffers speech segments and flushes them to Whisper either on a fixed cadence (e.g. every 2 s of speech) or on detected silence (e.g. 700 ms of non-speech).
3. Optionally trims silence from the head/tail of each segment before inference.

Reference implementations: **WhisperX** uses Silero VAD for diarization and segment boundaries; **whisper-streaming** (ufal) uses VAD-gated chunks; **whisper.cpp** has a `--vad-model` flag accepting a Silero ONNX file; **faster-whisper** has a `vad_filter=True` option that bundles Silero internally; **Whispering** and similar desktop tools all do the same.

Streaming cloud ASR — VAD is genuinely native

The big streaming providers ship endpointing as a first-class feature:

- **Deepgram** — `endpointing` (ms of silence before final), `utterance_end_ms`, and a separate `vad_events: true` that emits `SpeechStarted` messages on the WebSocket.
- **AssemblyAI Universal-Streaming** — `end_of_turn_confidence`, `min_end_of_turn_silence_when_confident`, `max_turn_silence`. Tunable per-session.
- **Soniox** — emits `<end>` tokens in the response stream when it detects an utterance boundary.
- **Speechmatics** — `end_of_utterance_silence_trigger` parameter; emits `EndOfUtterance` messages.
- **Google Cloud Speech-to-Text v2** — `voice_activity_events: true` plus `voice_activity_timeout` for begin/end-of-speech callbacks.
- **NVIDIA Riva** (self-hostable) — built-in two-pass endpointing in the streaming recognizer.

Whether this is one model or two internally varies — Deepgram's Nova models are RNN-T / Conformer streaming architectures where the model itself emits an end-of-utterance probability per frame; Google historically ran a separate VAD alongside the recognizer. From the API surface it doesn't matter; you get partial transcripts and endpointing events from a single connection.

Other open-source streaming ASR

- **NVIDIA NeMo** streaming Conformer / FastConformer models include an endpointing head; `nemo.collections.asr` exposes it.
- **Vosk** (Kaldi-based) has its own internal endpointer, exposed via `KaldiRecognizer.AcceptWaveform()` returning `True` on utterance end.
- **Wenet** has CTC-based endpointing built in.
- **Silero** publishes both an STT model line and a VAD, but its STT is not commonly used for English desktop dictation.
- **Moonshine** (Useful Sensors, 2024) — small streaming-friendly model; no native VAD, paired with Silero in their reference apps.

Apple / Microsoft built-ins

- **macOS Dictation** uses Apple's own on-device VAD + ASR (since 13.0 the ASR is fully local for many languages). Endpointing is internal, controlled by the OS, not user-tunable.
- **Windows Voice Access / Speech Recognition** uses an internal endpointer; same story.

Pairing a separate VAD with an ASR — the standard pipelines

For a Linux desktop live-typing tool built on Whisper or any non-endpointing ASR, the canonical pipeline is:

```
mic → resample to 16 kHz mono → ring buffer
        ↓
       VAD (every 30 ms frame)
        ↓
   speech segmenter (start on N speech frames, end on M silence frames)
        ↓
   completed segment → ASR worker → text → injector
```

Components people actually use

VAD models:

- **Silero VAD** (ONNX, ~1.5 MB, MIT licensed) — the de facto default. Frame-level, multilingual, runs on CPU at ~0.5 ms per 30 ms frame. Repo: `snakers4/silero-vad`.
- **WebRTC VAD** — older, signal-processing based (not ML), four aggressiveness levels (0–3). Built into Chromium and exposed via the `webrtcvad` Python binding. Lower quality than Silero, but zero dependencies and microscopically light.
- **pyannote VAD** / **pyannote/segmentation-3.0** — heavier, designed for offline diarization. Overkill for live typing but widely used in research.
- **TEN-VAD** (2024–25) — a newer transformer-based VAD positioned as a Silero alternative; worth tracking but Silero is still the default.

ASR engines paired with the above:

- **faster-whisper** (CTranslate2 backend) — `WhisperModel(...).transcribe(audio, vad_filter=True)` bundles Silero internally; or pass pre-segmented chunks yourself.
- **whisper.cpp** — `./main -m model.bin --vad-model silero-v5.1-small.onnx audio.wav`. Native C++ pipeline, lowest dependency footprint.
- **NVIDIA NeMo** — has its own endpointing; you can also stack Silero in front for tighter UX control.
- **Vosk** — endpointing built in; no external VAD needed.

Inference engines / runtimes

The two stages typically run in the same Python (or C++) process:

- **VAD runtime**: ONNX Runtime (`onnxruntime`) for Silero — CPU-only is more than enough. PyTorch is also fine but heavier to load.
- **ASR runtime**: depends on the model.
  - **CTranslate2** for faster-whisper (CPU/CUDA/Metal).
  - **whisper.cpp** uses GGML (CPU + optional CUDA / Metal / Vulkan / SYCL).
  - **PyTorch** for vanilla openai-whisper, NeMo, pyannote.
  - **TensorRT** / **Triton Inference Server** if you're scaling beyond a single user.

You don't need a special "dual-model orchestrator." VAD is so cheap that it runs in the audio capture thread; the ASR runs in a worker thread or `asyncio` task and consumes finished segments off a queue.

Concrete glue patterns

Pattern A — single-process Python, two threads (most common):

```
# pseudo-shape, not a working snippet
audio_q  = queue.Queue()    # raw 30 ms frames from sounddevice callback
seg_q    = queue.Queue()    # completed speech segments

def vad_loop():
    state = "silence"
    buf = []
    for frame in iter(audio_q.get, None):
        is_speech = silero_vad(frame)              # ~0.5 ms
        # state machine: silence ↔ speech with hangover
        # on speech end, push np.concatenate(buf) to seg_q

def asr_loop():
    for segment in iter(seg_q.get, None):
        text = whisper_model.transcribe(segment)   # tens of ms to seconds
        injector.inject(text)
```

This is exactly the structure used by nerd-dictation, whispering, and most "Whisper as keyboard" tools. The two stages overlap in time naturally — while the ASR is decoding utterance N, the VAD is already detecting utterance N+1.

Pattern B — single-process asyncio (good for streaming ASR):

When the ASR is itself streaming (Deepgram, NeMo streaming, etc.), VAD is often optional or used only as a "should I open the connection" gate. The ASR's own endpointer drives commit timing.

Pattern C — separate processes over a socket:

Useful when the ASR is heavy and you want it pinned to a GPU as a long-running daemon. The audio capture + VAD process sends segments over a Unix socket or gRPC; the ASR daemon returns text. This is how teams self-hosting Riva or Triton typically structure things.

Tradeoffs and tuning

- **Aggressiveness vs. clipping.** Too sensitive a VAD (low silence threshold) clips the start of words ("ext step" instead of "next step"). Too lax (long silence threshold) makes the system feel sluggish — the user pauses, looks at the screen, nothing commits. Typical live-typing values: speech-start after 100–200 ms of speech, speech-end after 500–800 ms of silence.
- **Hangover / pre-roll.** Always keep a 200–500 ms pre-roll buffer of audio *before* VAD says "speech started," so the first phoneme isn't cut. Most VAD wrappers handle this for you.
- **Music and background TTS.** Silero handles music as non-speech reasonably well; WebRTC VAD can fire on it. If the user is on a call or has TTS playing, a desktop tool needs an additional gate (mute the mic, or use system audio routing).
- **Endpointing vs. semantic turn detection.** Silence-based VAD can't tell a thinking pause from a finished sentence. The 2024–25 trend (AssemblyAI's Universal-Streaming, Pipecat's "smart turn") is to layer a small **turn-detection model** on top of VAD that uses partial transcript content + acoustic features to predict end-of-turn. Worth being aware of even if you don't implement it yourself.
- **Wayland injection lag.** None of this matters if `ydotool` / `wtype` injection is the bottleneck. Measure end-to-end latency, not just ASR latency.

Recommendation for this workspace

For a Linux desktop live-typing prototype:

1. **Use Silero VAD** as the front end. ONNX Runtime, ~1 MB model, ~0.5 ms per frame on CPU. Don't bother with WebRTC VAD unless you have a hard size constraint.
2. **Pick one of two ASR paths**:
   - **Local, Whisper-family**: faster-whisper with `vad_filter=True` (bundled Silero). Simplest possible setup.
   - **Streaming, cloud or self-hosted**: Deepgram / AssemblyAI / Riva — let their native endpointing do the work. VAD becomes a hotkey-gating concern, not an audio concern.
3. **Two threads + a queue** is the right concurrency model for the local path. Don't over-engineer with multiprocessing until you have measured a bottleneck.
4. **Treat VAD parameters as part of the UX**, not as model config. Silence threshold, hangover, and pre-roll all directly shape how the dictation feels — they belong in the user-facing settings, not buried in code.

References

- Silero VAD — https://github.com/snakers4/silero-vad
- faster-whisper VAD filter — https://github.com/SYSTRAN/faster-whisper#voice-activity-detection-filter
- whisper.cpp VAD support — https://github.com/ggerganov/whisper.cpp (`--vad-model` flag, README)
- Deepgram endpointing & VAD events — https://developers.deepgram.com/docs/endpointing
- AssemblyAI Universal-Streaming end-of-turn — https://www.assemblyai.com/docs/speech-to-text/universal-streaming
- Google Cloud STT voice activity events — https://cloud.google.com/speech-to-text/v2/docs/voice-activity-events
- Speechmatics end-of-utterance — https://docs.speechmatics.com/features/end-of-utterance
- NVIDIA Riva streaming ASR — https://docs.nvidia.com/deeplearning/riva/user-guide/docs/asr/asr-overview.html
- Vosk endpointing — https://github.com/alphacep/vosk-api
- WebRTC VAD Python — https://github.com/wiseman/py-webrtcvad
- Pipecat smart turn detection — https://github.com/pipecat-ai/pipecat
- Whisper hallucination on silence (paper) — Koenecke et al., "Careless Whisper" (2024)

Topic: `voice-dictation-hotkey-count-tradeoffs`

Prompt

Hotkey count tradeoffs for voice dictation control (single key, macro pads)

Asked: 14/04/26 **Slug:** `voice-dictation-hotkey-count-tradeoffs` **Response:** `ideas/voice-dictation-hotkey-count-tradeoffs.md`

The question

A friction point I'm hitting with Handy using its single-hotkey transcription model: there's no way to release or discard a dictation in flight. The single hotkey is a start/stop toggle, so there's no retake — if I misspeak, the only option is to let it finish and then delete what was inserted.

This is a fundamental limitation of the single-hotkey approach. You could overload it with something like a quick double-tap, but that's not a control I'd have proactively identified as a user.

For documenting macro-pad designs built specifically for voice dictation, the progression seems to be:

- **One key** — start/stop toggle. No retake.
- **Two keys** — separate start and stop triggers.
- **Three keys** — add restart (cancel current, begin a new utterance).
- **Four keys** — add discard without restarting (cancel without immediately starting again).

What's the full design space here, and what are the tradeoffs at each level?

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Hotkey count tradeoffs for voice dictation: from single toggle to a four-key macro pad

Question: `questions/voice-dictation-hotkey-count-tradeoffs.md` **Written:** 14/04/26 **Stack:** Desktop live voice typing, single-hotkey toggle as in Handy. Patterns generalise to any push-to-talk or toggle-dictation tool (nerd-dictation, Talon, Whisper-based custom hotkey wrappers, dedicated macro pads / footswitches).

TL;DR

A single hotkey can only express two states (on / off), so once dictation has started there is no way to abort the in-flight utterance — you must let it finish and then delete the result. As you add physical keys, you unlock a clean progression of capabilities: **1 key → toggle only**, **2 keys → independent start/stop**, **3 keys → add cancel-and-restart**, **4 keys → separate cancel-only from restart**. Beyond four, useful additions are a "commit now" key (force-end without waiting for VAD), a "hold-to-dictate" momentary key, and modal switches (e.g. dictation mode vs. command mode). The right layout depends on whether retakes are rare (one key is fine) or frequent (a foot-pedal or pad with at least three discrete actions pays for itself fast).

Background

Voice dictation control is a state machine: idle → recording → finalising → injected. The user needs to drive transitions on that machine, and the number of transitions you can express is bounded by the number of distinct inputs you provide.

A single hotkey gives you exactly one input event ("hotkey pressed"). Software has to disambiguate what that event *means* purely from current state:

- In *idle*, it means "start recording".
- In *recording*, it means "stop recording and commit".

That's it. There is no event left over to mean "stop recording and **throw it away**". Hence the friction — you can't cancel mid-utterance, you can only let it complete and clean up after.

This isn't a bug in any specific tool; it's an information-theoretic limit of single-input control. The way out is either (a) overload the single key with chorded gestures (double-tap, long-press) or (b) add more keys.

The progression by key count

1 key — toggle (start/stop)

The current Handy model. One press starts recording, the next press stops and commits.

- **Capabilities:** start, stop-and-commit.
- **Missing:** cancel, restart, discard.
- **Mitigations possible without adding keys:**
  - **Long-press** during recording = cancel. Press-and-hold disambiguates from a normal short tap that would commit.
  - **Double-tap** during recording = cancel and restart (or just cancel).
  - **Triple-tap** for a third action — usable but past the point where users will discover or remember it.
- **Why it's not enough:** chorded gestures on one key are invisible. Users only learn them from docs or accident, and they collide with the natural rhythm of just tapping the key. The friction Daniel hit with Handy is the canonical example: there's no "retake" mental model surfaced anywhere in the UI, so the only recovery is delete-and-redictate after the fact.

2 keys — independent start and stop

Now the two events are explicit. Press *Start* to begin, press *Stop* to commit.

- **Adds:** clean separation of intent. Stop is unambiguously “I am done”.
- **Still missing:** cancel.
- **Variant: hold-to-dictate / push-to-talk.** Two-key behaviour collapses into one *momentary* key — hold = recording, release = stop-and-commit. This is the walkie-talkie / Dragon NaturallySpeaking model. It physically maps “speaking” to “holding the key”, which is intuitive but ties up a hand.
- **Real-world example:** most footpedal-based dictation rigs (medical / legal transcription) use one or two pedals because the operator’s hands are on the keyboard; the foot does start/stop.

3 keys — add cancel-and-restart

Third key acts as **abort**: stop the current recording, throw away the audio, and (in the most common interpretation) start a new recording immediately.

- **Adds:** retake. The thing Handy lacks. You misspeak, hit *Restart*, and you’re recording again with no intermediate state.
- **Why “restart” not just “cancel”?** With three keys you have to decide what abort does after it kills the utterance. The most common choice is “cancel + immediately start a new recording” because the most common reason to abort is that you want to retake, and a single key for retake is faster than abort-then-start.
- **Tradeoff:** you’ve consumed your third key on the *combined* action. You can’t separately say “abort and stay idle” without committing the audio.

4 keys — separate cancel from restart

Now abort splits into two distinct keys:

- **Cancel** — stop recording, discard, return to idle.
- **Restart** — stop recording, discard, start a new recording.
- **Adds:** the ability to abandon dictation entirely (you decided not to dictate after all) without immediately re-entering recording. Useful when you got interrupted, when the context changed, when you want to type instead.
- **This is the first layout that covers the full natural state machine** of utterance-level dictation without any chorded gestures.

5+ keys — useful additions

Past four, you start adding controls that aren’t strictly state transitions but improve precision:

- **Commit now** — force endpointing immediately, don’t wait for VAD silence detection. Useful when the dictation tool is hanging on a trailing pause.
- **Pause / resume** — keep the session alive but stop sending audio (e.g. someone walks in to talk to you). Distinct from cancel: nothing is discarded, the partial transcript stays open.
- **Mode switch** — dictation vs. command mode, casual vs. formatted, short-form vs. long-form. Talon uses this heavily.

- **Punctuation / formatting injection** — dedicated keys for “newline”, “period”, “new paragraph”, which would otherwise have to be spoken as commands.
- **Undo last commit** — restore the document to before the last injection, useful when you didn’t catch the misrecognition until after it landed.

Tradeoffs at a glance

Keys	Covers	Misses	Best for
1 (toggle)	start, commit	cancel, retake	Casual users; short utterances; tools with strong correction-after-the-fact UX
1 (hold)	start, commit	cancel, retake	Hands-on-keyboard users who can dedicate a thumb / pinky
2	explicit start + commit	cancel, retake	Footpedal rigs; users who want unambiguous stop
3	+ cancel-and-restart	standalone cancel	The minimum “no-friction retake” layout
4	+ standalone cancel	force-commit, pause	The clean full-coverage layout
5+	+ commit-now / pause / mode	(diminishing returns)	Power users; transcriptionists; voice-coding

Recommendation

If you’re designing a macro pad specifically for voice dictation, **three keys is the inflection point**. One and two are common but force compromises (no retake, or chorded gestures). Three gives you proper retake without overloading any single key. Four removes the last awkward overload (combined cancel-restart). Beyond four is genuinely optional and depends on workflow.

For software defaults on a single-hotkey tool like Handy, the cheapest improvement is to add **long-press = cancel** during recording. It’s discoverable from a one-line tooltip, doesn’t conflict with the normal toggle gesture, and gives users an emergency exit without changing the hardware story. A double-tap or chord for “cancel and restart” is a reasonable next step.

Caveats

- **Discoverability of chorded gestures is poor.** If you overload a single hotkey with long-press / double-tap, surface it explicitly in the UI (status indicator, tooltip on the tray icon). Otherwise users will hit the friction Daniel hit and never know there was an escape hatch.
- **Hold-to-dictate (PTT) is a different ergonomic class** from any toggle layout. It’s not “1 key with extra capabilities”; it’s a different control philosophy. Don’t conflate the two when comparing.

- **Macro pads add a hardware dependency.** Beautiful in theory, but a layout that requires four pedals or a stream-deck-style pad won't move with the user to a laptop. Keep a working keyboard-shortcut fallback for every action.
- **Cancel must be visibly distinct from commit.** A silent cancel that looks identical to a commit (same audio cue, same overlay flash) will train users to second-guess what they pressed. Use distinct sounds or distinct visual states.
- **Foot pedals are the dark horse.** Three-pedal foot switches are the dominant pro-transcription input for a reason: they free both hands for keyboard editing and map cleanly onto start / stop / cancel-restart.

References

- Handy — single-hotkey local dictation tool (the immediate motivating example): https://github.com/cjpais/Handy
- nerd-dictation — toggle-based local Whisper dictation, illustrative of the one-key model: https://github.com/ideasman42/nerd-dictation
- Talon Voice — production voice-control system with rich modal/key-binding model: https://talonvoice.com/
- Dragon NaturallySpeaking footpedal conventions (PowerMic, Olympus, Infinity foot controls) — long-standing examples of 2- and 3-pedal dictation layouts.
- Walkie-talkie / radio PTT (push-to-talk) conventions — the original "hold to transmit" pattern that single-key hold-to-dictate inherits from.

Topic: `whisper-vs-streaming-asr-for-dictation`

Prompt

Why Whisper isn’t ideal for live dictation, and how live STT models rewrite on the fly

Asked: 14/04/26 **Slug:** `whisper-vs-streaming-asr-for-dictation` **Response:** `../ideas/whisper-vs-streaming-asr-for-dictation.md`

The question

Whisper is the gold standard in transcription, but I’ve often heard it observed that it’s not ideal for live dictation, and we commonly see that specific models like Deepgram are actually preferred for this use case.

Let’s explore technically:

1. What makes the difference between a speech-to-text model that’s inherently suitable for live dictation vs one that isn’t?
2. How can live STT models incorporate things like filler-word removal — not getting “ums” and “ahs” — without being audio-multimodal LLMs? How do they achieve that powerful rewriting ability while staying within this specific space?

Output

Generated 2026-04-14 · Model: Claude Opus 4.6

Why Whisper isn’t built for live dictation, and how streaming STT models rewrite on the fly

Question: `../questions/whisper-vs-streaming-asr-for-dictation.md` **Written:** 14/04/26 **Stack:** Desktop live voice typing. Compares Whisper (OpenAI, encoder-decoder offline transformer) against streaming-first commercial ASR (Deepgram Nova, AssemblyAI Universal-Streaming, Soniox, Google Streaming, NVIDIA Riva streaming Conformer/Parakeet, Speechmatics) and the small text-side post-processors that sit downstream of them.

TL;DR

Whisper isn’t a bad transcriber — it’s an *offline* transcriber. Its architecture (a fixed 30-second encoder window plus a fully autoregressive text decoder conditioned on the entire encoded clip) is structurally hostile to emitting incremental, low-latency hypotheses. Live-suited models like Deepgram Nova or Streaming Conformer-Transducers are built around

three design choices Whisper rejects: **causal/limited-lookahead audio encoding**, **monotonic alignment** (RNN-T or CTC instead of cross-attention), and a **training objective that rewards good partials**, not just a good final transcript.

Filler-word removal ("um/uh" cleanup), punctuation, casing, number formatting, and other "rewriting" effects are mostly *not* done by the acoustic model itself and *do not require* an audio-LLM. They come from two cheap places: (1) **transcript curation** — the ASR is trained on labels where fillers are already absent or marked, so the model learns to skip them; and (2) **a small text-only post-processing chain** — typically a punctuation/casing model, an inverse text normalizer (ITN), and a disfluency remover — that runs on the streaming token output. It's a pipeline of specialized sub-models, not a generative LLM doing end-to-end audio comprehension.

Background

"Live dictation" sets a bar that batch transcription doesn't:

- **First-token latency** under ~300 ms from the moment a phoneme lands in the mic.
- **Steady incremental output** — partials that get progressively stabilised, so the user can react to misrecognitions before they're committed.
- **Bounded finalization delay** — the gap between the user finishing a phrase and the text being "locked" should be a few hundred ms, not seconds.
- **Real-time factor (RTF) <<1** on a single audio stream while leaving CPU/GPU headroom for the rest of the desktop.
- **Stable behaviour on short clips** — three-word utterances must work, not just minute-long monologues.

Whisper was not engineered against any of those constraints. It was engineered to maximise WER on long-form, weakly-supervised internet audio. Different optimisation target, different shape of model.

Why Whisper is structurally offline

The Whisper paper and the ubiquitous `openai/whisper` reference implementation make four architectural commitments that each, individually, make true streaming awkward — and together make it nearly impossible without rebuilding the model.

1. Fixed 30-second mel-spectrogram windows

Whisper's encoder ingests an 80-channel log-Mel spectrogram of exactly 30 seconds (3000 frames at 10 ms hop). Shorter audio is *zero-padded* to 30 s. Longer audio is *chunked* into 30 s segments by the inference wrapper. The model has no concept of "more audio coming" — every forward pass sees a full 30-second tensor.

For live dictation that means:

- A 1-second utterance is processed as a 30-second tensor with 29 seconds of padding. You pay near-full compute regardless of utterance length.
- The encoder cannot start producing useful features until enough audio has been buffered (or the buffer is padded with silence, which biases the model toward hallucinating end-of-segment behaviour).
- "Streaming Whisper" tools (faster-whisper, whisper-streaming, WhisperLive, MacWhisper streaming, wispr) work around this by VAD-segmenting the audio into short utter-

ances and running batch inference on each segment — *chunked-offline*, not streaming. They inherit Whisper's inability to revise across chunk boundaries.

2. Bidirectional encoder

The Whisper encoder is a standard transformer encoder with full bidirectional self-attention over those 3000 frames. Every frame attends to every other frame, including future ones. This is great for accuracy — left and right context disambiguate phonemes brilliantly — but it means you cannot meaningfully run the encoder until you have the full window.

A streaming-friendly encoder uses **causal** or **chunked-causal** attention (each frame attends only to past frames and a small lookahead, e.g. 80–480 ms). That's a fundamentally different architecture, not a tunable knob.

3. Autoregressive cross-attention decoder over the whole encoded clip

The Whisper decoder is a transformer that emits text tokens autoregressively, with cross-attention over the *entire encoder output*. The first text token can only be emitted after the encoder has finished. There is no notion of "emit a partial token from the prefix of the audio I've seen so far." Decoding is a single pass over the whole utterance.

Streaming ASR replaces cross-attention with a **monotonic alignment** mechanism — most commonly **RNN-T (RNN Transducer)** or **CTC (Connectionist Temporal Classification)** — that is structurally constrained to emit text in lockstep with audio frames as they arrive.

4. Training objective rewards the *final* transcript

Whisper was trained on 680k hours of weakly-supervised internet audio with one objective: produce the right text for the whole clip. The loss does not penalise late, incoherent, or unstable partials, because partials don't exist in training. Even if you bolted a streaming inference loop onto Whisper, you'd be using a model whose weights have never been optimised for the partial→final stabilisation behaviour that defines a good live experience.

Side-effects users actually feel

These four choices produce predictable failure modes that anyone who has tried "Whisper for live dictation" recognises:

- **Hallucinated endings on short clips.** "Thank you for watching!" or "Subtitles by …" appearing on tiny utterances — the model has been trained on YouTube transcripts and confidently completes patterns when the audio is mostly silence/padding.
- **All-or-nothing latency.** A 5 s utterance produces nothing, then everything. The user can't tell whether the system is working until the result lands.
- **Chunk-boundary artefacts.** Sentences that span a 30 s VAD chunk lose context across the boundary; punctuation and capitalisation suffer.
- **Unstable timestamps.** Whisper's internal timestamp tokens are notoriously off by hundreds of ms; word-level timing is approximated by post-hoc tools (whisperX, stable-ts) that re-align with a separate model.

What live-suited models do differently

The Deepgram, AssemblyAI, Soniox, Google, Speechmatics, and NVIDIA-streaming families differ in implementation but share a recognisable design pattern.

Streaming acoustic encoder

Audio is encoded **causally** or with a small fixed **right-context window** (lookahead). Common choices:

- **Streaming Conformer** with chunked attention — each chunk (e.g. 160 ms) attends to a few past chunks and 0–2 future chunks. Used by NVIDIA Riva, Google, and several open-source streaming Parakeet/Conformer configurations.
- **Causal Conformer / causal transformer** — strictly no future context. Lower latency, slightly higher WER.
- **Hybrid CNN-Transformer stacks** with limited lookahead, popular in custom commercial models (Deepgram has historically described their stack as a custom end-to-end architecture rather than a published academic model).

The encoder is invoked every chunk; new audio frames produce new encoder outputs without re-processing the full history.

Monotonic alignment: RNN-T or streaming CTC

Instead of decoder cross-attention over the whole utterance, the model emits text via:

- **RNN-T (RNN Transducer)** — three networks: an audio encoder, a label-side prediction network (like a small language model), and a joint network that decides "given these audio frames and the tokens I've emitted so far, emit the next token or stay silent." This is the dominant streaming architecture in production. Apple Dictation, Google Live Caption, NVIDIA Parakeet-RNNT, AssemblyAI Universal-Streaming, and Deepgram Nova are all in or near this family.
- **CTC** — emits one token per frame (including a special blank token), then collapses the sequence. Simpler, very low latency, but slightly weaker on context-dependent words.
- **TDT (Token-and-Duration Transducer)** — RNN-T variant that predicts how many frames each token occupies, allowing the model to skip ahead and reduce per-token compute. Used in NVIDIA Parakeet-TDT.

The structural property all of these share: emission is **monotonic** in audio time. The model cannot "go back" and re-decode an earlier region after seeing later audio. This is the price of streaming, and the engineering job is to mitigate it (with limited lookahead, partial-then-final hypotheses, and post-hoc rewriting).

Partial / final hypothesis discipline

Streaming-first APIs expose two event types:

- **Partial / interim hypothesis** — the model's current best guess for the in-progress utterance. Mutable; will be revised as more audio arrives. Useful for "live caption" overlays; dangerous to commit into a focused window. (See `streaming-vs-batch-injection` for the injection-policy implications.)
- **Final hypothesis** — locked. Comes from an **endpointing** decision: the model or a separate VAD says "the speaker has stopped this segment," and the partials in that segment are flipped to final.

Endpointing thresholds are the hidden UX dial. Deepgram, AssemblyAI, and the rest expose them (`endpointing`, `utterance_end_ms`, `vad_events`) precisely because the right value is application-specific. Dictation usually wants more aggressive endpointing than meeting transcription.

Trained for the streaming objective

Streaming models are trained with auxiliary losses that reward stable partials, low emission latency (the time between when a phoneme lands and when its token is emitted), and good behaviour at chunk boundaries. Techniques include FastEmit, alignment-restricted RNN-T, and minimum-latency training. None of these have analogues in Whisper's training pipeline.

Optimised for short utterances

Production streaming systems are engineered around the observation that dictation utterances are typically 1–10 seconds. Per-utterance overhead (model warm-up, padding, beam-search initialisation) is amortised aggressively. Whisper's per-utterance overhead, by contrast, is roughly constant in the 30 s window length.

Summary of the architectural delta

	Whisper	Streaming ASR (Deepgram Nova / AssemblyAI Universal-Streaming / Streaming Conformer-RNNT)
Audio context	Fixed 30 s, bidirectional	Chunked, causal or short lookahead (~80–480 ms)
Text decoder	Autoregressive cross-attention over full encoding	Monotonic alignment (RNN-T / CTC / TDT)
Output cadence	One transcript per 30 s chunk	Partial + final events, every chunk
Training objective	WER on full transcript	WER + emission latency + partial stability
Per-utterance overhead	High and constant	Low, scales with utterance length
Native short-clip behaviour	Poor (hallucinates / pads)	Good
Best WER ceiling	Higher (sees full context)	Lower by 1–4 absolute WER points typically

How filler-word removal works without a multimodal LLM

This is the part that often looks magical and isn't. The "rewriting" power of Deepgram-class systems comes from a *pipeline of small specialised models*, each doing one thing, none of which is a generative audio-LLM. The acoustic model is still a sequence transducer that maps audio frames to text tokens. The text gets cleaned up by what's downstream.

There are four mechanisms in play, and production systems mix them.

Mechanism 1: Train the acoustic model on transcripts that already omit (or tag) fillers

The simplest and most powerful trick. If your training transcripts don't contain "um" and "uh" — or contain them as a special tag that you can choose to suppress — the model learns that those acoustic regions don't correspond to text output.

Several large conversational corpora (Switchboard, Fisher, internal customer-service recordings) are annotated with disfluencies marked explicitly: `[uh]`, `[um]`, `<filler>`. With those labels:

- Train one model on the verbatim transcripts → it emits fillers.
- Train another (or use a flag/conditioning token) on filler-stripped transcripts → it skips them.
- Or: train a single model with a special `<no-filler>` conditioning token that the user supplies at inference time. The model learns to emit fillers when the token is absent and skip them when it's present.

The acoustic side never "understands" what a filler is — it just learns, statistically, that certain acoustic regions (low-energy nasalised vowels surrounded by pauses, with no following content word) tend to map to no output token in the cleaned label distribution. This is the same machinery that lets the model learn to skip silence. Filler is just "vocalised silence" from the model's perspective.

Deepgram exposes this as a `filler_words` parameter (default behaviour suppresses them; set to `true` to keep them). AssemblyAI exposes `disfluencies`. Both names point to a single conditioning bit on a model trained both ways.

This is also, by the way, why **Whisper sometimes drops fillers and sometimes doesn't** — its 680k hours of internet training data contain a mix of verbatim and cleaned transcripts, with no explicit conditioning. The behaviour is uncontrolled.

Mechanism 2: A small text-only post-processing chain

After the acoustic model emits raw tokens, production systems run a chain of tiny specialised text models. None of them is an LLM in the GPT sense — they're typically small transformers (10s of millions of parameters, often distilled), each fine-tuned on one task:

- **Punctuation and casing restoration** — adds commas, periods, question marks, capitalisation. Trained on text pairs (lowercase-no-punct → properly-cased-with-punct).
- **Inverse Text Normalisation (ITN)** — converts spoken forms to written forms: "twenty twenty six" → "2026", "doctor smith" → "Dr. Smith", "at gmail dot com" → "@gmail.com". Often a finite-state transducer (Sparrowhawk, NeMo's text processing module) plus a small neural fallback.
- **Disfluency removal (sometimes called *cleanup*, *smoothing*, or *fluency*)** — a small encoder-only model (often a distilled BERT) trained on the Switchboard disfluency-tagged corpus to delete fillers, false starts, and repetitions: "I think uh I think we should go" → "I think we should go". Repairing false starts is a richer task than dropping "um" and is where the pipeline starts to look like rewriting.
- **Entity formatting** — phone numbers, currency, dates, addresses.

Each of these models is small enough to run inline on the streaming output without breaking latency. The combined effect *looks* like an LLM rewriting your dictation; it's actually four narrow models in series, each doing pattern-matching it was specifically trained for.

Deepgram bundles much of this under `smart_format`. AssemblyAI calls it `format_text` plus `disfluencies`. NVIDIA Riva exposes the components individually.

Mechanism 3: Two-pass / delayed-rewrite

For richer cleanup, some systems run a second pass over each finalised utterance:

- **Pass 1 (real-time):** the streaming transducer emits raw partials and finals into the live caption.
- **Pass 2 (utterance-final, ~100–300 ms after endpointing):** a small seq2seq model (encoder-decoder, often a distilled T5 or BART variant) takes the finalised raw chunk and rewrites it — fluent disfluency removal, sentence restructuring, light grammar repair. The rewritten chunk replaces the raw chunk in the transcript.

This is effectively a small text-to-text rewriting model with a very narrow task definition. It's not an LLM in the conversational sense — it's a domain-specific seq2seq trained on `(disfluent → fluent)` pairs. Total parameter count is typically <500M and inference fits in tens of ms on CPU.

This pattern shows up in dictation products that promise "clean dictation" output (e.g. several enterprise medical/legal dictation systems, Apple's on-device dictation post-iOS 17). When live captions briefly show "um" before it disappears, that's pass 1 → pass 2 in action.

Mechanism 4: Lexical biasing and constrained decoding

Less about filler removal, more about why the rewriting feels accurate: streaming ASR APIs let you bias the decoder toward expected vocabulary (Deepgram's `keywords`, AssemblyAI's `word_boost`, NVIDIA's `boosted_lm_words`). This isn't rewriting per se, but it sharpens the raw transcript so that the downstream cleanup chain has less to fix. Combined with mechanisms 1–3, the user experience is: a clean, well-punctuated, filler-free, domain-aware transcript appearing in near-real-time, with no LLM in the loop.

Why it stays out of "audio-LLM" territory

A true audio-multimodal LLM (Gemini Live, GPT-4o realtime, Qwen2-Audio) does end-to-end audio understanding: it can answer questions about the audio, summarise it, translate, or rewrite freely. That capability comes at a cost — billions of parameters, hundreds of ms of latency per token, and unpredictability in the rewrite (the model may editorialise).

Live STT pipelines stay deliberately on the *narrow* side of this line:

- The acoustic model is trained for transcription, not understanding. It doesn't "know" what it transcribed.
- Each post-processor is trained on a narrow input/output pair, so its behaviour is bounded and predictable.
- Total parameter budget across the pipeline is typically <1B, vs. >100B for a frontier multimodal LLM.
- The pipeline can be audited and individual stages disabled (turn off `smart_format`, get raw tokens).

This is why dictation users prefer the pipeline approach: it produces the exact same clean output every time, doesn't hallucinate beyond what was said, and runs at <300 ms first-

token latency on commodity hardware. The “rewriting” is real, but it’s the predictable rewriting of a small specialist, not the open-ended rewriting of a generalist LLM.

Where this leaves Whisper

Whisper isn’t going away — it’s the right tool when:

- You have full audio in advance (post-hoc transcription, podcast subtitles, meeting recordings).
- You care about WER ceiling more than latency.
- You want one model that handles 99 languages without engineering a pipeline.
- You’re willing to do your own cleanup downstream (you can absolutely run the same punctuation / ITN / disfluency chain on Whisper’s output).

It’s the wrong tool when:

- You need first-token latency under a second.
- You need stable partials that the user can react to.
- You need predictable behaviour on short utterances.
- You need controllable filler suppression (Whisper’s behaviour is uncontrolled).

The cleanest mental model: **Whisper is a transcription model; Deepgram/AssemblyAI/Streaming-Conformer are dictation models.** They’re optimising for different objectives, and the architectural differences follow from that.

Caveats

- **“Deepgram is a single model” is marketing shorthand.** All production streaming STT vendors run pipelines internally. The end-to-end-deep-learning framing refers to the acoustic model, not the full transcript shaped by `smart_format`.
- **Streaming Whisper variants are improving.** `whisper-streaming` (UFAL), `WhisperLive`, and faster-whisper with VAD all narrow the gap on perceived latency. But they remain chunked-offline under the hood and inherit the architectural ceiling described above. They cannot cross the partial-stability / first-token-latency line that true streaming RNN-T crosses.
- **Open-source streaming is catching up.** NVIDIA’s streaming Parakeet/Conformer, Moonshine (designed for streaming on edge), Vosk’s streaming Kaldi models, and several streaming wav2vec2 fine-tunes all exist. Quality is approaching commercial closed models but they generally lack the polished post-processing chain.
- **Latency budgets are end-to-end.** Audio capture buffer + VAD lookahead + model RTF + post-processing chain + injection round-trip all stack. A 200 ms model with 600 ms of buffering doesn’t feel fast.
- **The post-processing chain is where vendor lock-in lives.** Swapping the acoustic model is easy; replicating Deepgram’s `smart_format` quality with open components is non-trivial. NeMo and Riva ship the closest open equivalents.
- **“Multimodal” overloading.** All of these models are technically multimodal in input (audio in, text out). The relevant distinction in this answer is whether the model is an *open-ended audio-LLM* (Gemini Live class) versus a *narrow audio-to-text transducer* (Whisper, Deepgram, etc.). Both consume audio; only the former does flexible reasoning over it.

References

- Radford et al., *Robust Speech Recognition via Large-Scale Weak Supervision* (Whisper paper): https://arxiv.org/abs/2212.04356
- Whisper reference implementation (note the 30 s windowing in `audio.py`): https://github.com/openai/whisper
- Graves, *Sequence Transduction with Recurrent Neural Networks* (RNN-T): https://arxiv.org/abs/1211.3711
- Gulati et al., *Conformer: Convolution-augmented Transformer for Speech Recognition*: https://arxiv.org/abs/2005.08100
- Yu et al., *FastEmit: Low-latency Streaming ASR with Sequence-level Emission Regularization*: https://arxiv.org/abs/2010.11148
- NVIDIA NeMo streaming ASR documentation: https://docs.nvidia.com/deeplearning/nemo/user-guide/docs/en/main/asr/intro.html
- NVIDIA Parakeet-TDT (TDT architecture): https://huggingface.co/nvidia/parakeet-tdt-0.6b-v2
- Moonshine — designed for streaming on-device ASR: https://github.com/usefulsensors/moonshine
- Deepgram API reference (`smart_format`, `filler_words`, `endpointing`): https://developers.deepgram.com/docs/
- AssemblyAI Universal-Streaming announcement: https://www.assemblyai.com/blog/universal-streaming
- Switchboard disfluency annotations (the corpus most disfluency-removal models are trained on): https://catalog.ldc.upenn.edu/LDC99T42
- NVIDIA NeMo text processing (ITN / punctuation / casing): https://github.com/NVIDIA/NeMo-text-processing
- Whisper-Streaming (UFAL) — example of chunked-offline approximating streaming UX: https://github.com/ufal/whisper_streaming
- Related in this workspace: `streaming-vs-batch-injection` — separates the recognition-mode axis (this guide) from the injection-mode axis.